北京大学内部教材
未经本校允许，不得翻印

智能硬件应用实验



北京大学信息科学技术学院

2026 年 9 月

前言

“智能硬件应用实验” 是一门面向低年级本科生的基础课程，目的是让同学们可以在大学学习的早期阶段了解人工智能的基本概念和应用领域，同时了解计算机硬件设备的简单工作原理。课程内容分成三个主线：Python 语言编程、计算机硬件控制和人工智能算法。

本讲义通过单元实验把相关的知识串联起来，希望学生通过实验可以对本课程三条主线的知识有更加深入的了解。讲义仅对实验本身需要了解的内容进行了简要的介绍，通过实验步骤帮助同学们完成实验。同学们如果需要深入了解相关知识还需要参考讲义中提供的其它参考书籍、网络资料或者在线论文。

每个实验内容具有一定的独立性，学生在实验之前必须做好预习，了解实验中所使用硬件的基本原理，掌握相关算法的实现方案，这样才能在实验的过程中充分利用实验室的资源，在有限的实验时间内完成实验，并在学有余力的情况下对相关算法进行深入研究。如果在做实验的过程中遇到困难，也要及时和老师或助教沟通，以免耽误太多时间，不能按时完成实验。

课程的最后还需要同学们完成一个综合项目，利用现有的硬件资源，实现一个有一定趣味性、创新性的智能应用。项目独立完成，可以参考网络上的现有资源，可以使用开源的代码和程序，但必须有自己设计的部分，最后的作品应可以演示。

目　录

第一章　智能硬件课程简介 **1**
1.1　硬件平台 1
1.2　操作系统环境 3
1.3　Python 编程语言简介 5
1.4　人工智能简介 14

第二章　硬件开发板介绍 **16**
2.1　连接开发板 16
2.2　扩展 IO 接口 17
2.3　Pioneer 扩展板 22

第三章　简单游戏策略 **36**
3.1　电脑游戏中的智能 36
3.2　Python 语言控制硬件 37
3.3　动手实验：猜拳游戏 40
3.4　思考题 41

第四章　聚类算法 **42**
4.1　聚类算法简介 42
4.2　用到的 Python 库 42
4.3　动手实验：模式识别 43
4.4　思考题 44

第五章　OpenCV 基础 **46**
5.1　OpenCV 简介 46
5.2　图像的存储与显示 46
5.3　图像的基本处理 47
5.4　图像形状检测 49
5.5　动手实验：图像处理及图形识别 52
5.6　思考题 54

第六章　支持向量机分类器 **55**
6.1　支持向量机 55
6.2　用到的 Python 库 58
6.3　动手实验：手写数字识别 60
6.4　思考题 61

第七章 构建专家系统 **62**
7.1 专家系统 . . . 62
7.2 开发板的舵机控制 . . . 68
7.3 动手实验:动物识别专家 . . . 72
7.4 思考题 . . . 73

第八章 OpenCV 进阶 **74**
8.1 图片的统计特性 . . . 74
8.2 人脸检测算法基本原理 . . . 75
8.3 运动物体的追踪 . . . 78
8.4 动手实验:视频处理及人脸隐私保护 . . . 80
8.5 思考题 . . . 81

第九章 人工神经网络 **82**
9.1 神经网络 . . . 82
9.2 从线性回归到 MLP . . . 82
9.3 PyTorch 简介 . . . 84
9.4 卷积神经网络(CNN)模型 . . . 86
9.5 动手实验:通过手势识别实时控制发光二极管亮度 . . . 88
9.6 思考题 . . . 95

第十章 语音识别和处理入门 **96**
10.1 开发板音频输入输出方法 . . . 96
10.2 HMM-GMM 模型介绍 . . . 97
10.3 动手实验:语音控制系统 . . . 101
10.4 思考题 . . . 103

第十一章 自然语言处理入门 **104**
11.1 词向量的编码与处理方法 . . . 104
11.2 Twitter 文本情感分析 . . . 108
11.3 未名教学二号集群使用说明 . . . 111
11.4 动手实验:文本的情感分析 . . . 111

第十二章 大语言模型简介 **113**
12.1 Transformer 模型 . . . 113
12.2 大语言模型 . . . 114
12.3 动手实验:调用大语言模型 . . . 117

第十三章 综合实验 **121**
13.1 实验目的 . . . 121
13.2 实验内容 . . . 121
13.3 注意事项 . . . 126

第一章　智能硬件课程简介

随着人工智能（Artifical Intelligence，缩写为 AI）相关技术的不断发展，AI 已经深入到了社会生活的方方面面。本书将 AI 技术与电子设备硬件相结合，通过“智能硬件”这一概念，把智能算法通过真实的硬件平台实现。读者在学习智能算法的同时还能了解微处理器系统的基本知识，掌握使用不同的硬件模块实现真实的智能设备的方法。

1.1　硬件平台

我们探讨的 AI 技术大部分只是一种算法，可以在通用计算机上进行研究和验证。如果要把相关技术实现为产品，就必须使用更加经济的硬件平台。这些硬件平台必须具有一定的运算能力，可以运行特定的算法，而实现算法最高效的电子器件就是微处理器，部分硬件平台还包括专门用来加速 AI 相关运算的协处理器。本书将使用华为公司的 Atlas 200I DK A2 作为主要的开发硬件，完成全部算法的实现。

1.1.1　Atlas 200I DK A2 开发者套件简介

Atlas 200I DK A2 开发者套件是华为技术有限公司推出的高性能 AI 开发平台，外观尺寸为 139mm × 136mm × 46mm，重量仅 330g，却集成了完整的昇腾 310 系列 AI 处理器硬件系统。该套件可提供 8TOPS INT8 的计算能力，支持图像、视频等多种数据分析与推理计算，广泛应用于教育、机器人、无人机等场景。开发者套件外观如图1.1所示。

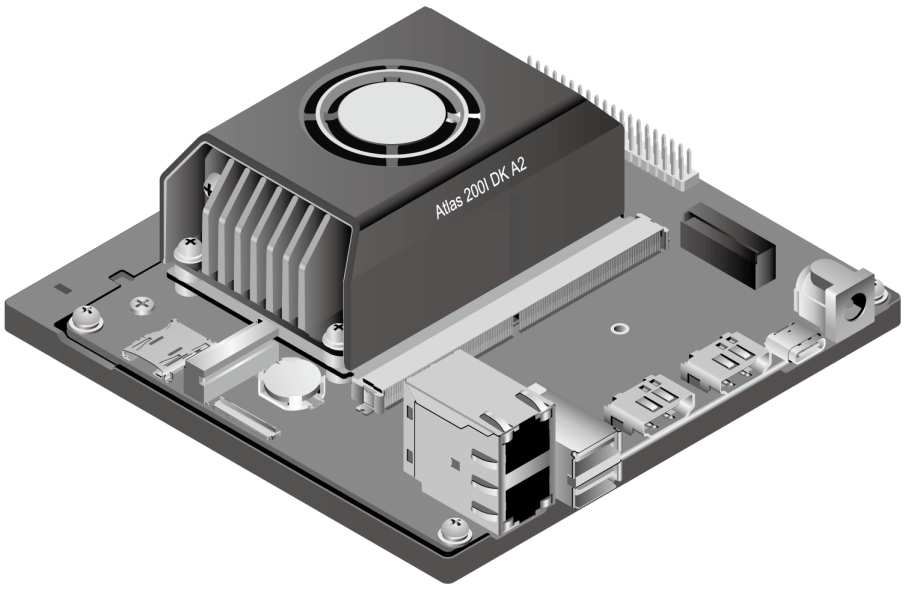


图 1.1: Atlas 200I DK A2 外观

Atlas 200I DK A2 开发者套件发布于 2023 年，搭载以下核心硬件配置：

- 昇腾 310 系列 AI 处理器：包含 1 个 DaVinciV300 AI core（主频 500MHz）和 4 个 TAISHANV200M 处理器核（主频 1.0GHz）
- 内存：4GB LPDDR4X（3200Mbps，支持 ECC）
- 存储：支持 Micro SD 卡（SD 3.0 标准）和 M.2 Key M 连接器（可扩展 NVMe SSD）
- 视频编解码：支持 H.264/H.265 4K 硬件编解码

该套件提供丰富的外设接口，包括：

- 2 个千兆以太网口
- 2 个 USB 3.0 Type-A 接口和 1 个 Type-C 接口
- 2 个 HDMI 接口（HDMI0 支持图形桌面）
- MIPI-CSI/MIPI-DSI 接口用于摄像头和显示连接
- 40Pin 扩展接口用于 GPIO 控制

开发者套件采用 12V 供电，典型功耗为 24W。为简化开发流程，华为提供了预置 Ubuntu 或 openEuler 操作系统的镜像，包含 NPU 驱动、CANN 计算架构、MindX SDK 及示例代码，用户只需通过一键制卡工具将镜像烧录到 SD 卡即可快速启动开发。

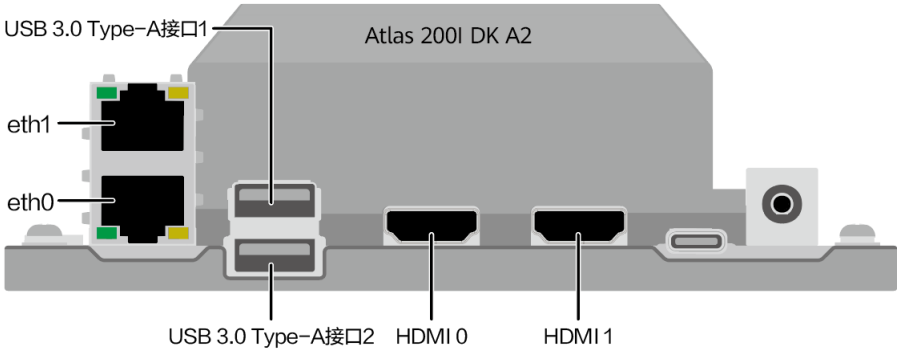


图 1.2: Atlas 200I DK A2 接口配置

开发者可通过以下方式使用该套件：

- 直接连接显示器和键鼠：通过 HDMI0 接口输出图形界面，USB 接口连接输入设备
- 网络访问：通过以太网口（eth1）或无线网络（需扩展 M.2 Key E 无线模块）使用 SSH 协议远程登录
- 开发工具链：使用华为提供的 MindStudio 集成开发环境和 MindX SDK 简化 AI 应用开发

详细的套件资料可以在华为公司网站获取[1]。

[1] https://www.hiascend.com/document/detail/zh/Atlas200IDKA2DeveloperKit/

1.2 操作系统环境

本书中的操作和示例代码只在 Ubuntu 操作系统上进行了测试。它是基于 Debian 的 Linux 发行版，专门为开发平台的硬件所优化。Linux 操作系统内核是 1991 年由芬兰学生 Linux Torvalds 首先公开发布的，它的发行遵循 GPL（GNU general public license）协议，在 Internet 上不断被发展和完善。由于 Linux 的开放源代码的特性，很多优秀的程序员加入到 Linux 的开发行列，使得 Linux 的发展非常迅速，成为今天无论在服务器平台还是在嵌入式平台都非常具有竞争力的操作系统。

1.2.1 Linux 命令行简介

登录到开发板操作系统中之后可以使用图形界面完成基本的文件操作和环境设置，但要更加灵活的使用 Linux 操作系统，或者需要通过 SSH 协议远程登陆开发板，就需要掌握 Linux 的常用命令行命令。在图形界面下可以打开模拟终端获得命令行操作界面，这里列举了一些命令的功能（方括号中内容为命令参数），具体的使用方法还需要通过在线帮助或其它资料来熟悉。

1. 文件操作

   - 更改当前目录：cd [dir]
   - 创建目录：mkdir [dir]
   - 删除目录：rmdir [dir]
   - 列出当前目录文件：ls
   - 将 file1 复制到 file2：cp file1 file2
   - 删除文件：rm [file]
   - 显示文本文件内容：more [file]

2. 网络操作

   - 查看 IP 地址：ip addr
   - 测试网络连通性：ping [IP]
   - 通过 ssh 协议复制文件：scp

3. 其它命令

   - 显示磁盘占用情况：df
   - 显示当前日期和时间：date
   - 显示命令的说明信息：man [command]
   - 安装软件 soft：apt-get install [soft]
   - 退出当前会话：exit

想熟悉命令行的使用就必须多尝试，充分理解命令的精确含义，许多命令都具有 “`--help`” 参数用来显示简单的帮助信息，或者使用`man`命令来查找手册。

1.2.2 Bash 使用技巧

在 Linux 系统中，响应用户输入指令的程序被称为 shell。比较常用的 shell 有 csh、zsh 和 bash。由于多数发行版的默认 shell 都是 bash，这里就简单介绍一下它的操作特点。

首先是命令补全功能，它可以大大提高用户在命令行工作的效率。当输入一个比较长的命令的时候，用户可以只输入这个命令的开头几个字母，然后按 `<Tab>` 键。如果当前系统可以搜索到的命令中以这几个字母开头的只有一个，系统就会自动把这个命令补全；否则再次输入 `<Tab>` 键可以显示出所有满足条件的命令，用户可以继续输入字母直到满足的命令只有一个的时候再按 `<Tab>` 键补全。

命令补全功能还适用于目录名和文件名的补全，在其他软件的辅助下还可以支持命令行参数的补全。例如当前目录中只有一个目录，用户可以如下输入将目录名补全。

```
$ cd <Tab>
```

另外一个功能是命令历史记录，bash 会自动记录用户输入的所有命令，用 history 命令可以查看输入命令的历史。默认情况下，bash 会记录最近输入的 500 条命令。在命令提示符下，按 “上” 方向键可以调出以前输入的命令，也可以用 “下” 方向键回到之后输入的命令。使用 `Ctrl+r` 按键组合还可以对历史进行查找，可以根据接下来输入的内容在历史记录中寻找最佳的匹配。

1.2.3 编辑器使用

开发板自带的 Python 编辑和调试器是 Thonny，大多数情况可以满足编程的要求，这里另外还要介绍的是 Linux 系统自带的 vi 编辑器。

vi 是所有 Linux 发行版都提供的一个文本编辑软件。但 vi 和 Windows 下的任何其他编辑器都不类似，初次接触很可能完全无法了解它的用法。但在某些特殊环境，如嵌入式系统中，也许 vi 就是唯一的选择。另外，vi 是所有 Linux 和 UNIX 都包括的命令，掌握了它的用法就可以在所有 Linux 系统中进行编辑了。

vim 是 vi 的 “超集”，它提供更加丰富的功能，更加适合编程的使用。扩展的功能包括语法高亮、多级撤销、可视选择等。大多数 Linux 发行版提供的都是 vim。vim 的另一个扩展是 gvim，它提供了图形化的工作前端，可以使用鼠标和操作菜单。执行 `vi file_name` 进入 vi 的界面。vi 是一个有 “模式” 的编辑器，它具有 “命令”、“命令行” 和 “编辑” 三种主要模式。刚执行的 vi 处于命令状态。按 “i” 键进入插入状态就可以进行文字编辑了。编辑完成后按 “Esc” 键则可以返回命令状态，在命令状态按 “:” 键进入命令行状态，在命令行状态输入 `wq<`回车`>`（write and quit）就可以存盘退出。

vi 的命令状态主要是通过按键完成特定的功能，例如：

1. x：删除光标位置的字符
2. dd（连续按两次 d 键）：是删除并保存当前行到缓冲区
3. p：将缓冲区中内容输出到光标位置。
4. yy: 保存当前行到缓冲区
5. u：取消上一次操作
6. h，l，j，k：左、右、下、上移动光标

上面提到的命令都要注意区分大小写，它们前面还可以加数字前缀，例如 `20j` 就是下移 20 行光标。

在命令状态按 “:” 进入命令行状态，输入的 “:” 在窗口的最低端显示，如下面的例子所示。删除这个输入的冒号可以退出命令行状态。这个状态用来输入比较复杂的命令，例如查找替换：

```
:s/One/1/g
```

s 命令代表替换。第一个 “/” 号后面的是查找内容，第二个 “/” 后面是替换内容，第三个 “/” 后面是一个标志，g 代表替换本行的所有目标。

vi 的查找命令支持正则表达式，例如使用 “*” 号进行模式匹配，熟练掌握正则表达式可以完成复杂的编辑功能。

在命令行状态的一个重要功能就是帮助，使用

```
:help :s
```

就可以查看上面介绍的 s 命令的说明。在查看帮助的时候，如果有其他条目的链接，则可以按 “Ctrl+]” 组合键跳转过去，然后按 “Ctrl+T” 组合键返回。

到目前为止已经介绍过好多 “求助” 的方法了。主要目的是告诉大家如何利用 Linux 来自己帮助自己，这是加快掌握 Linux 的捷径。因为没有任何一本书可以包揽 Linux 的所有功能和用法，只有在遇到难题的时候学会如何自己解决，才能真正掌握 Linux 的精髓。

1.3 Python 编程语言简介

Python 是一种通用、开源的高级编程语言，由荷兰计算机科学家 Guido van Rossum 于 1989 年圣诞节期间开始开发，1991 年发布首个公开版本。语言名称源自 Guido 喜爱的喜剧团体 Monty Python，这也体现了 Python 社区幽默、开放的文化特质。

作为当前最流行的编程语言之一（TIOBE 和 Stack Overflow 开发者调查长期排名前三），Python 具有以下核心特征：

- **多范式支持**：全面支持面向对象、函数式和过程式编程范式
- **动态类型系统**：变量类型在运行时确定，无需显式声明
- **强类型检查**：类型错误会在运行时被严格捕获
- **跨平台性**：基于解释器实现” 一次编写，到处运行” 的特性
- **丰富的标准库**：内置 200+ 个标准模块，覆盖网络、文件系统、数据处理等常见需求

Python 的典型应用领域包括：

- 科学计算与数据分析（NumPy, Pandas, SciPy）
- 人工智能与机器学习（TensorFlow, PyTorch）
- Web 开发（Django, Flask）
- 自动化运维与 DevOps
- 教育领域（因其语法简洁性成为编程入门首选语言）

语言的主要优势在于其清晰的语法结构和丰富的第三方库生态（PyPI 仓库包含超过 40 万个包）。通过严格的缩进规则和接近自然语言的表达方式，Python 代码具有极佳的可读性。其性能劣势（相比 C/Java 等编译型语言）可通过 C 扩展、JIT 编译（如 PyPy）等方式部分弥补。

1.3.1 Python 运行环境

Python 是一种脚本语言，需要在解释器的帮助下运行。如果采用 ssh 的方式登录到开发板，直接运行 python 或者 python3 就可以打开解释器的交互环境。如果是采用 VNC 登录或者直接操作开发板，需要首先打开一个模拟终端环境。开发板自带的 Thonny 或 Geany 也都是比较适合新手使用的集成开发工具。

在交互模式的提示符 >>> 下，直接输入代码，按回车，就可以立刻得到代码执行结果，例如

```
1 >>> 100+200
2 300
```

如果要让 Python 打印出指定的文字，可以用 print() 函数，然后把希望打印的文字用单引号或者双引号括起来。

```
1 >>> print("hello, world!")
2 hello, world!
```

在 Python 交互模式下输入 exit() 并回车（或者按 Ctrl+D），就退出了 Python 交互模式，并回到终端命令行。

Python 交互模式的代码是输入一行，执行一行。而命令行下直接运行.py 文件是一次性执行该文件内的所有代码。如

```
$ python3 hello.py
Hello, world!
```

Python 交互模式主要是为了调试 Python 代码，也便于初学者学习。在 Python 的交互式命令行写程序，优点是立刻就能得到结果，缺点是没法保存。因此实际编程的时候通常是采用文本编辑器编写 Python 代码，编写完成后保存为一个文件，程序就可以反复执行了。Python 程序的源代码是纯文本文件，通过缩进来标识不同的语法层次关系，而 “#” 号后面的内容为注释，可用来对程序进行解释，提高代码的可读性。

1.3.2 数据类型

Python 语言支持的数据类型包括整数、浮点数、字符串、布尔值以及列表等。Python 可以处理任意大小的整数，当然包括负整数，在程序中的表示方法和数学上的写法一模一样，例如：1，100，-8080，0 等。计算机由于使用二进制，所以，有时候用十六进制表示整数比较方便，十六进制用 0x 前缀和 0-9，a-f 表示，例如：0xff00，0xa5b4c3d2 等。

浮点数可以用数学写法，如 1.23，3.14，-9.01 等。但是对于很大或很小的浮点数，就必须用科学计数法表示，把 10 用 e 替代，1.23×10^9 就是 1.23e9，或者 12.3e8，0.000012 可以写成 1.2e-5 等。

字符串是以单引号或双引号括起来的任意文本，比如’abc’，”xyz” 等等。需要注意的是，单引号或双引号本身只是一种表示方式，不是字符串的一部分，因此，字符串’abc’ 只有 a ，b ，c 这 3 个字符。如果单引号本身也是一个字符，那就可以用双引括起来，比如”I’m OK” 包含的字符是 I ，’ ，m ，空格，O ，K 这 6 个字符。

布尔值和布尔代数的表示完全一致，一个布尔值只有 True、False 两种值，要么是 True，要么是 False，在 Python 中，可以直接用 True、False 表示布尔值。

None 是 Python 中的特殊空值对象，用于表示缺失或无意义的状态。典型应用场景：

1. 作为函数的默认返回值
2. 表示可选参数的缺省值
3. 初始化变量时的占位符
4. 作为哨兵值标记特殊状态

注意事项：判断变量是否为 None 时应使用 is 运算符而非==

```
>>> var = None
>>> var is None # 正确方式
True
>>> var == None # 应避免
True
```

1.3.3 变量

变量是指向各种类型值的名字。在 Python 中，变量的使用环境非常宽松，没有明显的变量声明，等号=是赋值语句，可以把任意数据类型赋值给变量。同一个变量可以反复赋值，而且可以是不同类型的变量。这种变量本身类型不固定的语言称之为动态语言，与之对应的是静态语言。静态语言在定义变量时必须指定变量类型，如果赋值的时候类型不匹配，就会报错。

在 Python 中，如果你不能确定变量或数据的类型，就用解释器内置的函数 type 确认。例如：

```
>>> a = 'name'
type(a)
<class 'str'>
>>> classmates = ['Michael', 'Bob', 'Tracy']
>>> type(classmates)
<class 'list'>
```

1.3.4 元组、列表、字典和集合

在程序设计的时候经常需要把多个元素包装到一起来使用，例如纸张的规格、训练的多组数据等。这种可以容纳多个变量的数据结构叫做容器，Python 有多个容器类别，分别适合不同的应用场景。

元组（tuple）是最简单的容器，所有元组元素使用括号包围，例如：

```
BrickSize = (15, 23, 56)
Classmates = ('Bill', 'Carl')
```

元组一旦建立就不可以改变，里面的元素也可以是不同类型，常用的元组操作可以参考下面示例：

```
len(BrickSize)       # 获取元组长度
print(Classmates[1]) # 通过索引获取元素（第一个元素的索引为 0）
BrickSize.index(15)  # 查找元素的索引号
```

需要注意的是如果元组只有一个元素，那么定义的时候要在这个元素后面加上逗号，如 `a = (1,)`。

列表（list）是 Python 内置的另外一种容器类型，他和元组一样是一种有序的集合，但可以随时添加和删除其中的元素。列表中包含的元素可以具有相同的数据类型，也可以具有不同数据类型，例如：

```
Names = ['Michael', 'Sarah', 'Tracy']
Objects = ['Apple', 123, True]
Sizes = [[10, 20], [15, 25], [20, 35]]  # 列表当然也可以嵌套
```

列表在程序中经常使用，用法也十分灵活，下面是一些应用的例子：

```
Names.append('Sarah')     # 在末尾增加元素
Objects.insert(2, 456)    # 在索引 2 的位置插入新元素
Objects.pop(2)            # 删除索引 2 的元素（不加参数缺省删除末尾元素）
Sizes.reverse()           # 元素顺序颠倒
Objects.clear()           # 删除全部元素
Sizes.sort()              # 对元素进行排序
Names.remove('Sarah')     # 根据内容删除元素（只删除第一个匹配）
```

注意列表在复制的时候只是新建一个引用，所代表的内容是相同的，如果需要完整复制（深复制）一个列表元素，要用 copy 方法，例如：

```
>>> Names = ['Michael', 'Sarah', 'Tracy']
>>> K=Names                 # 浅复制
>>> Q=Names.copy()          # 深复制
>>> Names.append('Violet')# 修改原始变量的值
>>> K                       # K 的值同时被改变
['Michael', 'Sarah', 'Tracy', 'Violet']
>>> Q                       # Q 的值没有变化
['Michael', 'Sarah', 'Tracy']
```

和列表类似，字典（dict）也是可以保存多个元素的变量类型，其每个元素都是一个冒号分隔的键值对，例如：

```
>>> dict = {'Alice': '1234', 'Bob': '5678', 'Charlie': '90'}
```

可以通过“键”对“值”进行查询，也可以对“键”进行赋值，例如：

```
>>> dict.keys()          # 返回全部的键列表
dict_keys(['Alice', 'Bob', 'Charlie'])
>>> dict.values()        # 返回全部的值列表
dict_values(['1234', '5678', '90'])
>>> print(dict['Alice'])
1234
>>> dict['John']='45'    # 赋值的同时新建了一个键值对
```

字典也包含一些内置的函数，如 `clear()` 清除字典内容；`key in dict` 查询键是否存在；`len(dict)` 查询字典元素个数等。

集合（set）也与列表类似，但是不能存储重复的数据，并且是没有存放顺序的区别，集合可以通过 `add()` 增加元素，也可以通过 `remove()` 删除元素。比较特别的，集合可以求交集和并集，例如：

```
>>> Names1 = {'Michael', 'Sarah', 'Tracy'}
>>> Names2 = {'Sarah', 'Violet', 'Alice'}
>>> Names1.intersection(Names2)          # 交集
{'Sarah'}
>>> Names1.union(Names2)                 # 并集
{'Sarah', 'Alice', 'Tracy', 'Violet', 'Michael'}
>>> Names1.difference(Names2)            # 集合区别
{'Michael', 'Tracy'}
```

1.3.5 序列的下标访问

Python 中的字符串、元组和列表都是有序结构，都可以用下标进行访问，可以统称为序列。序列的下标访问主要有下面几种方式：

1. `seq[index]` 获得索引所在位置元素
2. `seq[index1:index2(:stride)]` 获得索引从 index1 到 index2 的全部元素，stride 是步长，缺省为 1，index1 空缺表示从头开始，index2 空缺表示到最后一个元素，`seq[:]` 表示全部元素。
3. `seq1 + seq2` 连接两个序列
4. `seq * expr` 序列元素重复 expr 遍

当索引值为负时，它表示从序列最后一个元素开始计数，例如，`seq[-1]` 可以获得序列的最后一个元素。

1.3.6 基本输入输出

在 Python 中，用 print() 函数在括号中加上字符串，就可以向屏幕上输出指定的文字。例如：

```
>>> print('hello, world')
hello, world
>>> name = 'Tom'
>>> print(name)
Tom
```

print() 函数也可以接受多个字符串，用逗号“,”隔开，就可以连成一串输出，print() 函数会依次打印每个字符串，遇到逗号“,”会输出一个空格，例如，

```
>>> print('The brown fox', 'jumps over', 'the lazy dog.')
The brown fox jumps over the lazy dog.
>>> print('100 + 200 =', 100 + 200)
100 + 200 = 300
```

在 Python 中，可以利用 input() 函数完成基本输入。利用 input() 函数可以让用户输入字符串，并存放到指定的变量里。例如，

```
>>> name = input()
Michael
```

当用户输入 name = input() 并按下回车后，Python 交互式命令行就在等待输入了，用户可以输入任意字符，然后按回车后完成输入。另外，可以在 input() 函数加入提示信息。例如，

```
>>> name = input('please enter your name: ')
please enter your name: Michael
>>> print('hello,', name)
hello, Michael
```

1.3.7 条件判断

在 Python 程序中，用 if 语句实现条件判断。例如，

```
total = query() # 查询库存
if total == 0:
    print('Out of stock')
    order(100)
```

根据 Python 的缩进规则，如果 if 语句判断是 True，就执行缩进的两行语执行，否则，什么也不做。也可以给 if 语句添加一个 else 语句，如果 if 判断是 False，不执行 if 后的语句，而是执行 else 后的语句。需要注意的是 if 语句和 else 语句后面要添加冒号。例如：

```
food = input('What do you like to eat? ')
if food == 'Apple':
    print('Please enjoy')
else:
    print("Sorry, We don't have any")
```

另外，可利用 elif 语句添加多个条件判断，elif 是 else if 的缩写，可以有多个 elif 语句。例如：

```
age = int(input('Enter your age: '))
if age >= 18:
    print('You are an adult!')
elif age >= 6:
    print('You are a teenager')
else:
    print('You are a kid')
```

1.3.8 循环

Python 的循环有两种，一种是 for...in 循环，另一种是 while 循环。for...in 循环可以依次将列表中的每个元素迭代出来。例如：

```
for i in [0,1, 2, 3, 4, 5, 6, 7, 8, 9]:
    print(i)
```

可以利用 Python 提供的 range() 函数控制 for 循环，range() 函数可以产生一个整数序列。上面利用 for 循环的求和代码可用 range() 函数进行简化。

```
for i in range(0,10): # 从 0 循环至 9
    print(i)
```

另外，range() 函数产生的整数序列可以指定步进值，例如：

```
for i in range(0,10,2): # 从 0 循环至 9, 步进值为 2
    print(i)
```

Python 的 while 循环只要条件满足就不断循环，条件不满足时退出循环。例如：

```
n = 1
while n <= 100:
    print(n)
    n = n + 1
```

对于循环语句，可以通过 break 语句退出循环或通过 continue 语句跳过当前循环。例如：

```
n = 1
while n <= 100:
    if n > 10: # 当 n = 11 时, 条件满足, 执行 break 语句
        break # 结束当前循环
    print(n)
    n = n + 1
print('END')
n = 0
while n < 10 :
    n = n + 1
    if n % 2 == 0 : # 如果 n 是偶数, 执行 continue 语句
        continue # 跳过当前循环
    print (n)
```

1.3.9 Python 的文件操作

在操作文件之前，需要先把文件打开，操作之后还需要把文件关闭，与这两个操作对应的函数是 open 和 close。open 操作返回一个文件类，后续对这个文件的操作都通过这个类来进行。open 函数的定义如下：

```
open(name[, mode[, buffering]])
```

其中 name 参数为文件的名称，mode 为文件的打开方式，buffering 表示缓冲区的大小。最常用的 mode 参数为”r” 和”w”，分别代表读和写的方式，当用”w” 的方式打开时，如果文件已经存在，则里面的内容会被清空。还有一种”a” 方式用来在文件后面添加新的内容。另外”r+”、”w+” 和”a+” 都以读写方式打开文件，但”w+” 会清空文件，”a+” 会将文件指针放到文件末尾。

文件操作包括文件的读和写，对于文本文件，可以认为其内容是一个字符串数组，每个字符串代表文件的一行，这样就可以利用下面的示例代码将整个文件读入。

```
with open('file.txt', 'r') as f:
    content = f.read()
```

文件主要使用 write 函数将数据写入，简单示例如下：

```
with open('file.txt', 'a') as f:
    f.write('Last line\n')
```

1.3.10 Python 线程与进程

Python 语言也支持多线程及多进程操作，进程（Process）是操作系统分配资源（CPU、内存等）的基本单位，线程（Thread）是进程中的执行单元，一个进程可以包含多个线程。多进程之间相互独立，适合于 CPU 密集型任务，如大规模计算、图像处理等；多线程可以共享同一进程的内存空间，适合于 I/O 密集型任务，当多线程同时写一块内存时需要加锁避免竞争。下面是一个根据按键状态来控制 LED 灯状态的例子，通过多线程实现了较好的效果。需要注意的是受 GIL (全局解释器锁) 的限制，Python 中的多线程不能真正并行执行多个 CPU 任务，此外一个线程崩溃可能导致整个进程挂起。

```
import threading
import time
import select

# 全局变量，判断按键状态
button_state = 0

def button_task():
    global button_state
    while True:
        events = poll.poll(1000)
        for fd, event in events:
            if event & select.POLLPRI:
                current_time = time.time()
                if current_time - last_click_time < DOUBLE_CLICK_THRESHOLD:
                    # double click
                    button_state = 2;
                    last_click_time = 0;
                else:
                    # single click
                    button_state = 1;
                    last_click_time = current_time;

def led_task():
    global button_state
    while True:
        if button_state==1:
            #LED state 1
        else if button_state==2:
            #LED state 2
```

```
        else :
            #LED state 3

if __name__ == "__main__":
    # 创建两个线程
    t1 = threading.Thread(target=button_task, daemon=True)
    t2 = threading.Thread(target=led_task, daemon=True)

    # 启动线程
    t1.start()
    t2.start()
```

1.3.11 Python 库

Python 语言可以通过库来进行扩展，实现一些在特定领域比较常见的算法，供所有软件开发人员来使用，大大提高了 Python 语言的开发效率。本书后续介绍的人工智能算法很多都有相应的库来支持，复杂的运行过程可以通过简单几个 Python 语句来实现，降低了人工智能的应用门槛，这也使得 Python 成为人工智能开发中最常用的编程语言。

在 Python 的发行版中还包含了大量的标准库，可以在不额外安装软件的前提下使用。例如文件系统的操作、常用数据类型、常用算法等等。例如 `random` 库实现了常用随机数的产生，`random()` 函数在 `[0,1)` 区间生成均匀分布的随机浮点数。`randint(a,b)` 函数返回在 `[a,b]` 区间的均匀分布整数。例如，下面代码生成了十个随机整数。

```
    import random     # 导入 random 模块
    data = []         # 定义一个空的列表 data
    for i in range(0,10):
        data.append(random.randint(0,99))
```

Python 标准库的资料可以参考官方的在线文档（https://docs.python.org/zh-cn/3/library/），了解标准库所包含的内容并在程序中进行运用可以简化代码设计，保证代码的质量。

1.3.12 Python 特色语法

1.3.12.1 列表推导式

Python 具有一些很有特色的语法可以通过单行语句高效生成列表，基本语法为：

```
[expression for item in iterable if condition]
```

- 基础示例：

```
    squares = [x**2 for x in range(10)]  # [0, 1, 4, ..., 81]
```

- 条件过滤：

```
    evens = [x for x in range(100) if x % 2 == 0]
```

- 嵌套循环：

```
    matrix = [[1,2], [3,4]]
    flat = [num for row in matrix for num in row]  # [1,2,3,4]
```

1.3.12.2 生成器表达式

使用圆括号实现惰性求值，适合大数据处理：

```
sum_of_squares = sum(x**2 for x in range(10**6))  # 不生成中间列表
```

1.3.12.3 字典/集合推导式

- 字典推导式：

```
    {key: value for key, value in iterable if condition}
```

- 集合推导式：

```
    {expression for item in iterable}
```

1.3.12.4 海象运算符(Python 3.8+)

在表达式中赋值：

```
[sqrt(y) for x in data if (y := f(x)) > 0]
```

1.3.12.5 模式匹配(Python 3.10+)

```
match response:
    case {'status': 200, 'data': data}:
        process(data)
    case {'status': 404}:
        raise NotFoundError()
```

表 1.1: 推导式性能对比

类型	内存效率	典型场景
列表推导式	低	小型数据转换
生成器表达式	高	流式数据处理
字典推导式	中	键值重构

1.3.13 Python 程序的编写和调试

对于新手来说，直接使用 Vim 编辑源代码来开发 Python 程序确实有些难度，开发板操作系统中安装的 Thonny 则是一个更好的选择。它是一个简单的 Python 集成开发环境，支持语法高亮、代码补全、单步调试等功能，可以方便的编写 Python 代码。图1.3 是 Thonny 的运行界面，上半部分是编辑的程序代码，下半部分是程序的运行结果。

点击工具栏上的调试按钮或者在 Run 菜单中选择 Debug 命令就可以对程序进行单步调试。单步调试时可以选择 Step Over(快捷键 F6)一次执行一条语句，对于函数和类方法等可以一次跳过；也可以选择 Step Into(快捷键 F7)在遇到函数和类方法时可以进入模块内部单步执行。Step Out(快捷键 F8)是跳出当前模块，表示进入到函数和类方法时执行到这个模块的末尾，跳到上一层级的代码。

在编写程序时熟练使用调试功能可以帮助新手快速熟悉语言功能，找出程序中存在的 bug，加深对 Python 程序运行过程的理解。

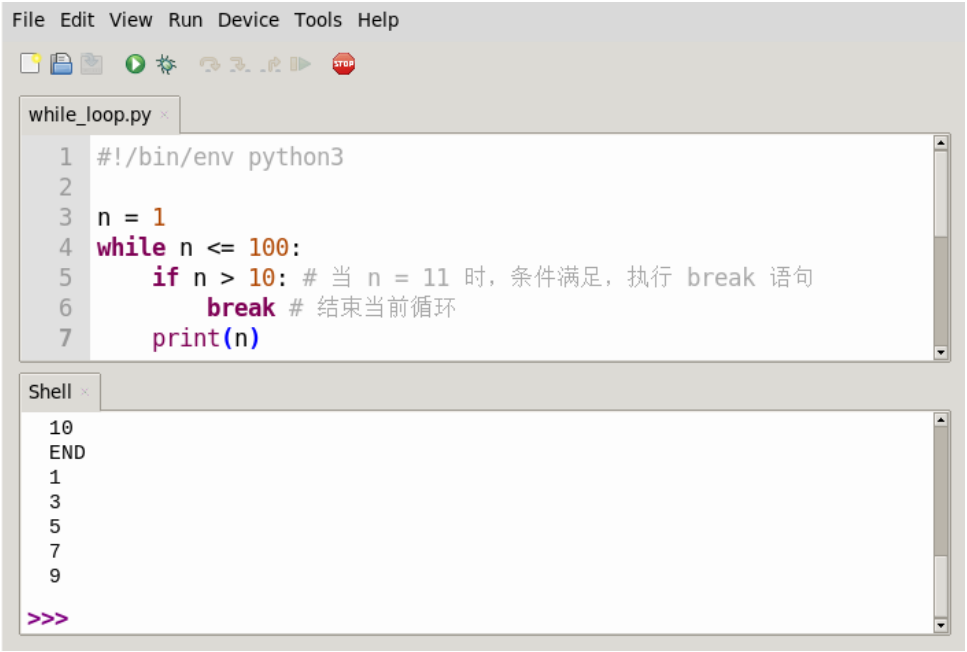


图 1.3: Thonny 的运行界面

1.4 人工智能简介

“智能” 在新华字典中的定义是 “智慧和能力”。这个过分简单的解释并不能让我们清楚的了解什么是智能。实际上，对于人类如何思考，如何解决问题，科学家们依然没有明确的答案。大自然经过千百万年的进化发展，使人类具备了智慧的大脑，可以对物理世界进行观察和分析，并创造出自然界不存在的工具来为人类服务。如今，人类的创造目标包括了智能，这也就是人工智能的主要研究方向。

人类对智能的探索其实古已有之，在《列子》中就记载了偃师设计的木人为周穆王跳舞的故事，这个木人不但可以跳舞说话，还可以抛媚眼。中国古书中记载的类似故事还有很多，不过都没有原理说明，更没有图纸，只能是当时人们的美好幻想。也从另外一个侧面说明人类对自身的思维和运动能力一直具有强烈的好奇心，想了解其背后的原理，并制造出相应的工具。可以认为这种非天然的人类工具具有的智能就是人工智能，与之相关的科学研究组成了人工智能这一学科方向。

人类的智能有很多类别，比如运动能力、计算能力、语言能力、视觉识别能力、逻辑分析能力等等。其中一些能力需要与其它能力配合才可以算作具有智能。比如运动能力，人类很早就可以制造出远超人类力量与速度的工具，但这些还需要人的操作，并不具有智能。最近发展起来的类人机器人则需要复杂的计算模型，并配合视觉处理、逻辑推理等多个模块才可以获得差强人意的效果；智能汽车自动驾驶也是需要多种传感器和算法的支持才能在公路上自由行驶。

下面表格列举了一些常见的人工智能研究方向，而本书也只是涉及了其中的一小部分。

虽然目前人类对自己大脑的了解还不能解释智能的产生，但人工智能的发展却丝毫不受影响。大部分的人工智能研究并不基于人类智能的原理，而主要依靠背后的数学原理和电子学技术。随着计算能力的提升和人工智能研究的深入，很多从前看起来非常困难的任务已经被实现。机器图像识别已经在很多工业领域获得应用，自然语言处理也可以在电话和网络客服中很好的工作，围棋比赛也没有人类可以战胜电脑，人工智能正进入一个蓬勃发展的阶段，不断有新的算法出现，计算机的 “智能” 水平正不断获得提升。

在人工智能领域，python 语言的使用十分普遍，很多算法都通过 python 语言实现并以开源的方式发布在网络上。为了更方便的实现人工智能应用，python 语言还发展出很多程序框架，比较著名的有 scikit-learn、TensorFlow、PyTorch、Caffe、Theano 等。对于比较复杂的人工智能应用，由于需要强大的计算资源，很多 AI 公司也都公开了网络调用 API，可以利用云资源来实现特定的功能。

表 1.2: 人工智能研究方向

研究方向	简单描述
自然语言处理	研究通过文本或者音频进行语言交流的设备或者程序
逻辑推理	一种专门用于推理的编程语言，如 Prolog、Datalog 等
机器视觉	让计算机从图片或者视频中获取信息并进行处理
机器学习	使计算机可以通过学习提高算法的性能
机器人	通过算法使机器可以执行复杂的动作任务
情感计算	通过视觉或文本等识别和处理或者模拟人类的情感
认知理论	研究人类思维的结构，通过计算机对大脑进行建模
游戏智能	在游戏中让非人类玩家具有类似真实人类的表现
智能助理	类似目前手机中常见的语音助手，帮助人们完成一些简单任务
专家系统	将知识用计算机可以理解的方式进行表达，使计算机可以完成需要大量知识的复杂任务

第二章　硬件开发板介绍

2.1　连接开发板

华为的 Atlas 200I DK A2 开发套件（以下简称华为开发板）是由 Atlas 200I A2 核心板和配套底板组成的完整开发平台。它可以连接键盘、鼠标与显示器作为微型电脑使用，也可以通过串口、USB 口和网络接口进行连接。由于开发板的性能有限，在本课程中，主要采用 USB 连接的方式完成实验，利用 PC 的方便性和高性能，提高开发的效率。PC 的操作系统使用 Linux 进行介绍，如果需要使用 Windows，连接方法可以参考华为文档。

使用 USB 连接线连接 PC 主机与开发板的 Type-C 接口，接通开发板的电源，等待系统启动完成（电源接口附近三个绿色 LED 全部点亮）。打开模拟终端程序，在命令行输入（#是注释，输入注释之前内容）：

```
1 $ ssh hwusb   # hwusb 是定义的开发板别名，实际对应 192.168.0.2 的 IP 地址
```

登陆成功后，使用学号创建个人目录，全部实验内容都请保存在个人目录内。

```
1 $ mkdir 2200017003   # 使用你的学号创建目录
2 $ cd 2200017003      # 进入到学号目录
```

2.1.1　使用 JupyterLab

我们使用 JupyterLab 环境完成 Python 语言的编写：

```
1 $ jupyter lab --ip 192.168.0.2 --allow-root --no-browser
```

注意命令的输出内容，找到输出的可打开链接部分，如：

```
To access the server, open this file in a browser:
    file:///root/.local/share/jupyter/runtime/jpserver-10592-open.html
Or copy and paste one of these URLs:
    http://192.168.0.2:8888/lab?token=76337b80235e6a25e238b6fed0a002d4eb2ab105
    http://127.0.0.1:8888/lab?token=76337b80235e6a25e238b6fed0a002d4eb2ab105
```

在 PC 端打开 192.168.0.2 那个地址的链接，就可以看到 JupyterLab 的界面。编写 Python 代码可以使用它的 Notebook，点击图标进入交互式的 Python 调试环境。

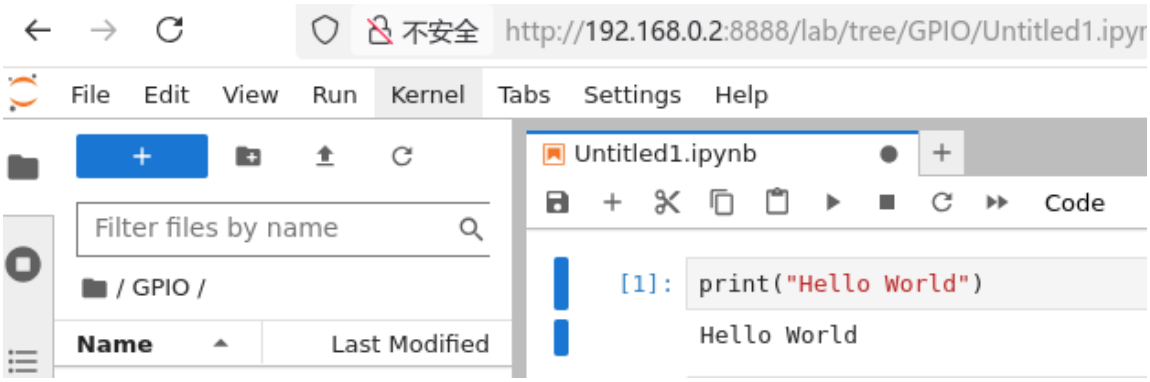


图 2.1: JupyterLab 调试环境

完成实验后，在终端界面按 `Ctrl+C` 键停止 JupyterLab 进程，然后输入 `sudo poweroff` 命令关闭开发板，等待开发板关机（仅剩一个 LED 点亮），断开开发板电源。

JupyterLab 使用 IPython 作为 Python 代码的解释程序，可以认为是 Python 的超集，它用来处理我们创建的 `.ipynb` 文件，运行里面的代码，并将结果显示出来。

2.1.2 使用 VS Code 远程开发

除 JupyterLab 外，也可以在 PC 上使用 Visual Studio Code（VS Code）连接开发板。首先在 PC 端安装 `Remote - SSH`、`Python` 和 `Jupyter` 扩展。可以在 PC 的 SSH 配置文件 `~/.ssh/config` 中添加开发板信息：

```
1 Host hwusb
2     HostName 192.168.0.2
3     User root
```

在 VS Code 中打开命令面板，运行 `Remote-SSH: Connect to Host...` 并选择 `hwusb`。连接成功后，使用 `File -> Open Folder` 打开开发板上的个人目录。此时 VS Code 的集成终端、Python 程序和调试器都运行在开发板上，文件不需要在 PC 和开发板之间反复复制。

VS Code 通过 Jupyter 扩展支持 `.ipynb` 文件。直接打开 Notebook 后，在右上角选择开发板上的 Python 解释器作为内核，即可逐个运行代码单元。使用 GPIO 等硬件模块时，应选择已经安装 `python-periphery` 等依赖的 Python 环境。采用这种方式时，不需要手动启动前面的 `jupyter lab` 服务。

2.2 扩展 IO 接口

华为开发板包含一个 40 管脚的扩展接口，它的完整定义如表 2.1 所示。

2.2.1 GPIO 接口

通用输入输出（General Purpose Input/Output，GPIO）是嵌入式系统中常用的数字信号接口，允许开发者通过软件控制引脚的电平状态或读取外部信号。每个 GPIO 引脚可独立配置为输入或输出模式，具有灵活性强、使用简单的特点。

GPIO 引脚通常有以下两种工作模式：

- **输入模式**：用于读取外部设备信号（如按键状态、传感器输出），可检测高电平（逻辑 1）或低电平（逻辑 0）。
- **输出模式**：用于控制外部设备（如 LED、继电器），可主动输出高电平或低电平。

表 2.1: 开发板扩展接口管脚定义

管脚号	名称	电平	管脚	名称	电平
1	3.3V	3.3V	2	5.0V	5V
3	I2C7_SDA	3.3V	4	5.0V	5V
5	I2C7_SCL	3.3V	6	GND	–
7	GPCLK0	3.3V	8	UART0_TX	3.3V
9	GND	–	10	UART0_RX	3.3V
11	GPIO17	3.3V	12	PCM CLK	3.3V
13	GPIO27	3.3V	14	GND	–
15	GPIO22	3.3V	16	GPIO23	3.3V
17	3.3V	3.3V	18	GPIO24	3.3V
19	SPI0_MOSI	3.3V	20	GND	–
21	SPI0_MISO	3.3V	22	GPIO25	3.3V
23	SPI0_SCLK	3.3V	24	SPI0_CS0	3.3V
25	GND	–	26	GPIO7	3.3V
27	I2C6_SDA	3.3V	28	I2C6_SCL	3.3V
29	GPCLK1	3.3V	30	GND	–
31	GPIO6	3.3V	32	PWM0	3.3V
33	GPIO13	3.3V	34	GND	–
35	PCM FS	3.3V	36	GPIO16	3.3V
37	GPIO26	3.3V	38	PCM DIN	3.3V
39	GND	–	40	PCM DOUT	3.3V

表2.1 中有 11 个 GPIO 可以供用户使用[1]。其它管脚的功能会在后续内容中进行介绍。实际上这个接口的定义和树莓派[2]相兼容，这些 GPIO 的编号与树莓派中的定义相同。实际在华为处理器内部，GPIO 会分成多个组，每组一般有 32 个管脚，而在 Linux 操作系统内部，全部 GPIO 会顺序编号，在实际应用中会更加常用。表 2.2 展示了 GPIO 的对应关系。

2.2.2 I2C 接口

I²C 总线最初由 Philips Semiconductor（NXP Semiconductors）提出，用于处理微处理器与芯片之间的串行低速数据交互。相比于 SPI 连接信号更少，不需要额外的地址译码器及片选信号，多个器件可以简单的连接到同一条 **I²C** 总线，在与传感器、低速 AD/DA、低速 OLED 显示屏等低速数据交互方面获得了较为广泛的使用。SMBus（System Management Bus）是 **I²C** 的一个子集，最早在 1995 年由 Intel 公司定义，最广泛的应用在电脑主板及外部电源模块、温度传感器、电压传感器等数据交互。典型的 **I²C** 应用如图2.2所示。

一般的，使用 **I²C** 设备连接中由主（master）、从（slave）设备组成。主设备发起/结束一次传输，并维护时钟信号，从设备根据主设备的寻址来响应数据传输。在同一时刻允许一个设备发送数据至总线，其余设

[1]管脚的功能可以通过内核配置（dtb 文件）进行调整，这里显示的是出厂设置。

[2]一款非常流行的嵌入式开发平台

表 2.2: GPIO 编号对应表

管脚号	树莓派编号	GPIO 组与编号	Linux 编号	扩展板设备[3]
11	GPIO17	2-18	82	
13	GPIO27	1-06	38	
15	GPIO22	2-15	79	
16	GPIO23	2-16	80	
18	GPIO24	0-25	25	
22	GPIO25	0-02	2	摇杆
26	GPIO7	2-19	83	
31	GPIO6	2-20	84	
33	GPIO13	4-00	128	OLED
36	GPIO16	2-17	81	OLED
37	GPIO26	0-03	3	LED

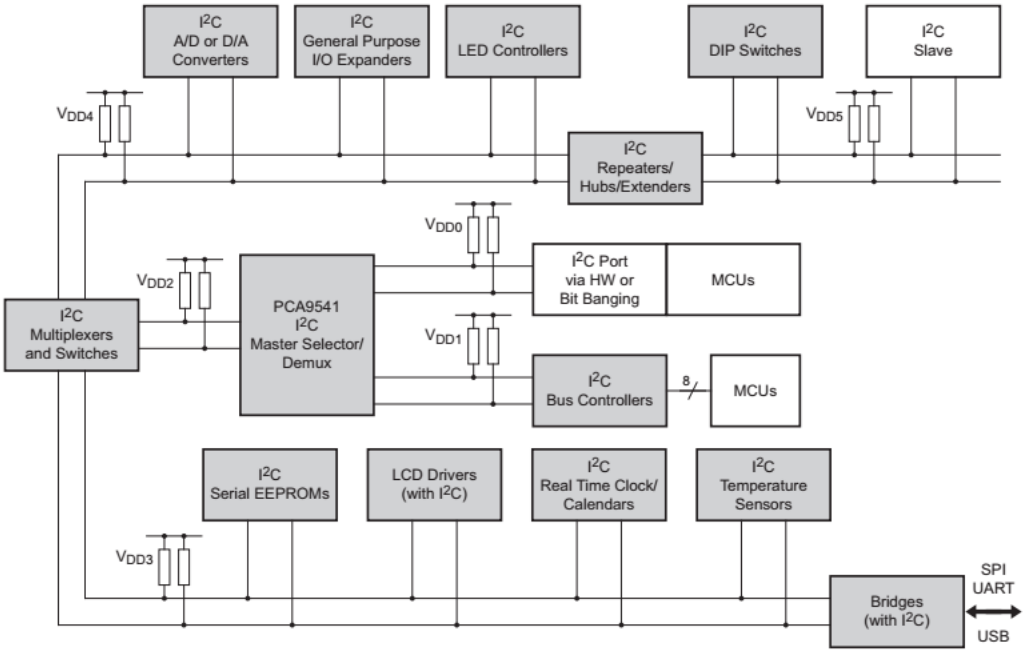


图 2.2: 典型 **I²C** 应用

备接收总线的信号。**I²C** 总线由两根线 SDA 及 SCL 两根线连接主从设备，两根线都采用上拉方式连接到 VDD。

I²C 支持多种模式下速率传输，分别是标准模式、快速模式、快速模式 +、高速模式。传输速率从 100kbit/s（标准模式）至 3.4Mbit/s（高速模式）。一次典型的数据传输过程如图2.3所示。

I²C 数据传输由主设备产生一个起始位开始，然后传递 7 个地址位（对于 7 位地址模式），指定通信的从设备。接下来传递的一位是读写位，0 表示写入，1 表示读出。再接下来的一位由从设备驱动，如果从设备的地址与主设备发出的地址相同，从设备将把 SDA 信号拉低，表示确认这次数据传输（ACK）。当从设备地址被确认，真正的数据传输就开始了。如果是写入数据，则数据线仍由主设备驱动，从设备在每个字节

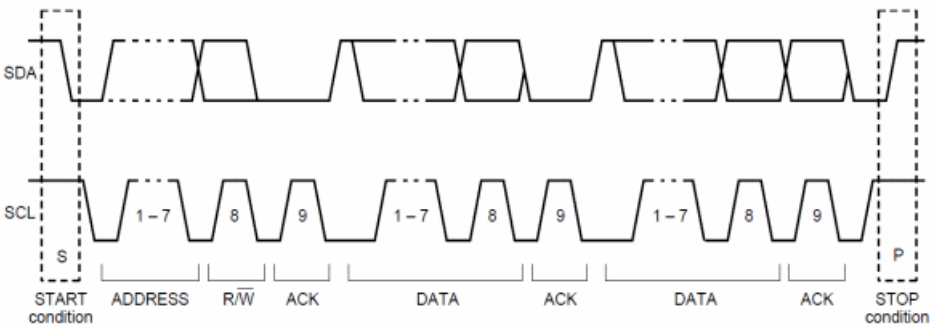


图 2.3: **I²C** 数据传输过程

传输之后也要驱动一位的 ACK 信号。如果是读出数据，则数据线由从设备驱动，主设备仅在应答时输出 ACK 位。最后由主设备产生一个停止位表示数据传输结束。所有的 **I²C** 数据与地址，都是先传递最高位，最后传递最低位。

2.2.3 I²C 设备访问

通过查看 sys 文件系统中的内容，可以知道开发板包含 10 条 I2C 总线：

```
$ ls /sys/class/i2c-adapter/
i2c-0  i2c-1  i2c-13  i2c-2  i2c-3  i2c-5  i2c-6  i2c-7  i2c-8  i2c-9
```

当连接了 Pioneer 扩展板后实际使用的是第七号总线。`i2c-tools` 中 i2cdetect 可以的对指定的 **I²C** 总线进行扫描，下面列表是对第七号总线的扫描结果[4]：

```
$ sudo i2cdetect -y 7
0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:          -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: 20 -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- 48 -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- 68 -- -- -- -- -- -- --
70: -- -- -- -- -- -- 76 --
```

根据扫描的结果，开发板共接入了四个 **I²C** 设备[5]，其中地址 20 为 PCF8574 IO 扩展芯片（参考图2.13 和图 2.14，它连接了五方向按键的其中四个方向、LED2 发光管和 BUZ 蜂鸣器）；地址 48 为 PCF8591 AD/DA 扩展芯片（参考图2.17）；地址 68 为 DS3231 实时钟芯片（参考图2.11）；地址 76 为气压传感器 BMP280 芯片。

`i2cget` 和 `i2cset` 分别用来对 I2C 总线上的设备进行读写。例如：

```
$ sudo i2cset -y 7 0x20 0x8F
$ sudo i2cget -y 7 0x20
0x8f
```

[4] 华为开发板的 I2C 总线驱动暂时还不支持总线扫描

[5] Pioneer 开发板上芯片

2.2.4 SPI 接口

2.2.5 SPI 总线传输原理

SPI（Serial Peripheral Interface）接口标准广泛用于微处理器与外部设备之间的数据交互，如一些传感器芯片、控制芯片、LCD 等。该标准最早由 Motorola 公司提出，目前由 NXP 公司进行标准的维护。

SPI 接口采用主从方式进行全双工的数据传输，其引脚包含四条信号线，具体定义如表2.3所示。

表 2.3: SPI 引脚定义

名称	位宽	功能描述	备注
SCLK	1	串口时钟	由主设备（Master）提供
MOSI	1	主设备数据输出	Master Output Slave In
MISO	1	从设备数据输出	Master In Slave Output
SS / CS	1	从设备片选信号	Slave / Chip Select

2.2.6 SPI 连接模式

典型主从设备连接如图2.4 所示。

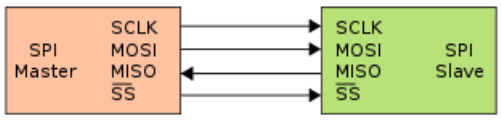


图 2.4: SPI 典型主从设备连接方式

SPI 主设备支持多个 SPI 从设备连接，主要有两种方式：并行连接及链式连接。并行连接方式 SPI 需要多个片选信号线区别多个从设备；链式链接仅需要一根片选线，从设备的输入连接到下一个从设备的输入。

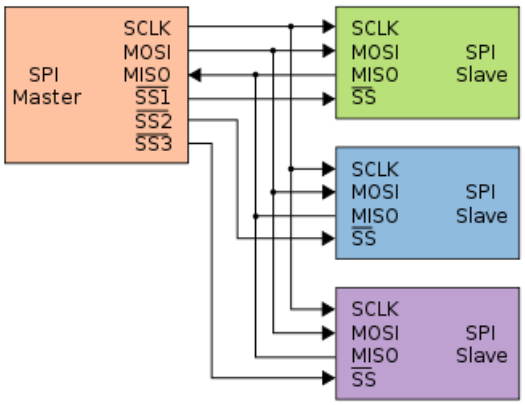


图 2.5: SPI 主从设备并行连接方式

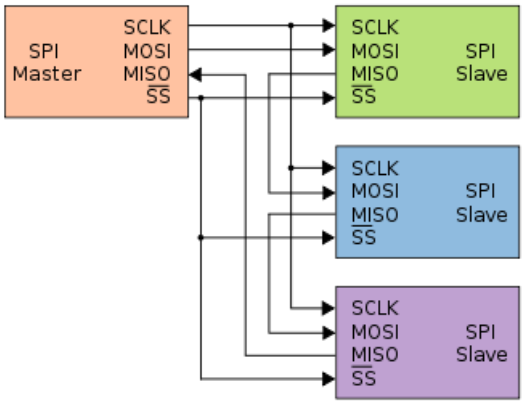


图 2.6: SPI 主从设备链式连接方式

2.2.7 SPI 传输方式配置

如图2.7 所示，当有数据传输时，SPI 主设备产生串行时钟，通过移位寄存器将数据逐 bit 进行传输，根据所配置的传输模式，从设备进行数据采样，完成数据接收。

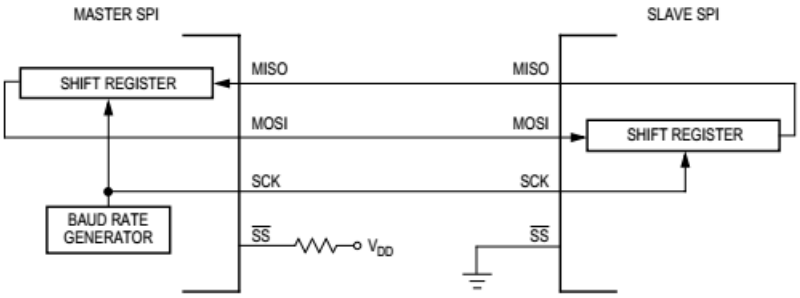


图 2.7: SPI 传输

时钟极性（CPOL）和时钟相位（CPHA）用于设定从设备何时采样数据，图 2.8和 2.9 给出了不同配置下的数据采样位置。两种传输模式中，SCK 与数据的相对位置不同，因此，在两种模式下对于数据采样位置有不同要求，CPHA=1 模式下，采样位置比 SCK 晚半个时钟周期。

SPI 波特率时钟由波特率控制寄存器决定，波特率由如下公式决定，具体算法需要参考处理器的芯片手册：

$$BaudRate = \frac{BusClock}{BaudRateDivisor}$$

2.3 Pioneer 扩展板

Pionner 扩展板是一款为树莓派设计的外围扩展板，带有丰富的板载资源，提供了一些简易的 I/O 设备，例如 LED 指示灯、五向遥杆等。Pionner 扩展板的外观与配置如图 2.10 所示，具体的器件参考表 2.4，在实验过程中可以通过 40 针的排线与转接板将 Pionner 扩展板和华为开发板连接到一起。

Python 语言的 periphery 模块提供了操作各种外设模块的接口，在 Python 代码中可以通过这个模块控制 Pioneer 扩展板上的各种外设。

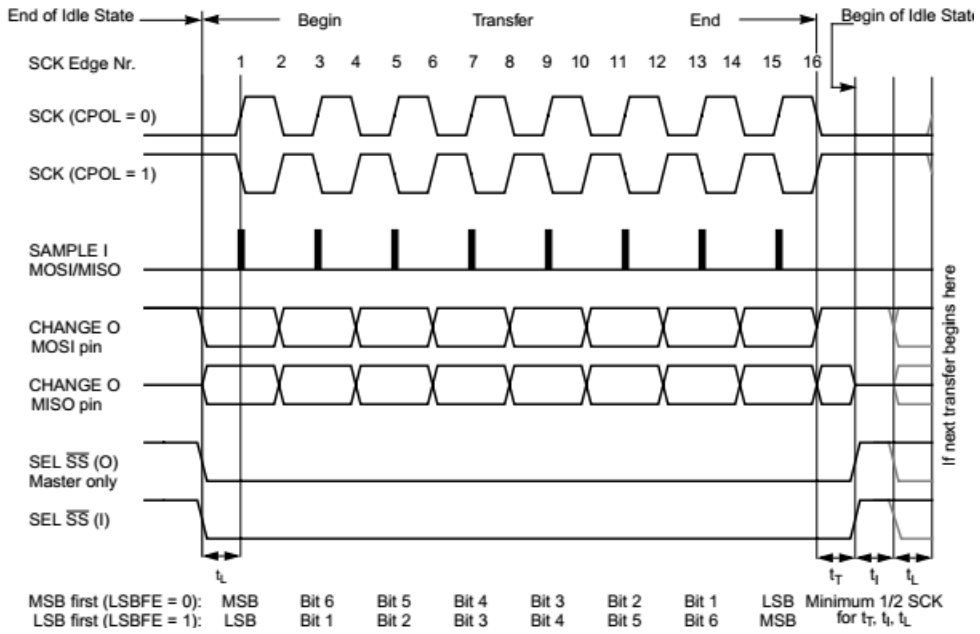


图 2.8: CPHA=0 SPI 传输时序图

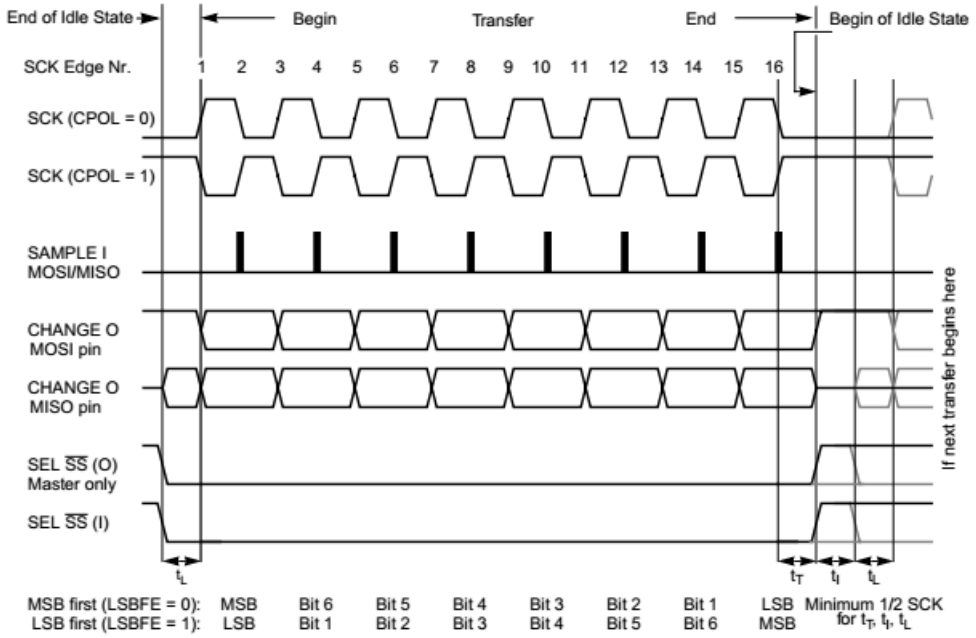


图 2.9: CPHA=1 SPI 传输时序图

2.3.1 GPIO 设备

扩展板上的 LED2 连接到了 GPIO26，可以使用 Python 代码进行控制，相关代码如下所示：

```
1    led = GPIO(3, "out")     # 配置 3 号 GPIO 为输出
2    while True:
3        led.write(True)      # LED ON
4        time.sleep(0.5)      # 等待半秒钟
5        led.write(False)     # LED OFF
6        time.sleep(0.5)
```

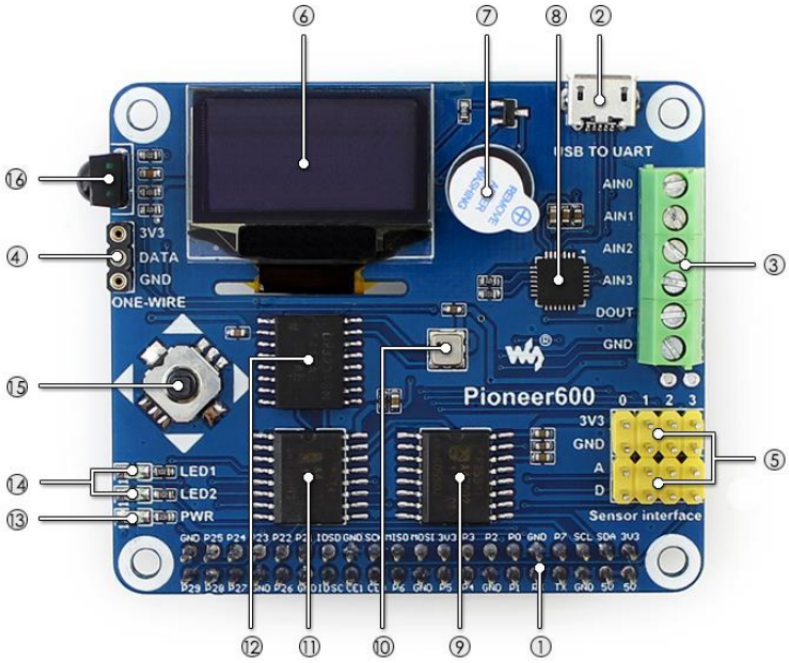


图 2.10: Pioneer600 树莓派扩展板

扩展板的按键连接到了 GPIO25，相关的代码如下所示：

```
key = GPIO(2, "in")      # 配置 2 号 GPIO 为输入
while True:
    value = key.read()  # 读取 GPIO 的状态
    print(f"GPIO 2 value: {value}")
    time.sleep(0.5)
```

2.3.2 I²C 设备

在扩展板上连接了多个 **I²C** 设备，前面小节中 `i2c-detect` 查询到的有 DS3231、BMP280、PCF8574、PCF8591。

2.3.2.1 DS3231 RTC

DS3231 是一款高度集成的 RTC（Real Time Clock）时钟芯片，自动维护时分秒、年月日时间信息。其内部框图如图 2.11所示，内部寄存器如图2.12所示。

RTC DS3231 的地址为 0x68，下面示例给出了 RTC 的访问流程

```
from periphery import I2C
import time

address = 0x68                  # I2C 地址

i2c = I2C("/dev/i2c-7")     # 初始化i2c Bus
# FixTime 定义为 2019 年 6 月 12 日 18 时
FixTime = [0x00,0x00,0x18,0x03,0x12,0x06,0x19]

# 定义时钟操作函数
```

表 2.4: 扩展板接口与器件简介

核心接口		8	CP2102，USB 转串口芯片
1	Raspberry Pi GPIO 接口 方便接入开发板	9	PCF8591 8 位精度 AD/DA 芯片，I2C 接口
2	USB TO UART 接口 方便通过串口终端控制开发板	10	BMP280 气压传感器，I2C 接口
3	AD/DA 输入输出接口 (接线端子)	11	PCF8574 I/O 扩展芯片，I2C 接口
4	ONE-WIRE 接口 可接入 DS18B20 等单总线设备	12	DS3231 高精度 RTC 芯片，I2C 接口
5	传感器接口 方便接入各类传感器	13	电源 LED
器件简介		14	用户 LED
6	0.96 寸 OLED 128×64，SPI 接口	15	五向摇杆
7	蜂鸣器	16	LFN0038K 红外接收头

```
11      def ds3231SetTime():
12          msgs = [I2C.Message([0x00] + FixTime)]  # 地址为 0x00
13          i2c.transfer(address, msgs)
14
15      def ds3231ReadTime():
16          msgs = [I2C.Message([0x00]), I2C.Message([0x00] * 7, read=True)]
17          i2c.transfer(address, msgs)
18          print(msgs[1].data)
19
20      ds3231SetTime()  # 设置时间
21      ds3231ReadTime() # 读出时间
```

根据图 2.12 中寄存器的描述。`ds3231SetTime` 写入了前 7 个地址，分别设置了年月日时分秒和星期的内容。`ds3231ReadTime` 读出前 7 个地址，也包含了全部的时间信息。注意这里 DS3231 中保存的数据都是 BCD 编码类型，也就是 0x19 代表十进制数 19，在操作和应用的时候要注意转换。

Message 可以看作是 I2C 总线传递信息的容器，定义了传输的内容，包括传输的数据和读写方向。transfer 函数可以把一个 Message 的列表进行传输。例如读时间函数中的定义中，就定义了两个传输的片段，一个是写入的一个字节：0x00，代表后续数据的起始地址；另外一个就是七个字节的实际数据，是从芯片中读取的时间信息。

2.3.2.2 PCF8574 扩展 IO

PCF8574 是基于 **I²C** 接口的 8bit IO 接口扩展芯片，通过其接收方向杆的输入及控制蜂鸣器的输出，其内部框图如图2.13所示，外设连接如图2.14 所示。从图2.14 中可以看出：五方向摇杆按钮的四个方向接到了 PCF8574 的 P0 到 P3 端口（分别对应 “左”、“上”、“下”、“右” 四个方向），P4 端口接了发光管 LED2，P7 端口接了蜂鸣器输出。

下面的示例程序给出了五向摇杆输入的读取。

```
1       address = 0x20
```

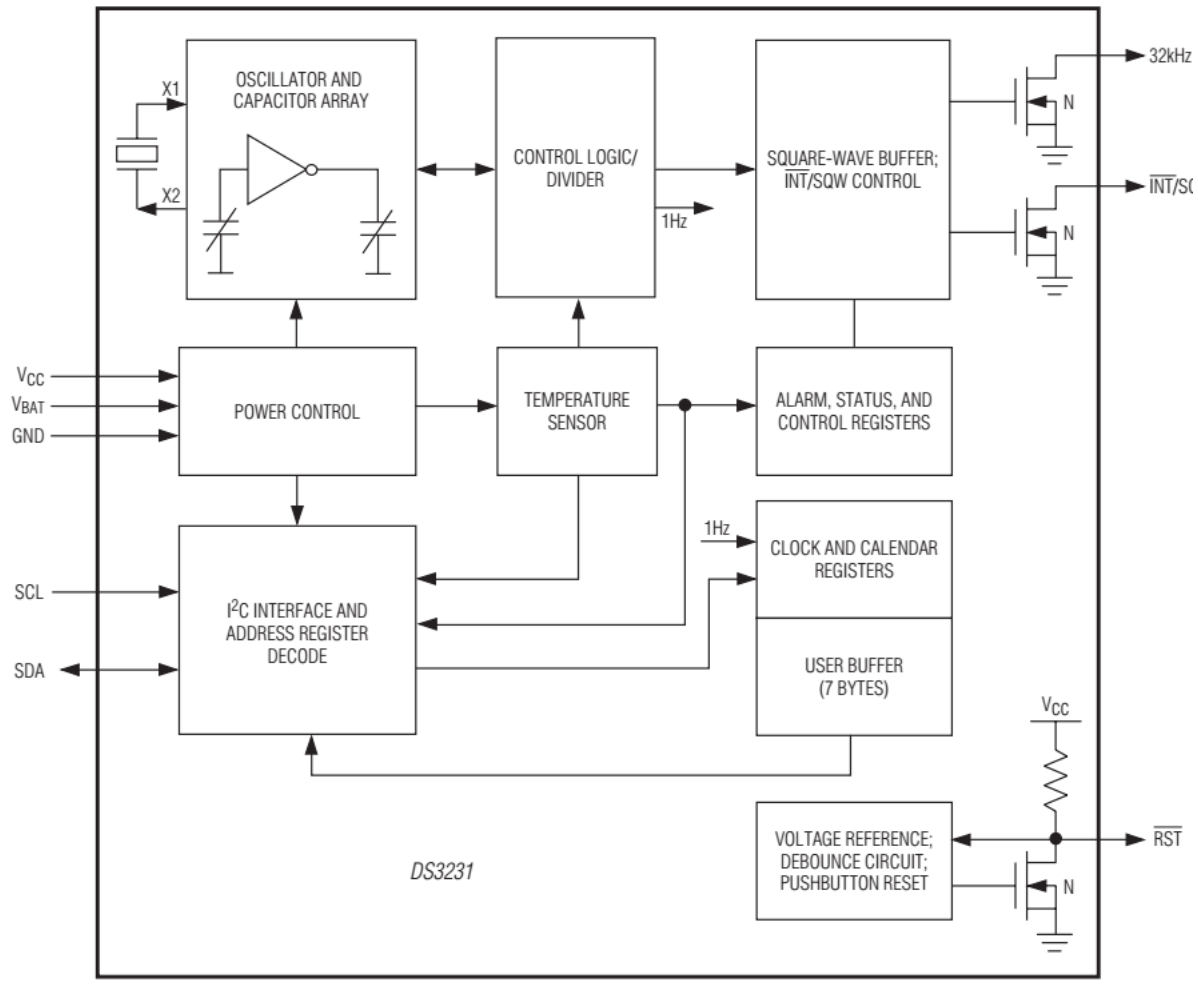


图 2.11: DS3231 内部框图

```
    i2c = I2C("/dev/i2c-7")     # 初始化i2c Bus

    msgs = [I2C.Message([0xff])]
    # 关闭蜂鸣器、初始化准双向 IO 口为输入
    i2c.transfer(address, msgs)

    # 摇杆输入
    msgs = [I2C.Message([0x00], read=True)]
    i2c.transfer(address, msgs)
    joystick_val = msgs[0].data[0] # P 口的读出状态，低四位是摇杆状态
```

2.3.2.3 PCF8591 AD/DA 转换芯片

把模拟量转换为数字量的设备称为模数(A/D)转换器。在单端输入情况下,A/D 转化启动时,对 A/D 输入的模拟信号与地之间的差值与 A/D 的最小分辨率比较，根据比较的值量化输出得到最终的 AD 值(如图2.15)。A/D 的最小分辨率与 A/D 的位数有关。如在 8bit 条件下,A/D 的最小分辨电压为:

$$V_{lsb} = \frac{V_{REF} - V_{AGND}}{256}$$

ADDRESS	BIT 7 MSB	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0 LSB	FUNCTION	RANGE
00h	0	10 Seconds			Seconds				Seconds	00–59
01h	0	10 Minutes			Minutes				Minutes	00–59
02h	0	12/$\overline{24}$	$\overline{AM}$/PM 20 Hour	10 Hour	Hour				Hours	1–12 + $\overline{AM}$/PM 00–23
03h	0	0	0	0	0	Day			Day	1–7
04h	0	0	10 Date		Date				Date	01–31
05h	Century	0	0	10 Month	Month				Month/ Century	01–12 + Century
06h	10 Year				Year				Year	00–99
07h	A1M1	10 Seconds			Seconds				Alarm 1 Seconds	00–59
08h	A1M2	10 Minutes			Minutes				Alarm 1 Minutes	00–59
09h	A1M3	12/$\overline{24}$	$\overline{AM}$/PM 20 Hour	10 Hour	Hour				Alarm 1 Hours	1–12 + $\overline{AM}$/PM 00–23
0Ah	A1M4	DY/$\overline{DT}$	10 Date		Day				Alarm 1 Day	1–7
					Date				Alarm 1 Date	1–31
0Bh	A2M2	10 Minutes			Minutes				Alarm 2 Minutes	00–59
0Ch	A2M3	12/$\overline{24}$	$\overline{AM}$/PM 20 Hour	10 Hour	Hour				Alarm 2 Hours	1–12 + $\overline{AM}$/PM 00–23
0Dh	A2M4	DY/$\overline{DT}$	10 Date		Day				Alarm 2 Day	1–7
					Date				Alarm 2 Date	1–31
0Eh	$\overline{EOSC}$	BBSQW	CONV	RS2	RS1	INTCN	A2IE	A1IE	Control	—
0Fh	OSF	0	0	0	EN32kHz	BSY	A2F	A1F	Control/Status	—
10h	SIGN	DATA	DATA	DATA	DATA	DATA	DATA	DATA	Aging Offset	—
11h	SIGN	DATA	DATA	DATA	DATA	DATA	DATA	DATA	MSB of Temp	—
12h	DATA	DATA	0	0	0	0	0	0	LSB of Temp	—

图 2.12: DS3231 内部寄存器

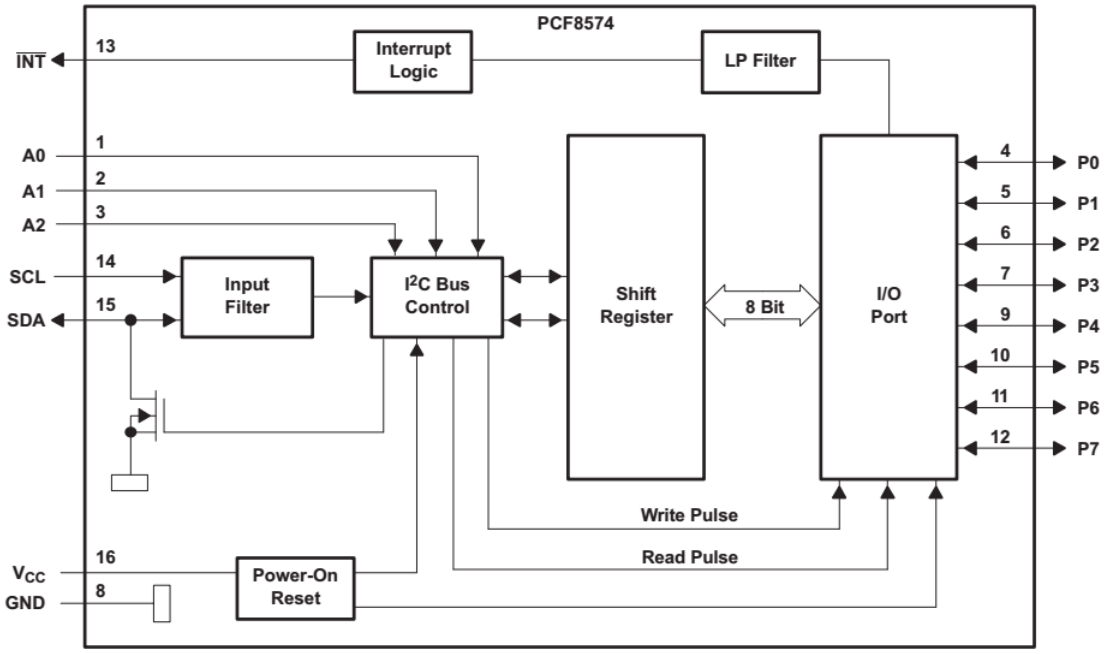


图 2.13: PCF8574 内部框图

把数字量转换为模拟量的设备称为数模（D/A）转换器。如图 2.16，数模转换根据输入（8bit 数据）进行 8bit->256 译码，将译码的输出值驱动分档开关，最后输出不同的电压值，完成 D/A 转换。

PCF8591 芯片内部框图如图2.17所示。

扩展板上 PCF8591 的 A0/A1/A2 均接低电平，根据 PCF8591 地址配置规则（表2.5），PCF8591 的地址为 0x48。PCF8591 集成了多路 AD/DA 功能，在使用其功能时，需要对其控制寄存器进行控制，控制寄存器的定义如图2.18所示：

控制寄存器共有 8bit，通过配置不同 bit，来实现 A/D 工作模式、通道选择及 D/A 输出功能的定义。

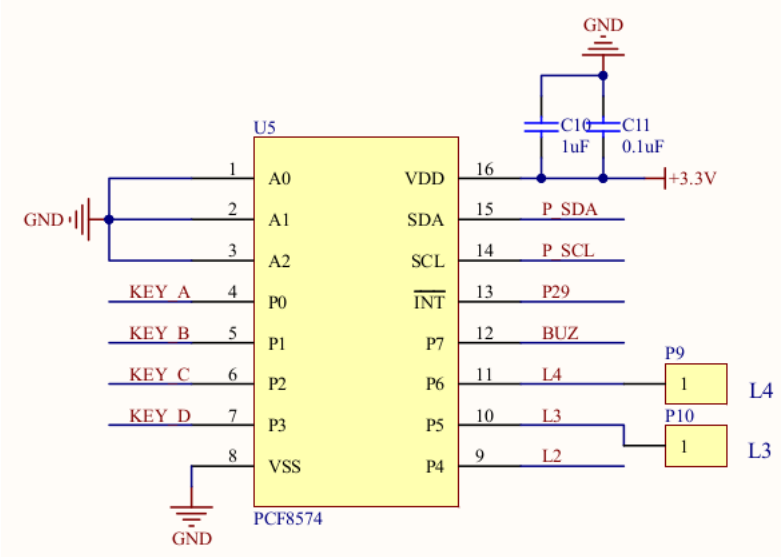


图 2.14: PCF8574 外设连接

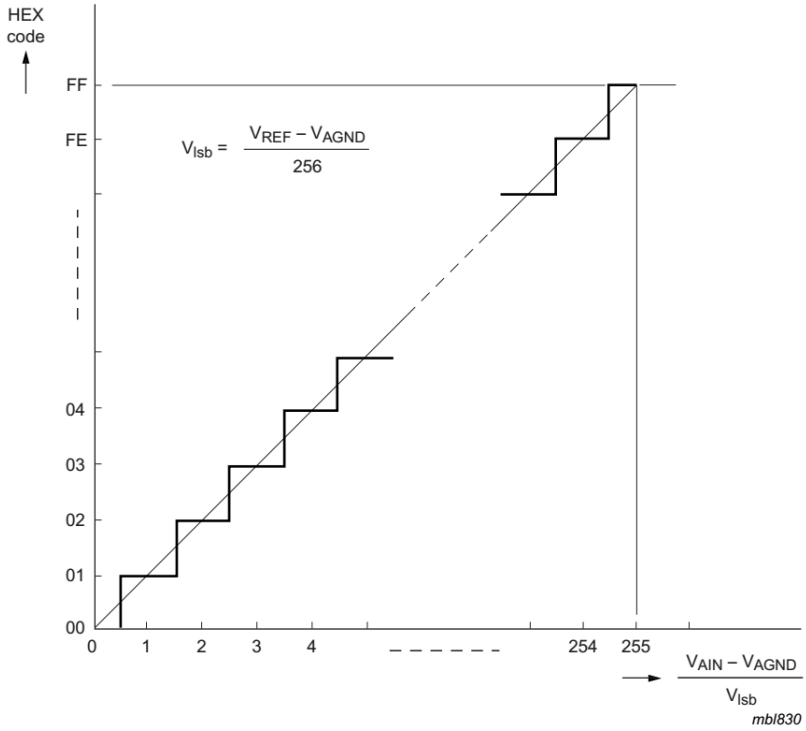


图 2.15: 单端 A/D 原理

表 2.5: PCF8591 地址配置

位	7	6	5	4	3	2	1
地址	1	0	0	1	A2	A1	A0

A/D 的参考代码如下：

```
1 def read_adc(i2c, channel=0):
2     if channel < 0 or channel > 3:
3         raise ValueError("Channel must be 0-3")
```

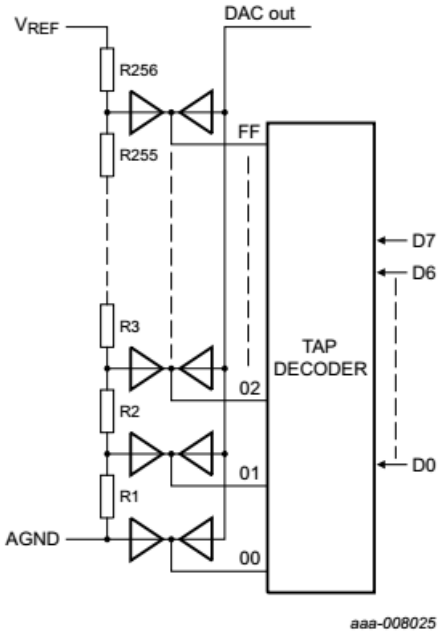


图 2.16: 数模转换原理

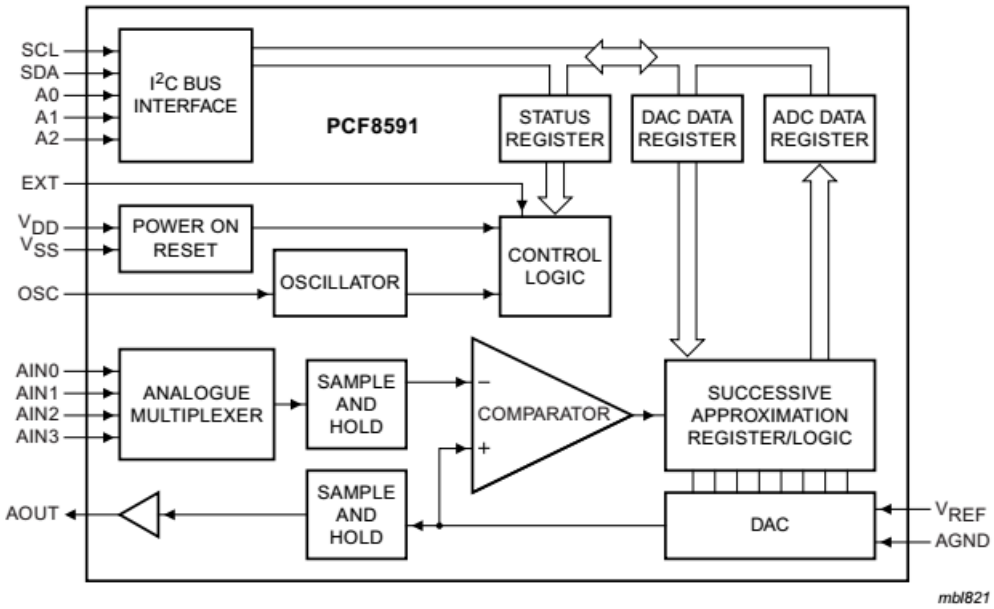


图 2.17: PCF8591 内部框图

```
4
5       control = 0x40 | channel  # 0x40 允许模拟输入
6
7       msgs = [I2C.Message([control])]
8       i2c.transfer(PCF8591_ADDR, msgs) # 写入控制命令
9
10      msgs = [I2C.Message([0x00], read=True)]  # 读出转换结果
11      i2c.transfer(PCF8591_ADDR, msgs)
```

D/A 的参考代码如下：

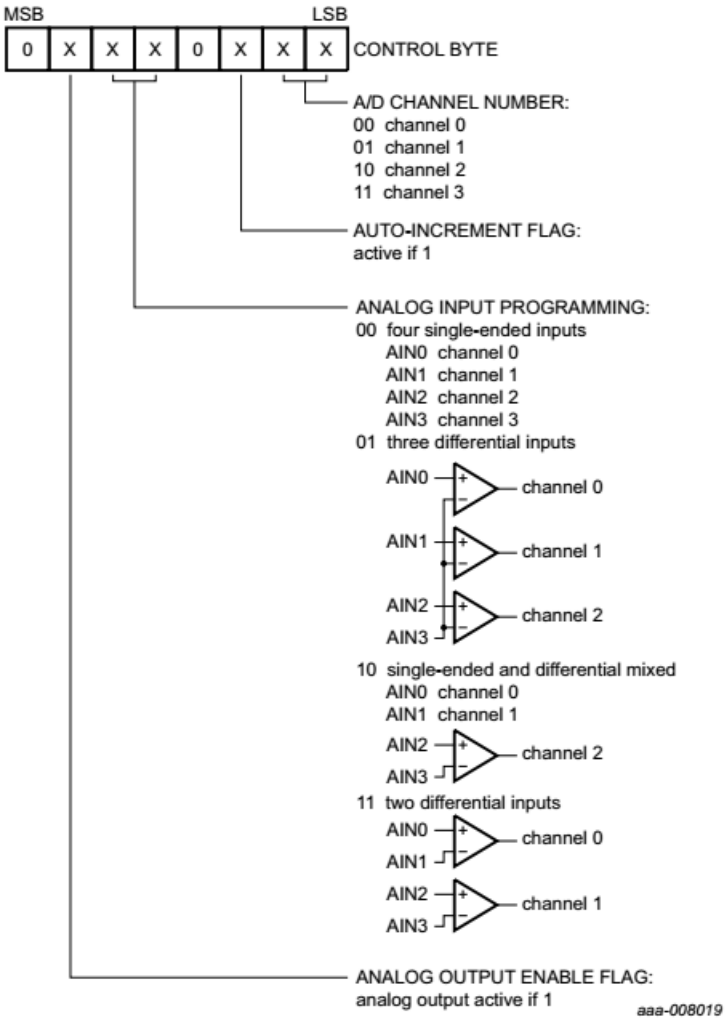


图 2.18: PCF8591 控制寄存器定义

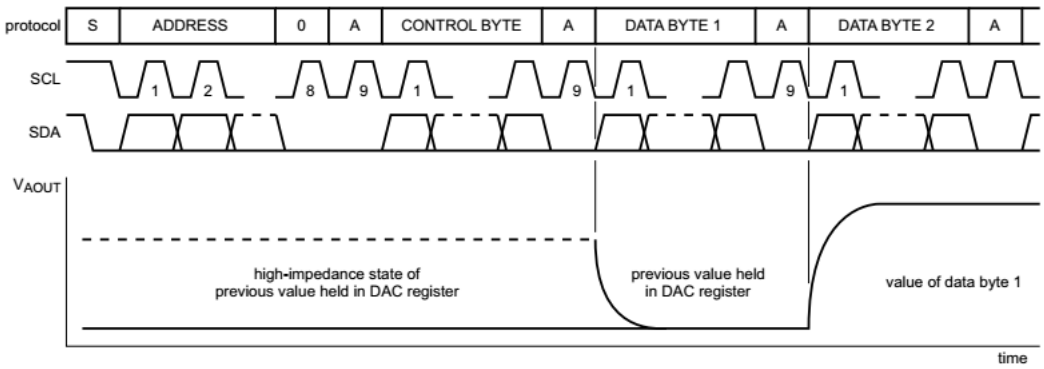


图 2.19: PCF8591 D/A 接口时序

```
def write_dac(i2c, value):
    if not (0 <= value <= 255):
        raise ValueError("Value must be 0-255")

    msgs = [I2C.Message([0x40, value])]
```

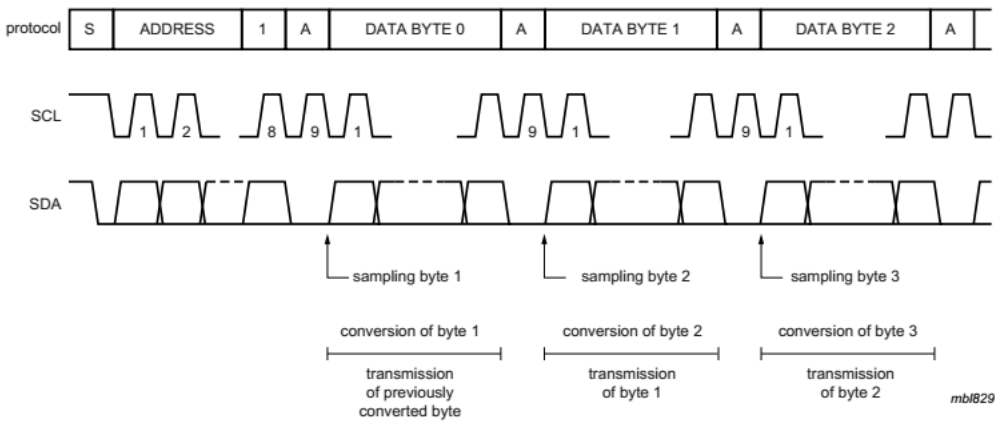


图 2.20: PCF8591 A/D 接口时序

```
6     i2c.transfer(PCF8591_ADDR, msgs)
```

2.3.2.4 BMP280 气压传感器

BMP280 是一款由博世(Bosch)推出的高精度数字气压和温度传感器,广泛应用于气象监测、海拔高度测量、室内导航等领域。它集成温度传感器用于补偿气压测量的温度漂移,同时也可独立输出温度值。传感器出厂时已校准,内置存储在寄存器中的校准系数,用于将原始数据转换为真实值。转换最终测量结果的算法比较复杂,已经封装为 Python 类,在实验中可以直接使用:

```
1 from bmp280 import BMP280
2 sensor = BMP280("/dev/i2c-7", 0x76)
3 print(f"Temperature:␣{sensor.get_temperature():.2f}␣°C")
4 print(f"Pressure:␣{sensor.get_pressure():.2f}␣hPa")
5 sensor.close()
```

2.3.3 SPI 设备

2.3.3.1 OLED 显示屏

128 × 64 点阵的 OLED 显示屏,使用 SSD1306 驱动芯片驱动。SSD1306 支持多种数据接口类型如 8bit 68xx/80xx 并行数据接口,SPI 数据接口、**I²C** 数据接口。具体支持接口类型由相关寄存器配置确定,在扩展板中采用 SPI 数据接口连接。

SPI 支持 3 线、4 线接口,其主要区别在于 4 线接口中多余的一根线用来做数据及命令指示,本实验中采用 4 线接口类型。SSD1306 与开发板的主要连接如图2.21所示,4 线接口的时序如图2.22所示。其中 `CS#` 为片选,低电平有效,`D/C#` 为数据命令指示信号,低电平表示当前传输的内容为命令,高电平表示当前传输的内容为数据。

显示屏由驱动芯片的列驱动(SEG0–SEG127)和行驱动(COM0–COM63)管脚进行控制,当行驱动有效时,128 个列驱动管脚用来决定当前行的显示,因此显示屏的显示内容会一行一行的依次更新,直到整个屏幕内容更新完毕。控制器内部通过一个存储器保存当前的显示内容,由于 SSD1306 驱动的是单色显示模块,因此每个点只需要一个比特信息进行保存,一共需要 128 × 8 个字节的存储空间,这些存储空间分成了 8 个页(从 PAGE0 到 PAGE7),每页存储器保存 8 个行的显示内容。具体如图 2.23 所示。

图 2.23 展示的是显示模块第 16 到第 23 行的显示内容,其中每个小方格代表一个像素点。在 SSD1306 中有三种地址模式: 页地址模式, 水平地址模式和垂直地址模式, 模块可以通过 SPI 接口写入控制命令,

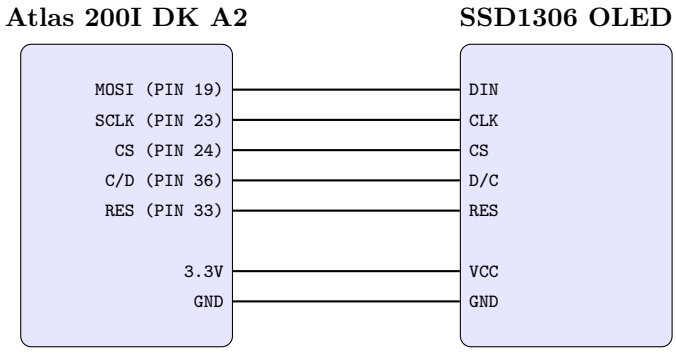


图 2.21: 开发板与 OLED 的连接

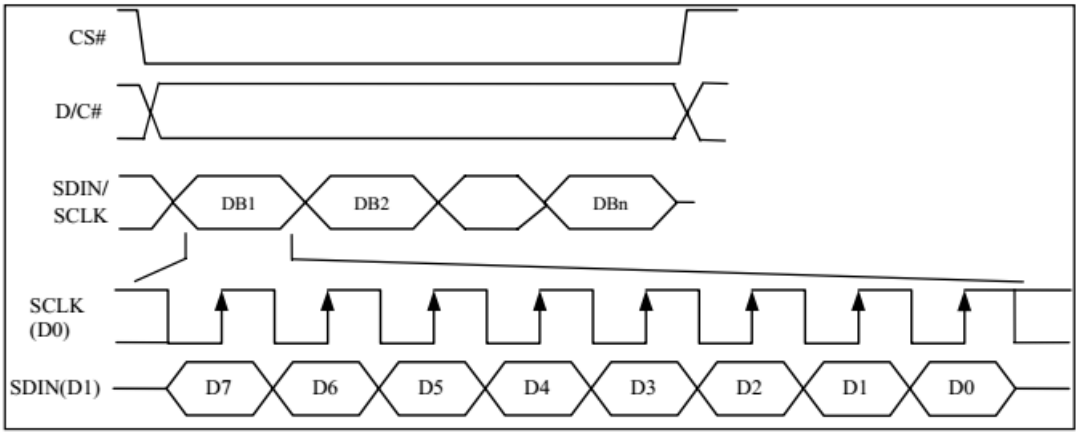


图 2.22: 4 线 SPI 传输时序

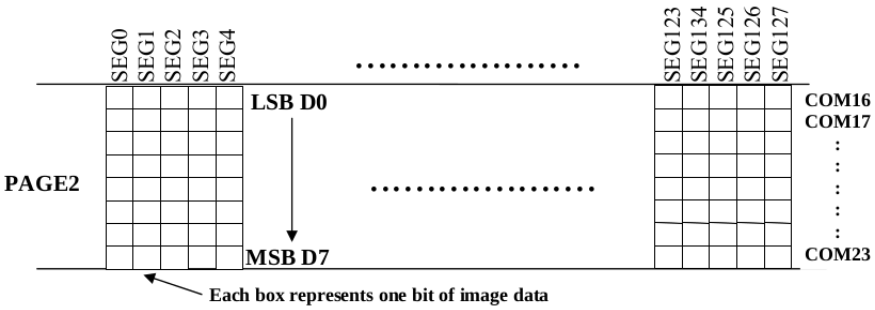


图 2.23: SSD1306 的 PAGE2 内存布局

其中控制地址模式的命令为 0x20，具体方式是通过 SPI 接口发送命令字 0x20，然后发送命令参数，0x00 代表水平地址模式，0x01 代表垂直地址模式，0x10 代表页地址模式（复位缺省值）。

- 在水平地址模式下，每次写入点阵数据后“列地址”自动加 1，准备好下一个存储地址，从而实现连续更改显示内容的功能。在此模式下写完一个页会自动换到下一个页的首地址，写完全部的页会自动换到第一页首地址。
- 在垂直地址模式下，每次写入点阵数据后“页地址”自动加 1，写完全部页后切换到第一页，同时“列地址”自动加 1，写完最后一列会自动换到第一列地址。此模式下可以按列更新显示内容，而水平模式是按行更新内容。
- 在页地址模式下，写入模式与水平地址模式类似，只是页地址不再更新。

完整的 SSD1306 命令列表可以参考其数据手册，这里不再举例。

2.3.3.2 Python 代码驱动 SPI Oled

Periphery 库 Linux 内核的 spidev 驱动来操作 SPI 总线。下面的示例通过 SPI 总线发送三个字节的数据。

```
    from periphery import SPI

    spi = SPI("/dev/spidev0.0", 0, 1000000)    # 新建 SPI 设备类
    to_send = [0x01, 0x02, 0x03]
    data_in = spi.transfer(to_send)            # 发送上面定义的三个字节
```

其中 SPI 类的初始化参数中，第一个是操作系统的设备名称，第二个是 SPI 的工作模式（0-3），第三个是 SPI 的工作频率。

下面一段代码是完整的在 OLED 上输出 “Hello World” 字符的代码，其中定义了 `show_image` 函数，可以用来将图片内容输出到 OLED。

```
from periphery import SPI, GPIO
from PIL import Image, ImageDraw, ImageFont
import time

# GPIO config
gpio_res = GPIO(128, "out")
gpio_dc = GPIO(81, "out")

# SPI config: bus 0, chip select 0 (i.e., /dev/spidev0.0)
spi = SPI("/dev/spidev0.0", 0, 1000000)  # 1MHz

# SSD1306 constants
WIDTH = 128
HEIGHT = 64

# Send command to SSD1306
def ssd1306_command(cmd):
    gpio_dc.write(False)  # Command mode
    spi.transfer([cmd])

# Send data to SSD1306
def ssd1306_data(buf):
    gpio_dc.write(True)  # Data mode
    spi.transfer(list(buf))

# Initialize display
def init_display():
    gpio_res.write(False)
    time.sleep(0.1)
    gpio_res.write(True)

    cmds = [
        0xAE,  # Display off
        0xD5, 0x80,  # Clock divide
        0xA8, 0x3F,  # Multiplex ratio
```

```
        0xD3, 0x00,  # Display offset
        0x40,        # Start line at 0
        0x8D, 0x14,  # Enable charge pump
        0x20, 0x00,  # Memory mode horizontal
        0xA1,        # Segment remap
        0xC8,        # COM scan dec
        0xDA, 0x12,  # COM pins config
        0x81, 0xCF,  # Contrast
        0xD9, 0xF1,  # Precharge
        0xDB, 0x40,  # VCOM detect
        0xA4,        # Resume display from RAM
        0xA6,        # Normal display
        0xAF         # Display ON
    ]
    for cmd in cmds:
        ssd1306_command(cmd)

# Render image to buffer and send to display
def show_image(img):
    # Convert image to 1-bit and get byte data
    img_bw = img.convert("1").resize((WIDTH, HEIGHT))
    pixels = list(img_bw.getdata())
    buffer = [0x00] * (WIDTH * HEIGHT // 8)

    for y in range(HEIGHT):
        for x in range(WIDTH):
            if pixels[x + y * WIDTH]:
                buffer[x + (y // 8) * WIDTH] |= (1 << (y % 8))
    # Set column and page address
    ssd1306_command(0x21)
    ssd1306_command(0)
    ssd1306_command(WIDTH - 1)
    ssd1306_command(0x22)
    ssd1306_command(0)
    ssd1306_command((HEIGHT // 8) - 1)

    # Send all data
    ssd1306_data(buffer)

# Main
def main():
    init_display()

    # Create image with text
    img = Image.new("1", (WIDTH, HEIGHT))
    draw = ImageDraw.Draw(img)
    font = ImageFont.load_default()
    draw.text((0, 0), "Hello␣World!", font=font, fill=255)
    # Show on OLED
```

```
85      show_image(img)
86
87      print("Displayed 'Hello World!' on OLED.")
88
89  if __name__ == "__main__":
90      try:
91          main()
92      finally:
93          spi.close()
94          gpio_res.close()
95          gpio_dc.close()
```

第三章　简单游戏策略

本章我们用开发板制作一个猜拳机器人，玩家可以通过按键输入手势类型与开发板对抗，开发板则通过一定的算法提高自己的胜率。通过本章的学习，读者可以掌握 GPIO 的基本用法，并加深对 Python 语言的理解。

3.1　电脑游戏中的智能

相信大部分读者都玩过电脑游戏，而电脑游戏中又有很大一部分是对抗性的。在电脑游戏中，玩家的对手——电脑——必然具有一定的 “智能” 才能使得游戏比较 “有趣”。对于不同的游戏类型，所需要的智能算法也不同，这里仅讨论有限状态、公开信息的简单对抗游戏，例如猜拳游戏。通过这个大家都很熟悉的游戏，探讨可能涉及到的算法。

3.1.1　随机策略

就像人类有时会通过抛硬币来决定一些事情，计算机通过产生随机数的方法来进行决策是最简单而往往也很有效的方法。比如在猜拳游戏中，如果一方采用完全随机的出拳策略，理论上胜负也同样随机，因此会有 50% 左右的胜率，不算好也不算差。

但目前大部分随机数发生器都是伪随机序列，虽然看起来分布比较均匀，却是通过公式产生，可以通过一定的算法来预测。经典的随机数产生方法是线性同余法（Linear Congruence Generator，LCG），其基本公式如下：

$$X_{n+1} = (aX_n + c) mod m \tag{3.1}$$

线性同余法定义了三个常数：乘量 a、增量 c 和模数 m，X_i 为生成的随机数序列。为了保证产生的随机数可以获得最大周期（不重复长度），这三个常数应该符合：

1. m 与 c 互质
2. m 的所有质因数都能整除 $a-1$
3. 若 m 是 4 的倍数，$a-1$ 也是 4 的倍数
4. a、c 都是正数，还要比 m 小

对于比较简单的情况，游戏一方仅初始化一次随机数（X_0 的取值，即随机数的种子），游戏另一方就有可能猜到这个种子，并正确预测出后续产生的全部随机数，所谓的随机性也就完全丧失了。为了获得更好的随机数，可以采用更好的算法，或者采用可以产生真随机数的物理硬件来实现。

3.1.2　根据历史数据进行决策

在多次重复游戏中，游戏历史数据可以作为分析对象，简单的历史模式匹配就可以成为很好的策略：当各方面条件都相同时，采用一个过去曾经成功的动作获胜的概率会比较大。而如果考虑到对方也会分析历史，那么曾经采用的动作有可能被针对，相对要降低选择的概率。

部分多次重复的游戏运行过程可以抽象成马尔可夫链：从一个游戏状态到另外一个状态的转换状态概率可以视为常数。这样经过几轮游戏之后，就可以积累一定的数据，并根据统计信息计算状态转移的概率。

假设模型的状态空间为 $S_i, i = 1, \cdots, n$，动作空间为 $A_i, i = 1, \cdots, m$。则在特定状态下采取某种动作的概率可以表示为 $P(A_i|S_j)$。对于多阶情况则可以表示为：$P(A_i)|S_{j_1}, S_{j_2}, \cdots, S_{j_k})$。

对于猜拳游戏，根据双方的出拳状态，一共有 9 种可能，那下一次的出拳概率可以根据这 9 种情况采取不同的概率应对。而对方的出拳概率可以根据历史统计资料获得。上面只是考虑了一阶的情况，对于高阶情况则需要考虑更多的历史状态。

3.1.3 利用深度学习进行决策

前面的方法都是利用了人类的经验来设计策略，能不能通过人工学习的方式获得更好的策略呢？这也是目前人工智能研究中的重要分支，在很多领域也已经获得了很好的结果。例如著名的 Alpha-Zero 就可以完全通过学习，在围棋游戏中打败人类顶尖高手。

3.2 Python 语言控制硬件

3.2.1 GPIO 的基本使用

GPIO 的输入、输出模式和 Linux 编号见第 2.2.1 小节，扩展板 LED（Linux 编号 3）与按键（Linux 编号 2）的最小 Python 示例见第 2.3.1 小节。本章在这些基础读写操作上继续介绍异常清理、按键事件轮询和软件 PWM，不再重复基础示例。

3.2.2 Python 的异常捕获机制

在程序设计中，及时释放已申请的资源是良好的编程实践。对于 GPIO 操作而言，恢复其初始设置能有效降低开发板意外损坏的风险。Python 通过 `try ... except ... finally` 语句实现异常捕获机制，该机制不仅能确保资源的可靠释放，还能显著提升程序的容错性。

异常捕获机制的工作原理是：当程序运行出现意外情况时，系统会自动生成相应的异常对象（Error object）。这种机制实现了 “业务逻辑代码” 与 “错误处理代码” 的分离，使程序结构更加清晰，可读性更强。其基本代码结构如下：

```
try:
    # 业务实现代码
    ...
except Error1:
    # 错误处理代码
    ...
finally:
    # 资源释放代码（无论是否发生异常都会执行）
    ...
```

以下是一个实际应用示例：当需要程序在用户按下 Ctrl+C 组合键时优雅退出，同时确保 GPIO 资源被正确释放：

```
try:
    led.write(True)
    ...                     # 其他业务代码
except KeyboardInterrupt:   # 捕获键盘中断异常
```

```
    print("\nExiting...")
finally:
    led.write(False)  # 关闭 LED
    led.close()       # 释放 GPIO 资源
```

3.2.3 检测按键输入

在 GPIO 作为输入时，实时检测按键状态是常见需求。periphery 库提供了 `poll` 和 `poll_multiple` 方法，可以高效地监控 GPIO 引脚的状态变化，特别适合实现按键检测功能。`poll` 方法用于监控单个 GPIO 引脚的状态变化，支持设置超时时间。其基本用法如下：

```
from periphery import GPIO

button = GPIO(2, "in")
button.edge = "falling"    # 使用下降沿触发
button.read()
try:
    while True:
        # 阻塞等待按键按下
        if button.poll(timeout=1.0):
            button.read()
            print("Button pressed!")
        else:
            print("Waiting for button press...")
except KeyboardInterrupt:
    button.close()
```

关键参数说明：

- `edge`: 触发边缘（"rising"、"falling" 或"both"）
- `timeout`: 超时时间（秒），设为 `None` 表示无限等待

当需要同时监控多个 GPIO 引脚时，可以使用 `poll_multiple` 方法：

```
from periphery import GPIO

# 初始化多个 GPIO 引脚
buttons = [
    GPIO(82, "in"),  # 按钮1
    GPIO(38, "in")   # 按钮2
]
buttons[0].edge = "falling"
buttons[1].edge = "falling"

try:
    while True:
        # 同时监控两个按钮（超时时间1000ms）
        events = GPIO.poll_multiple(buttons, timeout=1.0)
        if not events:
            continue
```

```
17
18          # 检查是哪个按键按下
19          for gpio in events:
20              pin = gpio.line
21              gpio.read()
22              print(f"Button␣{pin}␣pressed!")
23
24  finally:
25      for btn in buttons:
26          btn.close()
```

这两种方法为输入检测提供了高效解决方案，相比轮询方式能显著降低 CPU 占用率。

3.2.4 PWM 信号输出

数字端口的输出只能是高和低两种状态，如果用来控制 LED 就只有亮和灭两种状态。但如果这个连接了 LED 的端口输出一个方波，则这个 LED 就有一半的时间处于亮的状态，另外一半时间处于灭的状态。当方波的频率比较高的时候，人看起来这个 LED 就是一种半暗的状态。

更进一步，当控制输出的方波有不同的占空比的时候，LED 的亮度就会有不同的变化，从宏观上看就是端口的输出功率在不停的变化。这种输出信号的形式就叫做 PWM（Pulse Width Modulation，脉宽调制）。很多开发平台支持硬件的 PWM 设备，但也可以通过 GPIO 的方式来模拟 PWM 波形。下面的程序代码实现了控制 LED 灯产生呼吸灯效果，请在开发板编辑运行程序，并观察效果。

```
1   def main():
2       gpio = GPIO(3, "out")
3       print("Fading␣LED␣on␣GPIO␣3␣using␣software␣PWM␣(press␣CTRL+C␣to␣stop)...")
4
5       try:
6           while True:
7               # Fade in
8               for i in range(STEPS):
9                   duty = i / (STEPS - 1)  # 0.0 to 1.0
10                  pwm_cycle(gpio, duty)
11              # Fade out
12              for i in reversed(range(STEPS)):
13                  duty = i / (STEPS - 1)
14                  pwm_cycle(gpio, duty)
15      except KeyboardInterrupt:
16          print("\nExiting...")
17      finally:
18          gpio.write(False)
19          gpio.close()
20
21  def pwm_cycle(gpio, duty):
22      # Run a single PWM cycle
23      on_time = PWM_PERIOD * duty
24      off_time = PWM_PERIOD - on_time
25      gpio.write(True)
26      time.sleep(on_time)
```

```
27      gpio.write(False)
28      time.sleep(off_time)
```

3.3 动手实验:猜拳游戏

3.3.1 实验目的

1. 熟悉 PC 和开发板的操作系统。
2. 熟悉 Python 语言的编程和调试。
3. 掌握 GPIO 接口的编程方式和使用方法。
4. 设计简单的游戏策略

3.3.2 预习要求

1. 阅读第 2.2.1 和 2.3.1 小节,掌握 GPIO 编号及基本输入输出方法
2. 阅读 1.2 小节,掌握 Linux 环境基本用法
3. 理解本节 Python 语言代码,编写实验内容中要求的程序

3.3.3 实验内容

3.3.3.1 GPIO 输出

利用开发板上 GPIO 接口实现 LED 指示灯的输出控制。Pioneer 扩展板 LED 指示灯的 Linux 编号为 3。请实现以下功能:

1. 实现 LED 指示灯的闪烁,点亮 0.5 秒,熄灭 0.5 秒。
2. 完成脉宽调制功能的代码,实现呼吸灯的功能。

3.3.3.2 按键输入

利用按键对 LED 指示灯的输出进行控制。

1. 利用按键控制 LED 灯明灭,每按下一次按键就切换 LED 灯的状态,由明至暗或由暗至明。
2. 单击按键使 LED 灯进入闪烁模式,再次单击按键闪烁频率加倍,双击按键停止闪烁(提示:双击可以定义为 0.5 秒钟内按键两次,计时可以使用 time.time())。

3.3.3.3 实现猜拳游戏

猜拳是一种简单的游戏,共有剪刀、石头、布三个手势。二人同时用手做出相应形状,输赢判断规则为:剪刀赢布,布赢石头,石头赢剪刀。在开发板上实现猜拳游戏要通过三个按键来实现三种输入,三个按键分别接入到扩展接口的 11、13、15 号管脚(编号为 82、38、79)代表剪刀、石头、布三个手势。程序在用户按下按键前产生自己的手势编号,当用户按下按键时判断胜负。用 LED 灯作为按键输入成功的反馈,在屏幕上打印当前的比分(胜/局数)。

提升计算机的出拳策略,根据用户历史“出拳”信息,尽量提高开发板的胜率。策略可以仅仅根据上一次的用户出拳结果,也可以根据多次出拳结果的统计信息。

3.4 思考题

1. 用 GPIO 模拟的 PWM 信号有什么缺点？
2. 如何产生不同分布的随机数？
3. 如果猜拳游戏的对手是电脑，什么样的策略更好？

第四章　聚类算法

聚类是机器学习中非常重要的一类算法，它不需要对数据进行标记，属于非监督学习。本章将介绍最常用的 K 均值聚类算法，对从传感器获取的数据进行分类，进而实现预测的功能。本章所使用的传感器是 BMP280 气压和温度传感器，通过 I2C 总线进行通信。

4.1　聚类算法简介

聚类算法试图将数据集中的样本分成若干个通常不相交的子集，每个子集称为一个簇（cluster）。通过这样的划分，每个簇就可能对应于一个潜在的概念，例如可以把生物分成植物和动物，把书籍分成文史类和科技类等。这种分类方式在人类认识世界的过程中不断的在进行着，但如果让计算机掌握这样的分类能力就不那么容易了。

基于不同的策略，可以设计出多种类型的聚类算法，这里介绍一种非常简单的 K 均值聚类算法（K-means clustering）。

K 均值聚类算法是一种迭代求解的聚类分析算法，其步骤是随机选取 K 个位置作为初始的聚类中心，然后计算每个样本与各个聚类中心之间的距离，把每个样本分配给距离它最近的聚类中心。聚类中心以及分配给它们的样本就代表一个聚类。每分配完成后，聚类的聚类中心会根据聚类中现有的样本被重新计算，获得新的聚类中心。

这个过程将不断重复直到满足某个终止条件。终止条件可以是没有（或最小数目）样本被重新分配给不同的聚类，或者是聚类中心不再发生变化。

4.2　用到的 Python 库

4.2.1　NumPy 库

NumPy 是 Python 中科学计算的基础软件包。它是一个提供了多维数组对象，多种派生对象（如：掩码数组、矩阵）以及用于快速操作数组的函数及 API，它包括数学、逻辑、数组形状变换、排序、选择、I/O 、离散傅立叶变换、基本线性代数、基本统计运算、随机模拟等等。

我们这里可以用它生成特定分布的随机数。例如下面代码生成了 10 个方差为 sigma，均值为 mean 的正态分布随机数。

```
1    import numpy as np
2    mean = 5
3    sigma = 1.3
4    x=mean+sigma*np.random.randn(10)
```

NumPy 包的常用数据类型是 ndarray 对象。它与标准 Python Array（数组）之间有几个重要的区别：

1. NumPy 数组在创建时具有固定的大小，与 Python 的原生数组对象（可以动态增长）不同。更改 ndarray 的大小将创建一个新数组并删除原来的数组。

2. NumPy 数组中的元素都需要具有相同的数据类型,因此在内存中的大小相同。例外情况:Python 的原生数组里包含了 NumPy 的对象的时候,这种情况下就允许不同大小元素的数组。
3. NumPy 数组有助于对大量数据进行高级数学和其他类型的操作。通常,这些操作的执行效率更高,比使用 Python 原生数组的代码更少。

4.2.2 Matplotlib 库

Matplotlib 是一个 Python 2D 绘图库,可以生成各种格式和跨平台交互式环境的出版物质量图片数据。它的语法格式类似于 matlab,例如下面代码画出 x 数据的直方图。

```
1  import matplotlib.pyplot as plt
2  plt.hist(x,80,histtype='bar',facecolor='yellowgreen',alpha=0.75)
3  plt.show()
```

4.3 动手实验:模式识别

4.3.1 实验目的

1. 了解 I2C 总线基本原理。
2. 了解传感器 BMP280 的基本使用方式。
3. 使用 K 均值聚类算法对数据进行分类。

4.3.2 预习要求

1. 查看 Numpy 和 Matplotlib 的基本用法
2. 阅读 2.2.2 小节,学习 I2C 基本知识
3. 阅读 2.3.2.4 小节,了解 BMP280 的读取方法和示例程序
4. 理解聚类算法的基本流程,编写本节实验要求程序

4.3.3 读取温度传感器的值

使用 Python 代码读取 BMP280 的温度,记录一分钟内温度的变化(每秒记录一次),使用 Matplotlib 绘制时间/温度曲线。BMP280 的接口和读取示例见 2.3.2.4 小节。本实验可以直接导入配套目录`code/cluster`中的`bmp280.py`。

4.3.4 聚类算法实现

假定某共用办公室有两个人使用,他们使用办公室的时候都会用空调遥控器设置房间的温度。但空调遥控器显示面板坏掉了,只能通过加减温度的方式盲调整。已知他们习惯的温度不同,办公室的温度计有记录功能,每天记录 12 小时的温度,每 30 分钟记录一次,根据一个月所记录的数据,通过聚类算法,算出这两个人的喜好温度,并估算他们各使用办公室多少时间。

可以认为这两个人使用办公室的时候室内温度是不同方差和不同均值的正态分布随机数,采用 randn 函数生成模拟数据,并用 matplotlib 画出数据的直方图。

使用 K 均值聚类算法将模拟数据分成两个簇,解答前面的问题。

使用温度传感器读入温度,根据温度值,判断是谁在使用办公室。

4.3.5 二维数据的聚类

使用随机算法生成二维的模拟数据，实现聚类算法。考虑下面两种情况：

- 模拟数据围绕两个中心点正态分布，方差相同
- 模拟数据采用如下代码生成，分布如图 4.1 所示

```
import matplotlib.pyplot as plt
import numpy as np

r1 = 2
r2 = 4
p1 = []
p2 = []

for i in range(10000):
    x = np.random.rand() * r2 * 2 - r2;
    y = np.random.rand() * r2 * 2 - r2;
    if abs(x * x + y * y - r1) < 0.5:
        p1.append([x, y])
    if abs(x * x + y * y - r2) < 0.5:
        p2.append([x, y])
plt.scatter(np.array(p1)[:,0], np.array(p1)[:,1], c='r', s=3)
plt.scatter(np.array(p2)[:,0], np.array(p2)[:,1], c='b', s=3)
plt.show()
```

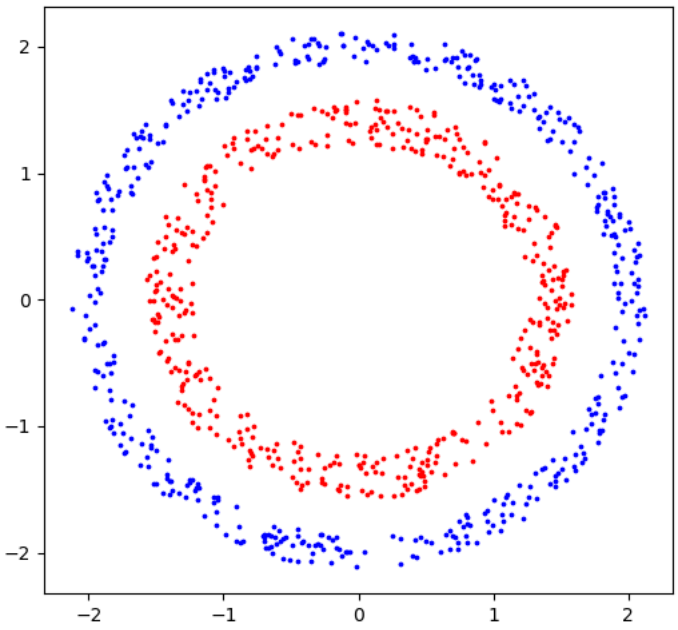


图 4.1: 环状数据

4.4 思考题

1. 分析实验结果，得到的聚类中心是否与模拟数据的均值相等，不等的话请解释偏差的原因。

2. 讨论 k 均值算法的局限性

第五章　OpenCV 基础

机器视觉一直是人工智能领域的研究热点，也是许多上层应用的基石。如何对图像、视频流数据进行处理，进而对目标进行提取、识别是该领域中的重要工作。本章内容以 OpenCV 程序库为基础，首先介绍图像处理的基本原理，再引入一些经典的图像识别算法，并在开发板上完成图像的处理及简单图形识别实践，进一步深入学习 Python 中对矩阵的处理及 Numpy 库的使用。

5.1　OpenCV 简介

OpenCV（Open Source Computer Vision Library）是一个开源的机器视觉程序库，源于 1999 年 Intel 的一个研究项目。OpenCV 包含多种常见的机器视觉算法，目前仍然处于积极开发中，不断有新的功能和算法被整合进来。OpenCV 支持 C++、Java、Python 等多种开发语言，本实验主要采用 Python 语言。

OpenCV 具有模块化结构，包含不同层次的功能供用户使用。下面是常用的模块列表：

核心功能（Core functionality）：定义了基本的数据结构，包括多维矩阵数组以及供其他模块使用的基本功能。

图像处理（imgproc）：包括线性和非线性图像滤波、几何图像变换（缩放、旋转、仿射与透视变换、通用重映射）、色彩空间转换、直方图等。

视频处理（video）：包括运动检测、背景减除与目标跟踪等算法。

三维重建与标定（calib3d）：基于多视图的几何算法，包括相机标定、物体姿态估计、立体匹配与三维重建等。

二维特征（features2d）：显著特征检测、特征描述与特征匹配。

对象检测（objdetect）：对目标及预定义类别实例的检测（如人脸、眼睛、人体、车辆等）。

高级图形界面（highgui）：简单易用的图形用户界面。

视频输入输出（videoio）：简单易用的视频采集与读写模块。

OpenCV 采用 C++ 语言开发，并提供了 Python 语言绑定，在开发板上可以直接通过 Python 代码调用 OpenCV 提供的功能。

OpenCV 包含的功能模块非常多，这章主要介绍图像、物体检测与识别中常用的功能，其它的模块与函数的用法请参考其它书籍或者在线文档。

5.2　图像的存储与显示

在计算机中，图像由二维的数据矩阵（高度，宽度）来表示及存储，如图 5.1 所示。矩阵中的每个元素表示一个像素点（Pixel），在彩色图像中每个像素点由三种基本颜色组成。

OpenCV 使用`imread()`读取图像，使用`imwrite()`保存图像；在 Notebook 中内联显示前，可先用`imencode()`将图像编码为字节流。`imread()`能够读取大多数类型的图像，并以 Numpy 数组保存像素信息。参数 flags 指定读取模式，常用的模式有灰度图像和 BGR 彩色图像。通过远程 JupyterLab 或 VS Code 访问开发板时，本地没有图形桌面，无法使用`cv.imshow()`弹出窗口。应先使用`imencode()`将图像编码为字节流，再借助 IPython.display 在 Notebook 中内联显示。例如：

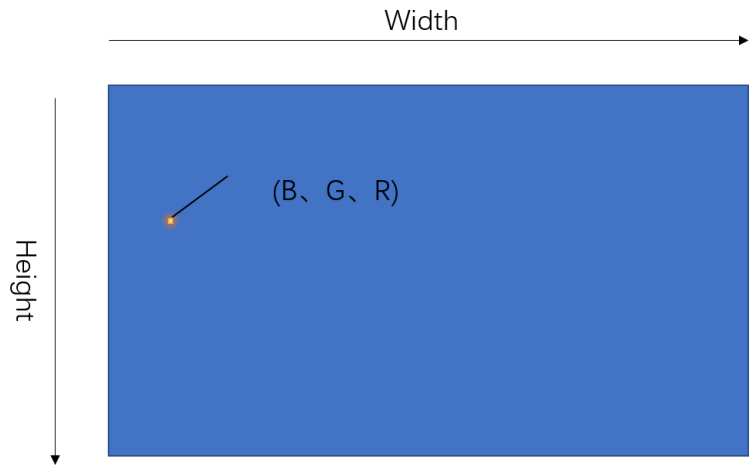


图 5.1: 图像及像素点示意图

```
1  import cv2 as cv
2  from IPython.display import Image, display
3
4  img = cv.imread('opencv.png', cv.IMREAD_COLOR)
5  if img is None:
6      raise FileNotFoundError('找不到图像文件␣opencv.png')
7
8  ok, encoded = cv.imencode('.png', img)
9  if not ok:
10     raise RuntimeError('图像编码失败')
11 display(Image(data=encoded.tobytes()))
```

IPython.display 模块是 IPython 项目中的一个核心组件，它提供了一套强大的工具，用于在 JupyterLab 或其他基于 IPython 的交互式环境中控制丰富内容的显示，可以处理图像、视频、音频等多种媒体元素。

需要说明的是，OpenCV 读取图像后得到的是 numpy.ndarray 对象，Notebook 无法直接将其作为图像展示，因此需要先将它编码为 PNG 或 JPEG 字节流，再交给`IPython.display.Image`显示。该方法同时兼容 JupyterLab 和 VS Code 的 Notebook 编辑器。

5.3 图像的基本处理

图像的基本处理即对二维图像数据的处理，主要包括颜色空间转换以及图像缩放、滤波等操作，OpenCV 中提供了丰富的函数来实现这些基本操作。

5.3.1 图像的颜色空间转换与过滤

颜色空间是采用数学的方式表示颜色的方法，常见的有 RGB、HSV 等。RGB 就是采用颜色的红色（R）、绿色（G）和蓝色（B）分量来表示颜色；HSV 则用色调（H）、饱和度（S）和明度（V）三个分量来表示颜色，如图 5.2 所示。由于 HSV 颜色空间更符合人的主观感受，因此在图像处理软件中有很多应用，在机器视觉领域也经常用来对色彩进行分析。但 OpenCV 中缺省的颜色空间采用 BGR 表示，常常需要进行图像的颜色空间转化。OpenCV 中提供`cvtColor()`函数来实现不同颜色空间的转化。

在图像的颜色处理中，常常需要过滤出图像中感兴趣的颜色。OpenCV 采用 inRange()函数对图像中的色彩进行过滤。需要注意的是，inRange()函数需要指定色彩的范围；为了方便调整范围，可以使用 **createTrackbar()、getTrackbarPos()** 函数在图形界面中增加滑动条[1]，参考代码如下（示例中未使用滑动条）：

```
import cv2
from IPython.display import display
import ipywidgets as widgets
from io import BytesIO
import numpy as np

cap = cv2.VideoCapture(0)          # 打开默认摄像头

if not cap.isOpened():
    print("无法打开摄像头")
else:
    # 创建两个 Image widgets 分别用于显示原始图像和过滤后的图像
    original_image_widget = widgets.Image(format='jpeg', description='原始图像')
    filtered_image_widget = widgets.Image(format='jpeg', description='HSV过滤图像')

    # 使用 HBox 将两个图像并排显示
    display(widgets.HBox([original_image_widget, filtered_image_widget]))

    try:
        while True:
            ret, frame = cap.read()    # 读取摄像头数据
            if not ret:
                print("无法接收帧（可能是流结束或摄像头连接丢失）")
                break

            hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)  # 转换颜色空间

            lower_green = np.array([40, 50, 50])
            upper_green = np.array([80, 255, 255])
            image_mask = cv2.inRange(hsv, lower_green, upper_green)

            output = cv2.bitwise_and(hsv,hsv,mask=image_mask)

            is_success, original_buffer = cv2.imencode(".jpg", frame)
            if is_success:
                original_io_buf = BytesIO(original_buffer)
                original_image_widget.value = original_io_buf.getvalue()
            else:
                print("原始图像编码失败")

            # 编码过滤后的图像（掩码之后）
            is_success, filtered_buffer = cv2.imencode(".jpg", cv2.cvtColor(output,
                cv2.COLOR_HSV2RGB))
            if is_success:
```

```
44                 filtered_io_buf = BytesIO(filtered_buffer)
45                 filtered_image_widget.value = filtered_io_buf.getvalue()
46             else:
47                 print("过滤图像编码失败")
48 
49     except KeyboardInterrupt:
50         print("用户中断")
51     finally:
52         cap.release()          # 释放摄像头资源
```

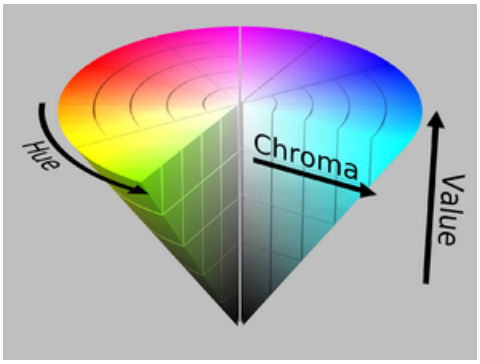


图 5.2: HSV 色彩空间

5.3.2 其它图像处理技术

图像还有一些基本处理包括旋转、缩放、透视、分割、混合等，其本质是对图像的矩阵进行变换，例如缩放时需要通过插值来估计新的像素值，混合则是将多幅图像合成为一幅图像。OpenCV 提供了丰富的图像处理函数，如`bitwise_and()`、`add()`、`resize()`等，具体可以参考 OpenCV 中提供的例子[2]。

5.4 图像形状检测

图像中不同形状的区分通常依赖颜色与灰度（强度）的变化，计算机识别形状的关键在于提取这些变化信息，在图像处理中常用梯度（gradient）来描述。梯度的计算与应用是形状检测的基础。在进行图像形状检测之前，通常需要对图像做灰度化、滤波、二值化等预处理。

5.4.1 图像滤波运算

在计算图像的梯度前，通常先采用滤波方法降低噪声的影响。图像滤波类似信号处理中的卷积运算，主要区别在于它是对二维数据进行的运算，能够有效抑制图像中的噪声。式 5.1 所示为一个 3×3 的归一化均值平滑滤波器，OpenCV 中提供了 `filter2D()`、`blur()` 等滤波函数，深入理解滤波运算是图像处理的基础。此外，OpenCV 还提供了边缘检测函数（如 `Canny()`）和轮廓检测函数（如 `findContours()`），将在后文介绍。

$$K = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \tag{5.1}$$

5.4.2 图像的梯度(gradient)计算

在 OpenCV 中，图像梯度的计算也可以看作是一种特殊的滤波运算。OpenCV 提供了多种梯度运算函数，如 Sobel()、Scharr()、Laplacian() 等：Sobel() 能够获得指定方向上的图像梯度值，Scharr() 是对 Sobel() 的进一步优化；Laplacian() 则计算图像在 x、y 两个方向上二阶导数之和（拉普拉斯算子）$\Delta src = \frac{\partial^2 src}{\partial x^2} + \frac{\partial^2 src}{\partial y^2}$，其核函数[1]如式 5.2 所示。通过计算图像的梯度可以实现对图像的边缘检测。

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \tag{5.2}$$

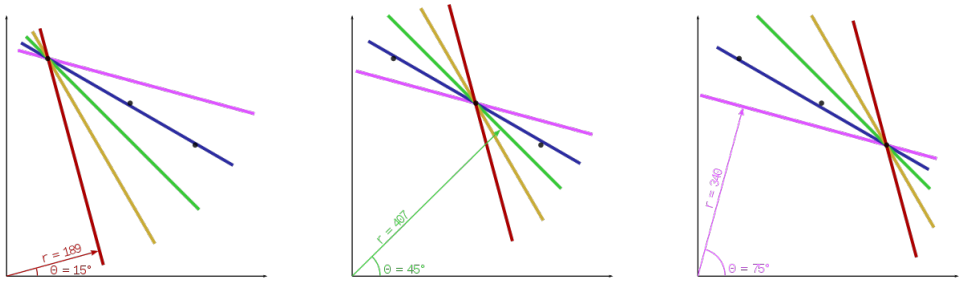
根据实际应用场景及检测需求，图像形状检测主要可以基于颜色、边缘与轮廓等信息进行。Hough 变换支持对直线、圆形等规则形状的检测；对于复杂形状，OpenCV 提供了 `findContours()` 来完成不规则形状的识别。下面分别介绍其基本原理及用法。

5.4.3 特定形状检测

对于直线、圆形等特殊形状，OpenCV 提供了基于 Hough 变换的特定检测方法。Hough 变换的基本原理是将图像中的点映射到另一个参数空间，如图 5.3 所示。对图中的每个点依次进行 Hough 变换：经过点 (x, y) 的每条直线都有唯一的 (r, θ) 与之对应，当直线绕该点旋转时，r 随 θ 按正弦规律变化。对图像中的所有点都进行 Hough 变换并累加后，位于同一直线上的点在 (r_0, θ_0) 处的计次值最大，据此即可检测出该直线。

类似地，对于圆形的识别，根据圆形满足的方程 $(x - x_0)^2 + (y - y_0)^2 = r^2$，其中 (x_0, y_0) 为圆心，r 为圆的半径。在已知半径 r_0 的情况下，以每个边缘点为圆心、r_0 为半径对候选圆心位置进行投票计次，计次最多的位置即为圆心，如图 5.4 所示；再根据确定的圆心确定半径。

由 Hough 变换的基本原理可以看出，它通过不同空间之间的映射完成特殊形状的检测，但运算量较大。因此通常先对图像进行灰度化和边缘检测（如 `Canny()`）等预处理，仅让边缘点参与投票，以降低运算量。



θ	r
15	189.0
30	282.0
45	355.7
60	407.3
75	429.4

θ	r
15	318.5
30	376.8
45	407.3
60	409.8
75	385.3

θ	r
15	419.0
30	443.6
45	438.4
60	402.9
75	340.1

图 5.3: 直线的 Hough 变换[3]

[1] 核函数：此处指滤波（卷积）核，即一个小的权重矩阵。它与图像的局部邻域逐元素相乘并求和，得到输出像素值；不同的核对应不同的滤波效果，例如均值核用于平滑、拉普拉斯核用于边缘检测。

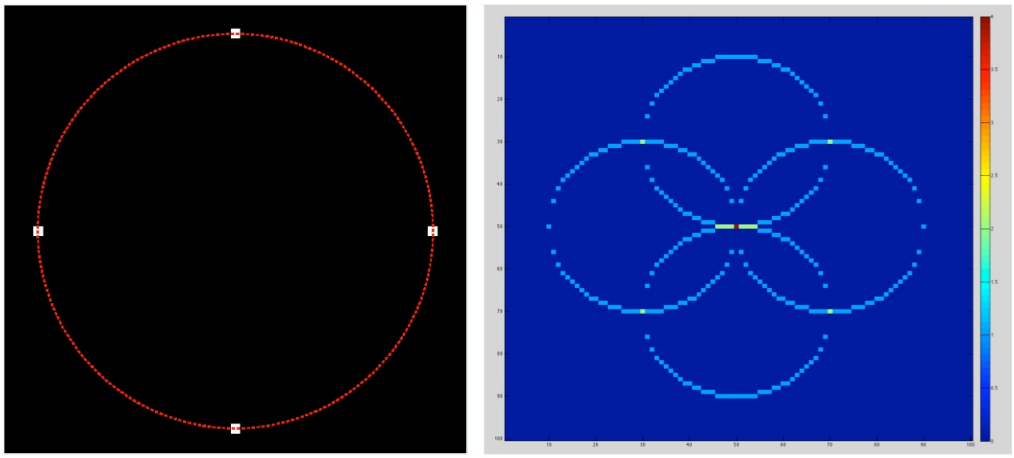


图 5.4: 圆形 Hough 变换[3]

5.4.4 轮廓检测(contours)

对于更为复杂的形状，可以使用 OpenCV 提供的轮廓检测函数 `findContours()` 及轮廓绘制函数 `drawContours()` 进行检测和识别。该检测函数主要针对二值图像,因此在调用之前需要采用 `threshold()` 函数对图像进行二值化处理。下面是一个轮廓检测的例子。

```
contours, _ = cv.findContours(thresh, cv.RETR_LIST,
                              cv.CHAIN_APPROX_SIMPLE)
if not contours:
    raise RuntimeError('未检测到轮廓')
cnt = max(contours, key=cv.contourArea)

# the moment returns a dictionary
M = cv.moments(cnt)
print( M )

# the centroid of the contour
if M['m00'] != 0:
    cx = int(M['m10'] / M['m00'])
    cy = int(M['m01'] / M['m00'])
else:
    cx, cy = None, None

# the area of the contour
area = cv.contourArea(cnt)

# the Perimeter of the contour
perimeter = cv.arcLength(cnt,True)
```

在图像的轮廓被检测出来后,轮廓的面积、周长、中心点等是描述轮廓的重要参数;此外,通常还会用一个标准形状(矩形、圆形、椭圆等)来近似表示轮廓。OpenCV 中提供了相应的`moments()`、`contourArea()`、`arcLength()`函数来计算轮廓的相关参数。

5.5 动手实验:图像处理及图形识别

5.5.1 实验目的

1. 掌握 OpenCV 处理图像的基本用法
2. 掌握从摄像头获取图像数据的基本方法
3. 学习从图像中提取特定图形

5.5.2 预习要求

1. 阅读讲义中提到的 OpenCV 参考文档
2. 理解在图像中识别直线和圆形的基本算法
3. 编写实验要求的程序代码

5.5.3 实验内容

5.5.3.1 图像的颜色处理

将摄像头连接到开发板的 USB 接口,参考讲义代码完成 HSV 色彩空间的转换。通过调整参数尝试将皮肤颜色过滤出来,使图像中仅保留人手所在的区域。

5.5.3.2 图像的透视变换

使用开发板连接的 USB 摄像头拍摄一张自己的校园卡或身份证的照片。利用 OpenCV 对该照片进行矫正处理。Perspective Transformation(透视变换)是 OpenCV 中的一个功能,它允许将图像从一个平面映射到另一个平面,常用于校正图像的透视畸变或者提取特定区域的图像[4]。

使用透视变换需要先定义原始图像中的四个顶点坐标以及目标图像中的对应顶点坐标:

```
import cv2 as cv
import numpy as np
original_points = np.float32([[x1, y1], [x2, y2], [x3, y3], [x4, y4]])
target_points = np.float32([[x1_prime, y1_prime], [x2_prime, y2_prime],
    [x3_prime, y3_prime], [x4_prime, y4_prime]])
# 计算透视变换矩阵:
perspective_matrix = cv.getPerspectiveTransform(original_points, target_points)
# 进行透视变换:
output_image = cv.warpPerspective(image, perspective_matrix, (width, height))
```

这里的 width 和 height 是输出图像的尺寸。

5.5.3.3 图像形状的识别

- 使用 Hough 变换来识别照片中的圆形使用 Hough 变换来识别圆形主要有以下几个步骤:

  1. 转化为灰度图像
  2. 图像滤波(去噪)
  3. 使用 HoughCircles() 来寻找圆形
  4. 在图像中将圆形标记出来

  参考代码如下所示:

```
import numpy as np
import cv2 as cv
from IPython.display import Image, display

img = cv.imread('opencv.png', cv.IMREAD_GRAYSCALE)
if img is None:
    raise FileNotFoundError('找不到图像文件 opencv.png')

blurred = cv.medianBlur(img, 5)
result = cv.cvtColor(img, cv.COLOR_GRAY2BGR)
circles = cv.HoughCircles(blurred, cv.HOUGH_GRADIENT, 1, 20,
    param1=50, param2=30, minRadius=0, maxRadius=0)

if circles is not None:
    circles = np.uint16(np.around(circles))
    for x, y, radius in circles[0]:
        cv.circle(result, (x, y), radius, (0, 255, 0), 2)
        cv.circle(result, (x, y), 2, (0, 0, 255), 3)
else:
    print('未检测到圆')

cv.imwrite('opencv_dec.png', result)
ok, encoded = cv.imencode('.png', result)
if ok:
    display(Image(data=encoded.tobytes()))
```

- 使用findContours()函数来识别轮廓

  使用findContours()函数来识别图像中的轮廓，并在原图像中将识别出的结果标记出来，另外计算轮廓的中心、周长、面积等参数。需要注意的是findContours()需要对图像进行二值化处理。参考代码如下所示：

```
import cv2 as cv
from IPython.display import Image, display

im = cv.imread('pku_logo.png')
if im is None:
    raise FileNotFoundError('找不到图像文件 pku_logo.png')

imgray = cv.cvtColor(im, cv.COLOR_BGR2GRAY)
# pku_logo.png 为白底深色图案，用 THRESH_BINARY_INV 使 logo 变为白色（前景）
_, thresh = cv.threshold(imgray, 127, 255, cv.THRESH_BINARY_INV)
contours, _ = cv.findContours(
    thresh, cv.RETR_TREE, cv.CHAIN_APPROX_SIMPLE)

result = im.copy()
if contours:
    largest = max(contours, key=cv.contourArea)
    cv.drawContours(result, [largest], -1, (0, 255, 0), 3)
```

```
else:
    print('未检测到轮廓')

cv.imwrite('pku_logo_contour.png', result)
ok, encoded = cv.imencode('.png', result)
if ok:
    display(Image(data=encoded.tobytes()))
```

5.5.4 注意事项

1. Numpy 库的使用

   Numpy 库主要用于 Python 数组的运算，也是 OpenCV 图像处理的基础。Numpy 数组的主要属性有 ndim、shape、size、dtype、itemsize 等，Numpy 提供了丰富的处理函数及矩阵运算函数，如`arange()`、`zeros()`、`reshape()`、`sort()`等。Numpy 库的使用可参考 [5]。

2. 数据位数扩展

   图像像素通常以 uint8 存储；在进行梯度、滤波等运算时，为保持计算精度，通常先将数据扩展为更高精度的类型（如 64 位浮点数），计算完成后再转回 uint8。

3. OpenCV 中性能的评估

   为评估算法的运算性能，OpenCV 提供了对应的函数，如 `getTickCount()`、`getTickFrequency()` 等，可通过统计执行时间来评估算法的性能。

5.6 思考题

1. 对图像进行滤波处理与信号处理中的滤波有何异同？其核函数如何设计？
2. Hough 变换的基本原理是什么？其他领域中有没有类似的处理方法？

第六章　支持向量机分类器

本章我们将使用支持向量机实现手写数字的识别，识别的图像和最终的结果将使用开发板的 OLED 显示屏来显示。图像识别所使用的数据集采用公开数据，而通过摄像头识别数字的任务将留给读者完成。

6.1　支持向量机

对于给定的样本集，分类算法的目的就是要找到一个划分超平面，将不同类别的样本分开，但能实现这个目的的超平面可能有很多，要如何确定最好的那一个呢？图6.1 是一个二维的例子，可以有不同的直线将两类样点分开。直观的看粗实线的分类效果最好，样点到直线的最短距离是最大，而且对于不同的样点相等。

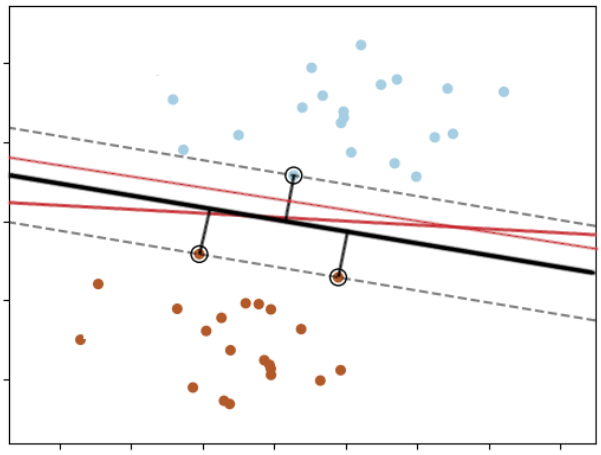


图 6.1: 支持向量示例

支持向量机（Support Vector Machine）就是利用这一数学特征寻找最优超平面的算法，距离超平面最近的几个训练样本（图6.1 中圈出的三个点）被称为支持向量。最后找到的超平面是可只根据支持向量来确定，因此 SVM 方法往往在样点数比较少的时候表现也很好，训练速度也比较快。

6.1.1　支持向量机的基本型

对于样本数据集 $(\mathbf{x}_i, y_i), y_i \in +1, -1$，划分超平面可以表示为：

$$f(mathbfx) = \mathbf{w}^T\mathbf{x} + b = 0 \tag{6.1}$$

样本空间中任意点到该超平面的距离可以写为：

$$r = \frac{\mathbf{w}^T\mathbf{x} + b}{||\mathbf{w}||} \tag{6.2}$$

如果这个超平面可以正确的对样本进行分类，可以令：

$$\begin{cases} \mathbf{w}^T\mathbf{x} + b \geq +1, & y_i = +1; \\ \mathbf{w}^T\mathbf{x} + b \leq -1, & y_i = -1. \end{cases} \tag{6.3}$$

其中使等号成立的样点就是支持向量，两个异类支持向量到超平面的距离和为：

$$\gamma = \frac{2}{\|\mathbf{w}\|} \tag{6.4}$$

寻求最佳分割超平面的过程就是最大化 γ 的优化问题，即：

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2, \quad \text{s.t.} \quad y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1, \quad i = 1, 2, \ldots, m \tag{6.5}$$

利用拉格朗日乘子法可以得到这个问题的等价问题，其拉格朗日函数可以写为：

$$L(\mathbf{w}, b, \alpha) = \frac{1}{2}\|\mathbf{w}\|^2 + \sum_{i=1}^{m} \alpha_i(1 - y_i(\mathbf{w}^T\mathbf{x}_i + b)), \quad (\alpha_i \geq 0) \tag{6.6}$$

令 $L(\mathbf{w}, b, \alpha)$ 对 $\mathbf{w}$ 和 b 的偏导为零可以解出 α_i 的值，同时由于约束条件为不等式，α_i 还必须满足 KKT（Karush-Kuhn-Tucker）条件，即：

$$\begin{cases} \alpha_i \geq 0; \\ y_i f(x_i) - 1 \geq 0; \\ \alpha_i(y_i f(x_i) - 1) = 0. \end{cases} \tag{6.7}$$

于是对于任意样本 (x_i, y_i)，若 $\alpha_i = 0$ 则该样本对 $f(x_i)$ 没有影响，若 $\alpha_i > 0$ 则必有 $y_i f(x_i) = 1$，即样点在最大间隔边界上，是一个支持向量。

6.1.2 软间隔

如果训练样本并不是线性可分，前面介绍的算法就会失效，而缓解这一问题的一个方法就是采用“软间隔”，允许部分样本不满足约束条件 $y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$，如图 6.2 所示，虽然在部分样本上会分类错误，但能获得一个可行的分类方案。

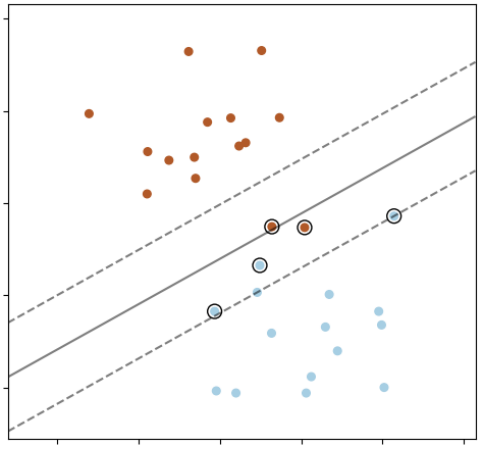


图 6.2: 软间隔示意图

此时的优化目标为

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{m} l_{0/1}(y_i(\mathbf{w}^T\mathbf{x}_i + b) - 1) \tag{6.8}$$

其中 $l_{0/1}$ 定义如下，被称为 “0/1 损失函数”：

$$l_{0/1}(z) = \begin{cases} 1, & \text{if } z < 0; \\ 0, & \text{otherwise.} \end{cases} \tag{6.9}$$

其中 $C > 0$ 是一个常数，其取值越小，约束对优化目标的贡献越小，可以有更多的样点不满足约束条件，而当 C 取值为无穷大时，优化目标等效于 “硬间隔”，所有样点都要满足约束条件。

然而 $l_{0/1}$ 函数非连续、非凸，不易进行数学分析。于是通常会用一些其他函数进行替代，常用的替代损失函数有：

- hinge 损失：$l_{hinge}(z) = max(0, 1 - z)$
- 指数损失：$l_{exp}(z) = exp(-z)$
- 对数损失：$l_{log}(z) = log(1 + exp(-z))$

6.1.3 核函数

对于训练样本线性不可分的情况，还可以将样本从原始空间映射到一个更高维的特征空间，使得这些样本在新的特征空间内线性可分。数学上可以证明如果原始空间是有限维度，那么一定存在一个高维特性空间使得样本线性可分。令 $\phi(\mathbf{x})$ 为映射函数，则变换后的模型可表示为：

$$f(\mathbf{x}) = \mathbf{w}^T \phi(\mathbf{x}) + b \tag{6.10}$$

在通过拉格朗日方程求解的过程中，会涉及到高维空间的内积计算，但由于特征空间的维数可能很高，甚至是无穷维，直接计算内积通常很困难，因此定义如下的核函数：

$$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j) \tag{6.11}$$

有了核函数就可以不必设计 $\phi()$ 函数，因为仅靠核函数就可以完成计算并且根据下面的定理可以更容易的设计核函数。

令 χ 为输入空间，$\kappa()$ 为定义在 $\chi \times \chi$ 上的对称函数，则 κ 是核函数当且仅当对于任意数据集 $D = \mathbf{x}_1, \ldots, \mathbf{x}_m$，“核矩阵”$\mathbf{K}$ 总是半正定的。‘

$$\mathbf{K} = \begin{bmatrix} \kappa(\mathbf{x}_1, \mathbf{x}_1) & \ldots & \kappa(\mathbf{x}_1, \mathbf{x}_j) & \ldots & \kappa(\mathbf{x}_1, \mathbf{x}_m) \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ \kappa(\mathbf{x}_i, \mathbf{x}_1) & \ldots & \kappa(\mathbf{x}_i, \mathbf{x}_j) & \ldots & \kappa(\mathbf{x}_i, \mathbf{x}_m) \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ \kappa(\mathbf{x}_m, \mathbf{x}_1) & \ldots & \kappa(\mathbf{x}_m, \mathbf{x}_j) & \ldots & \kappa(\mathbf{x}_m, \mathbf{x}_m) \end{bmatrix} \tag{6.12}$$

常用的核函数有如下几个：

- 线性核 $\kappa(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$
- 多项式核 $\kappa(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^T \mathbf{x}_j)^d$, $d \geq 1$ 为多项式的次数
- 高斯核 $\kappa(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma^2})$, $\sigma > 0$ 为高斯核的带宽
- 拉普拉斯核 $\kappa(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{\sigma})$, $\sigma > 0$
- Sigmoid 核 $\kappa(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\beta \mathbf{x}_i^T \mathbf{x}_j + \theta)$

6.2 用到的 Python 库

6.2.1 scikit-learn 中的 SVM

scikit-learn 是一套包含许多机器学习算法的 Python 库，包括前面介绍 KMeans 还有这节课介绍的 SVM。这里先看它的一个 SVM 示例代码。`datasets.load_digits()` 返回的是 scikit-learn 自带的手写数字小型数据集，无需另行下载。该数据集共有 1797 张 8×8 的 16 级灰度图像，标签为数字 0–9；每张图像展开后得到一个长度为 64 的特征向量。

```
import matplotlib.pyplot as plt
from sklearn import datasets, svm, metrics

digits = datasets.load_digits() # 1797 张 8x8、16 级灰度图像

# digits.images 是图片数据，digits.target 是标签
images_and_labels = list(zip(digits.images, digits.target))
# 使用 matplotlib 绘制前4个样本
for index, (image, label) in enumerate(images_and_labels[:4]):
    plt.subplot(2, 4, index + 1)
    plt.axis('off')
    plt.imshow(image, cmap=plt.cm.gray_r, interpolation='nearest')
    plt.title('Training: %i' % label)

n_samples = len(digits.images)
# 将二维数据变成一维
data = digits.images.reshape((n_samples, -1))

# 建立分类器
classifier = svm.SVC(gamma=0.001)

# 用前一半数据进行训练
classifier.fit(data[:n_samples // 2], digits.target[:n_samples // 2])

expected = digits.target[n_samples // 2:]
# 预测
predicted = classifier.predict(data[n_samples // 2:])

print("Classification report for classifier %s:\n%s\n"
      % (classifier, metrics.classification_report(expected, predicted)))
print("Confusion matrix:\n%s" % metrics.confusion_matrix(expected, predicted))

# 使用 matplotlib 绘制前4个预测样本
images_and_predictions = list(zip(digits.images[n_samples // 2:], predicted))
for index, (image, prediction) in enumerate(images_and_predictions[:4]):
    plt.subplot(2, 4, index + 5)
    plt.axis('off')
    plt.imshow(image, cmap=plt.cm.gray_r, interpolation='nearest')
    plt.title('Prediction: %i' % prediction)

plt.show()
```

这段代码的功能是对手写数字图像进行分类。datasets 模块返回的数据中包含手写数字的图像和标签，程序使用前一半数据进行训练，然后用后一半对训练结果进行验证，并使用 matplotlib 显示了部分训练样本和预测结果。程序的功能可以通过所带的注释大致了解，部分函数的说明可以参考网上 scikit-learn 的文档。

6.2.2 Python 图像支持库

Python 有很多可以进行图像处理的支持库，其中 PIL（Python Image Library）是历史比较悠久的一个，使用的人很多。但 PIL 仅支持 Python2，一个它的分支 Pillow 完成了对 Python3 的支持。PIL 中最重要的类就是 Image，使用它可以打开已有的图片或创建新的图片进行后续处理。下面的代码可以获取已有图像的基本信息。

```
1  from PIL import Image # 注意依然使用 PIL
2
3  image = Image.open("example.jpg")
4  print('width:', image.width)
5  print('height:', image.height)
6  print('size:', image.size)
7  print('mode:', image.mode)
8  print('format:', image.format)
```

width 表示图片的像素宽度，height 表示图片的像素高度，width 和 height 组成了 size 属性，size 是包含两个元素的元组。mode 属性表示图片的模式，表6.1 是常见的几种图像模式。

表 6.1: PIL 颜色模式

模式	描述
1	1 位像素，黑白图片
L	8 位像素，灰度图片
P	8 位像素，使用调色板定义颜色
RGB	24 位像素，真彩色
RGBA	32 位像素，真彩色加透明度模板
CMYK	32 位像素，色彩分离（常用于印刷）
HSV	24 位像素，色相、饱和度、颜色空间

不同的模式之间可以用 convert 函数转换，例如下面的代码将图像转为灰度图片。

```
1  from PIL import Image
2
3  image = Image.open("example.jpg")
4  image1 = image.convert('L')
```

图片的裁剪和缩放可以用 crop 和 resize 函数，例如：

```
1  from PIL import Image
2
3  image = Image.open("example.jpg")
4  image_crop = image.crop(box=(100,100,800,600))
```

```
5     image_resize = image.resize((400, 300))
```

其中 crop 函数通过 box 参数表示裁剪的区域，元组的四个参数分别是左上和右下的像素坐标（图片的左上角坐标为(0,0)）。

新建一个图片可以用 new 函数，利用 ImageDraw 类提供的函数可以进行简单图形的绘制。例如：

```
1     from PIL import Image, ImageDraw
2
3     image = Image.new('RGB', (400, 300), 'black')
4     draw = ImageDraw.Draw(image)
5     draw.line((0, 0, 400, 300), fill='red')  # 画一条红色对角线
6     image.show()  # 调用外部程序显示图片
```

new 函数的三个参数分别是图像的模式、尺寸和颜色，上面程序的执行结果是在黑色的背景下绘制了一条红色直线。

如果要在图片上生成字符串，可以使用 ImageFont 类选择字体，使用 text 函数绘制，如下代码所示：

```
1     from PIL import Image, ImageDraw, ImageFont
2
3     image = Image.new('RGB', (400, 300), 'black')
4     font = ImageFont.load_default()
5     draw = ImageDraw.Draw(image)
6     draw.text((0,0), 'This␣is␣first␣line', font=font, fill=255)
```

6.3 动手实验：手写数字识别

6.3.1 实验目的

1. 了解 SPI 总线传输的原理。
2. 了解 OLED 设备基本原理。
3. 掌握支持向量机的基本原理

6.3.2 预习要求

1. 阅读 2.2.4 小节，了解 SPI 的基本原理
2. 阅读 OLED 示例程序
3. 掌握本章介绍 SVM 原理，编写实验要求代码

6.3.3 实验步骤

6.3.3.1 OLED 设备测试

1. 通过 SPI 设备访问 OLED，在显示屏上显示 “hello world!”。
2. 使用 OLED 显示模块显示一幅图片

6.3.3.2 验证支持向量机示例代码

将示例代码在开发板上运行，查看运行结果，理解程序输出的评价指标。试着修改使用不同的核函数，查看运行结果。

6.3.3.3 使用 OLED 显示结果

将预测的图像和预测结果都显示在 OLED 上，每次显示一副图像，通过扩展板上的按键控制显示下一副图像。

由于 OLED 显示屏是单色显示，分辨率也比提供的图像高，因此需要利用 Image 库中的 resize 函数转换一下，示例如下：

```
1    # kk 中保存了图像的原始数据
2    digit = Image.fromarray((kk*8).astype(np.uint8), mode='L').resize((48,48)).
         convert('1')
3    img = Image.new('1',(disp.width,disp.height),'black')
4    img.paste(digit, (0, 16, digit.size[0], digit.size[1]+16))
5    disp.clear()
6    disp.image(img)
7    disp.display()
```

6.3.3.4 使用摄像头读入手写数字

在白纸上写一位数字，通过摄像头读入，并处理成符合算法输入的格式，通过训练好的模型进行识别。

6.4 思考题

1. 为什么不把全部的数据都用来做训练呢？然后在训练数据集上进行验证效果一定会更好！
2. 将具有 16 级灰度的图像显示在单色屏上，能保证显示效果的原理是什么？

第七章　构建专家系统

本章我们将使用开发板实现一个简单的专家系统，这个系统可以通过动物所具有的特征识别出动物的名称。在这个过程中，我们将学习专家系统的基本知识，学会使用 Prolog 语言编写简单的动物识别逻辑。最后我们使用 Python 语言编写用户的交互界面，并通过舵机将识别的结果输出。

7.1　专家系统

7.1.1　专家系统简介

所谓的专家系统（Expert System）是一种利用知识进行推理的程序。专家系统具有某些领域的知识的规则库（rules），例如鸟类识别专家系统关于几种鸟类的规则库可能是

喜鹊是留鸟、颜色黑白、体型中等
白鹭是旅鸟、颜色白、体型大
翠鸟是留鸟、颜色蓝、体型小

当给出一些事实（facts），专家系统可以利用它们进行推理，得到特定的结论。例如给出下面的查询条件：

居留类型：留鸟
颜色：蓝
体型：小

专家系统就可以得出查询的鸟是翠鸟的答案。

当然前面的示例过于简单，并不能体现专家系统的优势。一个复杂的规则库和完备的推理程序可以完成只有人类专家才可以做到的任务，这类任务往往规则库非常复杂，人类需要多年的学习和积累经验才能完成，这也是为什么这类软件被称为专家系统的原因。常用的应用方向有工程、医药、军事等。

编写专家系统一般需要特殊的编程语言，这类语言具有逻辑推理机制，可以提高编程的效率，减少程序出错的概率。其中比较有代表性的就是 Prolog。

7.1.2　Prolog 编程语言

Prolog 是一种逻辑型程序设计语言，它的英文就是 Programming in Logic 的缩写。它具有自动搜索、模式匹配、回溯等性能，主要特点是以谓词逻辑为理论基础。用 Prolog 编程时，人们注重和关心对问题的描述，而不是问题的求解过程。例如前面小节中的规则库和事实库可以用 Prolog 描述如下：

```
bird(magpie) :- color(black_white), season(all_year), size(medium).
bird(egret) :- color(white), season(spring_autumn), size(large).
bird(kingfisher) :- color(blue), season(all_year), size(small).
season(all_year).
color(blue).
size(small).
```

Proglog 的这种编程设计思想被称为声明式编程语言（Declarative language），其它如 C 语言是命令式编程语言（Imperative language）。命令式编程语言通过具体的流程控制来描述程序的执行过程，而声明

式语言采用更高层的方式对问题进行描述，只表达出要达到的目标而由运行环境完成对目标问题的计算过程。

在 Prolog 的程序中，首字母小写的单词表示函数或者常量（Prolog 术语叫做 predicate，中文一般翻译为“谓词”，本文后续都用函数来指代），不用声明就可以直接使用。Prolog 每行语句由句点结束，包含 `:-` 的表示规则，代表蕴含的关系，例如 `bird(magpie)` 为真的条件是 `:-` 后面的三个函数都为真。`:-` 后面的内容用逗号分隔表示逻辑的与关系；如果采用分号分隔则表示逻辑或的关系；逻辑或的关系也可以写成独立的两个语句。不包含 `:-` 的语句为事实，例如 `color(blue)` 这个函数永远为真。

首字母大写的单词在 Prolog 中是变量，如果出现在程序中就组成了一个查询语句。例如对于上面的程序，`bird(X)` 就是一条查询，而根据前面定义的规则和事实，唯一符合条件的查询就是 `X = kingfisher`。

下划线 `_` 是一个特殊的变量，被称为匿名变量，它的作用域仅限于当前语句，也就是说程序中不同语句所使用的匿名变量没有关系。匿名变量在使用的时候往往只关注是否这个变量可以取到值，而不关心这个值到底是什么。因此对于上面的程序，`bird(_)` 给出的结果是 `true`，即表示这个变量可以取到值。

7.1.3 Prolog 的回溯算法

从算法的角度来看，Prolog 的解释器或编译器需要在设定的事实和规则的前提下，在定义域内搜索特定问题全部的解。这是一个被广泛研究的计算问题，一般通过回溯算法（深度优先搜索）的算法来获得答案。了解回溯的基本过程可以为编写高效率代码提供指导。Prolog 也提供了一些语法来优化搜索的过程。

例如定义一个求最大值的函数 `max(X,Y,Z)`，其中 `Z` 是前两个变量的最大值。可以这样写这个程序：

```
max(X,Y,Y) :- X =< Y.
max(X,Y,X) :- X > Y.
```

假定查询语句是 `max(2,3,Y)`，那么通过第一条规则就可以得出 `Y=3`，可 Prolog 还会去检查第二条规则试图找到其它的答案，而这显然是徒劳的。因为程序设计者了解这个函数只能有一个答案，如果第一条规则满足就不必检查第二条规则，此时可以使用 Prolog 的 cut 功能，用 `!` 符号阻止后续的回溯搜索。具体写法如下：

```
max(X,Y,Y) :- X =< Y,!.
max(X,Y,X) :- X > Y.
```

Prolog 在遇到 `!` 之后，前面搜索过程中的可回溯节点全部不再回溯，包括最开始检查第一条规则的节点，不再会去检查之后的规则了。

对于 `max` 函数，通过一次判断就应该可以得到结论，第二条规则的判断有些浪费。在 Prolog 中，可以用一种特殊的语法表达 `if-then-else` 的结构：`(A -> B ; C)` 表示对 `A` 进行检测，如果正确那么检测 `B`，如果不正确则检测 `C`。此时 `max` 函数可以写为：

```
max(X,Y,Z) :- (X =< Y -> Z = Y ; Z = X).
```

7.1.4 Prolog 内置函数

在程序中如果要在屏幕上输出内容，可以使用 Prolog 的 `write()`函数，例如可以用

```
write('Hello World!').
```

打印一条信息。`nl` 可以用来打印一个回车符，`tab(X)` 用来打印 `X` 个制表符。`read(X)` 用来从键盘输入，并把输入的内容赋值给 `X` 变量：

```
hello :-
    write('What is your name?'), read(X),
    write('Hello'), tab(1), write(X), nl.
```

使用 read 输入时要以句点结束，并注意不要大写首字母（表示一个变量）。

在 Prolog 中，true 永远返回正确，而 fail 永远返回错误。

大部分的自定义函数具有确定的状态，要么返回真，要么返回假。有时候程序需要一些函数的返回值可以改变，此时需要用 dynamic 函数来进行声明，声明的方式有两种：

```
:- dynamic my_predicate/2.
dynamic(my_predicate/2).
```

上面的声明定义了 my_predicate 函数为动态类型，并包含 2 个参数。Prolog 中都在函数名后面加斜线和数字的方式表明函数中的参数个数。

使用 assert(X) 或者 assertz(X) 函数可以在规则库的尾部中添加一条新的规则，而 asserta(X) 将新的规则添加到规则库的开头位置。retract(X) 函数将 X 从规则库中去除。但要注意，使用 assert 增加新的规则只适合动态函数，而从文件中加载的规则缺省都是静态函数。对于之前包含了 color(blue) 的规则，就无法用 assert(color(red)) 进行修改，但可以增加新定义的函数，例如 assert(bill(long))，这个 bill 函数就是动态类型。

7.1.5 swipl 简介

swipl 是 Prolog 的一个实现，下面的交互示例假定前面的鸟类识别代码保存在 bird.pl 中。查询结束后按 Ctrl+D 或者 halt. 命令退出程序。

```
$ swipl bird.pl
Welcome to SWI-Prolog (threaded, 32 bits, version 8.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit http://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

?- bird(kingfisher).
true.

?- bird(magpie).
false.

?- bird(X).
X = kingfisher.

?- halt.
```

swipl 包含一个 trace 命令，可以对代码的每一步搜索进行跟踪，不但便于了解 Prolog 语句执行的原理，也可以帮助分析程序中可能的错误。下面是 bird(X) 查询的跟踪结果。在跟踪结束后，可以使用 notrace 命令停止后续查询的跟踪。

```
?- trace.
true.

[trace]  ?- bird(X).
   Call: (10) bird(_7862) ? creep
   Call: (11) color(black_white) ? creep
   Fail: (11) color(black_white) ? creep
   Redo: (10) bird(_7862) ? creep
   Call: (11) color(white) ? creep
   Fail: (11) color(white) ? creep
   Redo: (10) bird(_7862) ? creep
```

```
       Call: (11) color(blue) ? creep
       Exit: (11) color(blue) ? creep
       Call: (11) season(all_year) ? creep
       Exit: (11) season(all_year) ? creep
       Call: (11) size(small) ? creep
       Exit: (11) size(small) ? creep
       Exit: (10) bird(kingfisher) ? creep
    X = kingfisher.
```

其中 Call 事件表示 Prolog 在试图满足一个条件的要求;Fail 事件表示条件不能满足;Redo 事件表示回溯到前一个节点寻找下一个满足条件的值;EXIT 事件表示当前条件已经满足。上面示例的跟踪过程可以描述为:首先试图为 bird(X) 找到一个答案,根据第一个规则,检查 X 是不是 magpie;根据这条规则还要检查 color 是否满足;color 条件无法通过;回溯到 bird(X) 寻找第二个答案,此时根据第二个规则,选择答案为 egret;……;经过一系列的搜索过程,最终获得答案为 kingfisher。

7.1.6 Prolog 代码示例

```
/* animal identification game.
    start with ?- go.     */

go :- hypothesize(Animal),
      write('I guess that the animal is: '),
      write(Animal),
      nl,
      undo.

/* hypotheses to be tested */
hypothesize(cheetah)   :- cheetah, !.
hypothesize(tiger)     :- tiger, !.
hypothesize(giraffe)   :- giraffe, !.
hypothesize(zebra)     :- zebra, !.
hypothesize(ostrich)   :- ostrich, !.
hypothesize(penguin)   :- penguin, !.
hypothesize(albatross) :- albatross, !.
hypothesize(unknown).             /* no diagnosis */

/* animal identification rules */
cheetah :- mammal,
           carnivore,
           verify(has_tawny_color),
           verify(has_dark_spots).
tiger :- mammal,
         carnivore,
         verify(has_tawny_color),
         verify(has_black_stripes).
giraffe :- ungulate,
           verify(has_long_neck),
           verify(has_long_legs).
zebra :- ungulate,
         verify(has_black_stripes).

ostrich :- bird,
           verify(does_not_fly),
           verify(has_long_neck).
penguin :- bird,
           verify(does_not_fly),
           verify(swims),
```

```
           verify(is_black_and_white).
albatross :- bird,
             verify(appears_in_story_Ancient_Mariner),
             verify(flys_well).

/* classification rules */
mammal    :- verify(has_hair), !.
mammal    :- verify(gives_milk).
bird      :- verify(has_feathers), !.
bird      :- verify(flys),
             verify(lays_eggs).
carnivore :- verify(eats_meat), !.
carnivore :- verify(has_pointed_teeth),
             verify(has_claws),
             verify(has_forward_eyes).
ungulate :- mammal,
            verify(has_hooves), !.
ungulate :- mammal,
            verify(chews_cud).

/* how to ask questions */
ask(Question) :-
    write('Does the animal have the following attribute: '),
    write(Question),
    write('? '),
    read(Response),
    ( (Response == yes ; Response == y)
      ->
       assert(yes(Question)) ;
       assert(no(Question)), fail).

:- dynamic yes/1,no/1.

/* How to verify something */
verify(S) :-
   (yes(S)
    ->
    true ;
    (no(S)
     ->
     fail ;
     ask(S))).

/* undo all yes/no assertions */
undo :- retract(yes(_)),fail.
undo :- retract(no(_)),fail.
undo.
```

这个程序就是本章开始提到的动物识别程序，在 swipl 下运行 `go`. 可以通过回答一系列问题来判断动物的名称。下面是一个运行的例子：

```
?- go.
Does the animal have the following attribute: has_hair? yes.
Does the animal have the following attribute: eats_meat? |: no.
Does the animal have the following attribute: has_pointed_teeth? |: no.
Does the animal have the following attribute: has_hooves? |: y.
Does the animal have the following attribute: has_long_neck? |: n.
Does the animal have the following attribute: has_black_stripes? |: y.
```

```
I guess that the animal is: zebra
true.
```

上面代码中的 verify 函数实现的功能就是对每个需要验证的事实进行判断，如果 yes(S) 成立，就返回 true，如果 no(S) 成立，就返回 false，否则通过 ask(S) 函数询问用户，得到判断的答案，把这个答案通过 yes() 和 no() 添加到知识库中。

7.1.7 Python 语言的 Prolog 接口

PySwip 是 Python 语言与 SWI-Prolog 的接口，通过这个接口可以实现与 Prolog 程序的联合编程。PySwip 需要调用 SWI-Prolog 的共享库，因此只安装 Python 包并不够。运行示例前，应先在开发板终端检查两个环境是否都可用：

```
$ swipl --version
$ python3 -c "from pyswip import Prolog; \
  print(list(Prolog().query('X is 2+3')))"
```

第二条命令应输出包含数字 5 的查询结果。若在 VS Code 中运行 Notebook，还要确认当前选择的 Python 内核就是安装了pyswip的环境。

使用库中的 consult 函数可以加载保存在文件中的规则，通过函数 assertz 可以添加需要查询的事实条件，通过函数 query 可以运行 Prolog 的查询语句。现在将 bird.pl 中的事实部分删除，保存为 birdrule.pl 运行如下 Python 程序，可以获得对 bird(X) 的查询结果。

```
1  from pyswip import Prolog       # 导入模块
2  prolog = Prolog()
3  prolog.consult('birdrule.pl')  # 导入 Prolog 的代码（规则库）
4  prolog.assertz("color(blue)")  # 添加事实
5  prolog.assertz('season(all_year)')
6  prolog.assertz('size(small)')
7  for result in prolog.query('bird(X)'): # query 代表查询
8      print(result["X"])
```

程序的查询结果是一个字典的列表，对于上面的示例则只有一个元素：{'X': 'kingfisher'}。

PySwip 还支持设计可以在 Prolog 中调用的外部函数，例如：

```
1  from pyswip import Prolog, registerForeign
2
3  def hello(t):           # 包含一个参数，返回值为布尔类型
4      print("Hello,", t)
5      return True          # Prolog 谓词执行成功
6  hello.arity = 1         # 这个属性是必须的
7
8  registerForeign(hello)  # 注册函数，使 Prolog 代码可以使用
9
10 prolog = Prolog()
11 prolog.assertz("father(michael,john)")  # 事实1: michael 是 john 的父亲
12 prolog.assertz("father(michael,gina)")  # 事实2: michael 是 gina 的父亲
13 print(list(prolog.query("father(michael,X), hello(X)"))) # 查询
```

程序运行结果为：

```
Hello, john
Hello, gina
[{'X': 'john'}, {'X': 'gina'}]
```

外部谓词应明确返回`True`或`False`，分别表示 Prolog 查询成功或失败。在同一个 JupyterLab 或 VS Code Notebook 内核中，已加载的规则和已注册的外部谓词会保留。修改 `.pl`文件或注册代码后，建议重启内核并按顺序重新运行单元，避免旧状态影响结果。

对于前面的动物识别程序，可以将 Prolog 代码中的 `verify` 等函数去掉，而改为用 Python 代码实现。也就是用 Python 编写 `verify` 函数，实现用户交互接口，显示需要验证的事实，输入用户的选择并将用户的选择作为布尔类型返回。函数可以通过一个字典记录每个回答的结果（相当于 Prolog 代码中的 `yes` 和 `no`，这样面对重复的问题就可以不必每次都询问用户。Python 代码的编写将在动手实验部分完成。

7.2 开发板的舵机控制

7.2.1 舵机简介

电机也称作马达，是电子设备中常用的动力原件。电机从使用用途上可以分为驱动电机和控制电机，其中驱动电机常通过持续的旋转为设备提供动力，例如电转、电动车、洗衣机等电子产品的动力大多由驱动电机提供。控制电机主要分为步进电机和伺服电机。步进电机是一种将电脉冲转化为角位移的执行机构，可以驱动电路控制转向固定的角度，经常用于精密控制，例如数控机床、硬盘驱动器等。伺服电机的可以通过电压控制电机的转速或力矩，要求电机的反应要快，经常用于各种运动控制系统中。

舵机是一种通过电信号控制转动角度的设备，可以看作是简单的伺服电机，由于经常应用于航模的舵面控制，因而被称为舵机。虽然和步进电机都是用来控制转动角度，但步进电机多用于负载不大，精度要求高的场合，而舵机往往需要输出较大的力矩，用于大负载的场景。

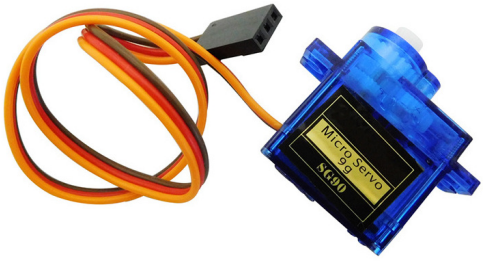


图 7.1: SG90 舵机

图 7.1 是一种比较常用的 9g 舵机（由于重量只有 9 克，所以一般称为 9 克舵机），它的主要指标如下：

- 工作电压：直流 4.8V 到 6V
- 工作电流：80mA 到 100mA
- 待机电流：5mA
- 扭力：1.3 到 1.7kg/cm
- 转速：0.09 到 0.10 秒每 60 度（4.8V）
- 信号周期：20 毫秒

舵机的连接线为三根，其中红色线为电源，棕色线为地线，黄色线为信号线。信号线的数据格式为 PWM，这与第 3.2.4 节中介绍的信号格式一致，只不过这里并不是用来输出功率，而是通过舵机内部的控制逻辑，将 PWM 信号转换为角度。由于 PWM 信号比较简单，很多处理器都有硬件的 PWM 控制器，可以通过一个 GPIO 管脚实现对舵机的控制。

控制舵机时，PWM 信号的周期为 20ms，当高电平时间为 0.5ms 时舵机输出的角度为 0 度，当高电平时间为 2.5 ms 时输出角度为 180 度（对于 90 度舵机则输出 90 度），占空比与输出角度为线性关系，通过计算可以得到占空比与角度的关系，下面示例程序展示了舵机在开发板上测试的代码。

```python
import time
from periphery import GPIO

SERVO_GPIO = 80
PWM_FREQ = 50  # 50 Hz = 20 ms period
PERIOD = 1.0 / PWM_FREQ  # 20 ms

def angle_to_pulse_width(angle):
    # 将角度转换为脉冲宽度
    return 0.0005 + (angle / 180.0) * 0.002

def set_servo(gpio, angle):
    pulse_width = angle_to_pulse_width(angle)
    gpio.write(True)
    time.sleep(pulse_width)
    gpio.write(False)
    time.sleep(PERIOD - pulse_width)

def main():
    gpio = GPIO(SERVO_GPIO, "out")
    print("Looping between 90° and 180° on SG90 (press CTRL+C to stop)...")

    try:
        while True:
            for _ in range(20):  # 发送 20 个脉冲 (400 ms)
                set_servo(gpio, 90)
            time.sleep(1)

            for _ in range(20):  # 发送 20 个脉冲 (400 ms)
                set_servo(gpio, 180)
            time.sleep(1)

    except KeyboardInterrupt:
        print("\nStopping servo control...")
    finally:
        gpio.write(False)
        gpio.close()

if __name__ == "__main__":
    main()
```

7.2.2 多舵机联合控制

对于机械臂、机器人等应用，舵机的数目一般较多，需要的电流比较大，无法通过开发板进行供电，此时一般选择使用专门的舵机驱动电路来驱动和控制舵机。一种常用的多舵机控制模块基于 PCA9685 芯片，可以同时支持 16 路舵机的控制。图 7.2 为该模块的外形。

PCA9685 模块的 V+ 接口可以用来接入外部电源驱动舵机，SDA，SCL 是开发板的 I^2C 接口。下面

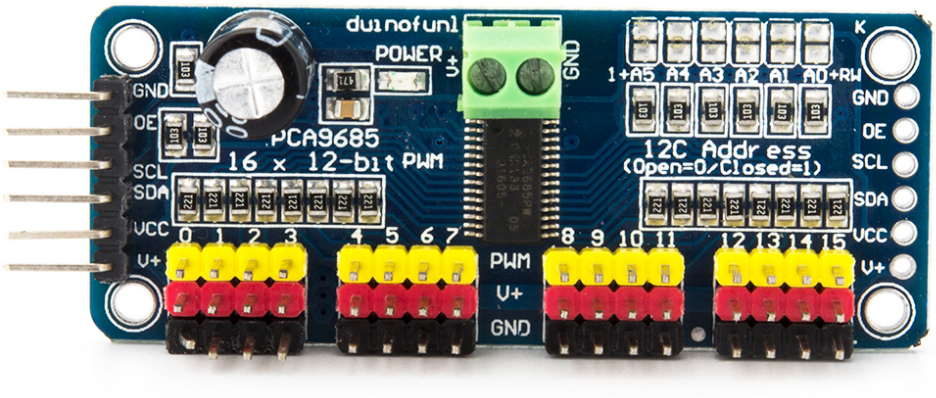


图 7.2: 16 路舵机驱动控制版

的参考代码用来操作第一个通道的舵机。路舵机。

```
from periphery import I2C
import time

I2C_BUS = "/dev/i2c-7"          # I2C总线设备
PCA9685_ADDR = 0x40             # PCA9685默认I2C地址
SERVO_FREQ = 50                 # 舵机标准工作频率
MAX_COUNT = 4095                # PCA9685的PWM分辨率
# 舵机脉冲宽度范围（不同品牌可能有差异，需校准）
MIN_PULSE_MS = 0.5              # 对应0°
MAX_PULSE_MS = 2.5              # 对应180°

MODE1_REG = 0x00                # 模式1寄存器（控制睡眠、自动增量等）
PRESCALE_REG = 0xFE             # 预分频器寄存器（设置PWM频率）
LED0_ON_L_REG = 0x06            # 通道0的ON时间低字节寄存器（每通道占4字节）

def pca9685_write_register(i2c, reg_addr, data):
    messages = [
        I2C.Message(bytes([reg_addr, data]))  # 第一个字节是寄存器地址，后续是数据
    ]
    try:
        i2c.transfer(PCA9685_ADDR, messages)
        return True
    except Exception as e:
        print(f"写入寄存器0x{reg_addr:02X}失败: {e}")
        return False

def pca9685_init(i2c):
    # 1. 唤醒芯片（关闭睡眠模式）
    # MODE1寄存器: 0x00 = 0b00000000（睡眠位bit4=0，自动增量bit5=0）
    if not pca9685_write_register(i2c, MODE1_REG, 0x00):
        raise RuntimeError("PCA9685唤醒失败，请检查接线和I2C地址")
    time.sleep(0.05)  # 等待芯片稳定唤醒
```

```
33
34      # 2. 设置PWM频率（50Hz）
35      # 频率计算公式: Prescaler = 25MHz / (4096 * 目标频率) - 1
36      prescaler = int(25000000 / (4096 * SERVO_FREQ) - 1)
37      # 进入睡眠模式才能修改预分频器（MODE1寄存器bit4=1）
38      pca9685_write_register(i2c, MODE1_REG, 0x10)
39      time.sleep(0.05)
40      # 写入预分频值
41      if not pca9685_write_register(i2c, PRESCALE_REG, prescaler):
42          raise RuntimeError("设置PWM频率失败")
43      # 退出睡眠模式，同时开启自动增量（bit5=1）: 0x20 = 0b00100000
44      pca9685_write_register(i2c, MODE1_REG, 0x20)
45      time.sleep(0.05)
46
47  def servo_set_angle(i2c, channel, angle):
48      pulse_ms = MIN_PULSE_MS + (angle / 180) * (MAX_PULSE_MS - MIN_PULSE_MS)
49      # 脉冲宽度→PCA9685计数映射（20ms是50Hz的周期）
50      count = int((pulse_ms / 20) * MAX_COUNT)
51
52      # 写入PWM数据（每个通道占4字节: ON_L, ON_H, OFF_L, OFF_H）
53      # 这里设置ON=0, OFF=count（即高电平持续count个计数周期）
54      reg_start = LED0_ON_L_REG + channel * 4  # 通道n的起始寄存器地址
55      # 构造4字节数据: ON_L=0x00, ON_H=0x00, OFF_L=count低8位, OFF_H=count高4位
56      data = [
57          0x00,                           # ON_L
58          0x00,                           # ON_H（12位计数，高4位有效）
59          count & 0xFF,                   # OFF_L（低8位）
60          (count >> 8) & 0x0F             # OFF_H（高4位，bit4-bit7保留0）
61      ]
62      # 构造I2C消息（寄存器地址+4字节数据）
63      messages = [
64          I2C.Message(bytes([reg_start] + data))
65      ]
66      try:
67          i2c.transfer(PCA9685_ADDR, messages)
68          return True
69      except Exception as e:
70          print(f"控制通道{channel}失败: {e}")
71          return False
72
73  if __name__ == "__main__":
74      try:
75          i2c = I2C(I2C_BUS)
76
77          pca9685_init(i2c)
78
79          # 舵机测试（通道0: 0°→90°→180°→90°→0°, 间隔2秒）
80          angles = [0, 90, 180, 90, 0]
81          for angle in angles:
```

```
82              servo_set_angle(i2c, channel=0, angle=angle)
83              time.sleep(2)
84
85      except Exception as e:
86          print(f"程序异常: {e}")
87      finally:
88          if i2c:
89              i2c.close()
```

7.3 动手实验:动物识别专家

7.3.1 实验目的

1. 了解专家系统的基本概念和设计方法
2. 掌握 Prolog 的基本语法
3. 学习使用 Python 语言编写 Prolog 接口
4. 学习使用开发板控制舵机

7.3.2 预习要求

1. 复习 GPIO 的用法
2. 理解 prolog 程序的基本结构和语法
3. 使用`swipl --version`和 Python 查询命令检查 SWI-Prolog、PySwip 及当前 Notebook 内核
4. 编写实验要求的程序

7.3.3 实验内容

7.3.3.1 测试示例 Prolog 代码

使用 swipl 运行本章介绍的 Prolog 代码，利用 trace 功能观察程序进行推理的过程，加深对 Prolog 实现的理解。特别对于动物识别代码，要充分理解 `verify` 函数的实现，便于接下来用 Python 进行实现。运行配套代码时，应进入`code/expert`目录，以保证 `animal.pl`、`bird.pl`和 Python 程序使用一致的相对路径。

7.3.3.2 使用 Python 完成专家系统

去掉 `animal.pl` 中 `go`，`ask`，`verify`，`undo` 的定义，利用 Python 代码实现 `verify` 函数，完成专家系统的实现。这里我们虽然使用命令行界面完成实验，与原来的 Prolog 代码运行效果区别不大。但 Python 的使用带来了更多的可能性，可以设计更友好的输入输出界面，使知识结构和用户接口分离开来，便于系统的升级。

7.3.3.3 使用舵机进行输出

这里的舵机作为一个转盘指针使用，可以将专家系统的输出结果直观的显示出来。采用扩展板上 5 向摇杆来替代键盘输入（如左键表示 n，右键表示 y）完成上述试验内容。后续可以将输入升级成语音，那这个系统就可以做成一个小盒子，成为一个很好玩的小玩具。

舵机和摇杆属于可选硬件。建议先设置`USE_HARDWARE = False`，完成命令行版专家系统；接好扩展板后再改为`True`。硬件初始化和循环应放在`try`语句中，并在`finally`中调用 GPIO 或 I^2C 对象的`close()`方法，使推理逻辑可以独立测试，程序中断时也能可靠释放资源。

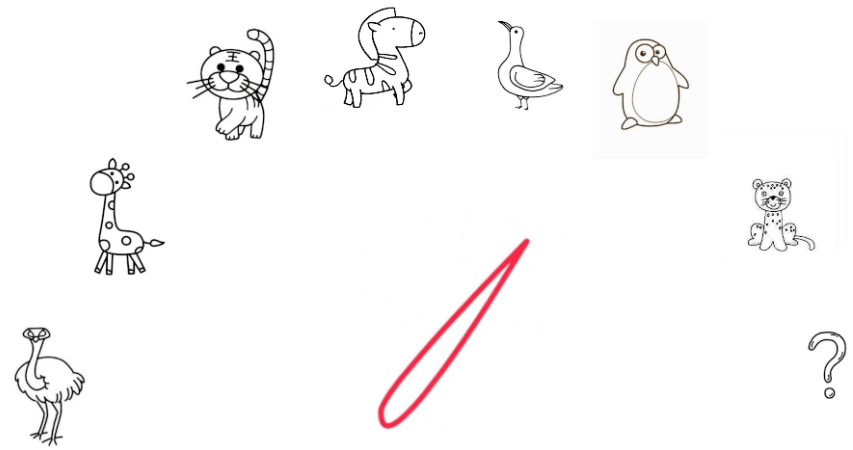


图 7.3: 动物识别的输出

7.4 思考题

1. Prolog 的使用场景并不仅限于专家系统，也可以应用在其他领域，通过查找资料说明。
2. 知识结构和用户接口分离的设计模式，你还可以举出类似例子吗？

第八章　OpenCV 进阶

在 OpenCV 基础中，介绍了 OpenCV 对图片的基本处理，并采用 Hough 变换实现了对规则形状的识别。本章在已有图片处理的基础上，进一步介绍人脸检测的基本原理，并采用 USB 摄像头完成人脸检测及运动物体跟踪。

8.1　图片的统计特性

在 OpenCV 基础中介绍了图片基本概念及图片的基本操作，如图片的颜色尺寸、颜色空间等。本章新一步介绍图片统计特性，如图片的直方图、反射投影直方图等基本概念，通过直方图均衡处理来实现对图片的处理，并为后续运动图像的处理奠定基础。

- 图片直方图

  直方图是对图片强度分布的描述，从统计学的角度来分析图片。直方图由 XY 二维表示，其中 X 值为像素值、Y 轴为落入对应 X 轴坐标区间内的图像中的对应像素数。OpenCV 中提供了直方图分析函数`calcHist()`，典型的用法如下所示。图 8.1 给出了直方图的示意。通过对直方图的均衡的处理，来实现对图片的处理，例如对图片的强度的均衡处理，OpenCV 提供了均衡函数`cv.equalizeHist()`。为进一步提升均衡性能，在全局均衡的基础上引入了局部自适应直方图均衡函数`cv.createCLAHE(),cv.clahe.apply()`。

```
1 img = cv.imread(filename,0)
2 hist = cv.calcHist([img],[0],None,[256],[0,256])
```

- 直方图反射投影

  Swain and Ballard 最早在论文 [6] 中提出了反向投影的概念，图片的直方图反应了图片在强度的概率分布。直方图反向投影用来实现图像分割，背景与对象分离，对已知对象位置进行定位。反向投影在模式匹配、对象识别、视频跟踪中均有应用，OpenCV 中经典算法之一 CAMeanShift 就是基于反向投影实现对已知对象的位置查找与标记、从而达到连续跟踪。OpenCV 提供了一个内置函数`cv.calcBackProject()`来实现反射投影，需要注意的是在传递给`calcBackProject()`函数之前，应该对象直方图采用函数`cv.normalize()`进行规范化。反射投影的 python 例子如下所示：

```
1 roi = cv.imread('pic_roi.png')
2 hsv = cv.cvtColor(roi,cv.COLOR_BGR2HSV)
3 target = cv.imread('pic.png')
4 hsvt = cv.cvtColor(target,cv.COLOR_BGR2HSV)
5 # calculating object histogram
6 roihist = cv.calcHist([hsv],[0, 1], None, [180, 256], [0, 180, 0, 256] )
7 # normalize histogram and apply backprojection
8 cv.normalize(roihist,roihist,0,255,cv.NORM_MINMAX)
9 dst = cv.calcBackProject([hsvt],[0,1],roihist,[0,180,0,256],1)
```

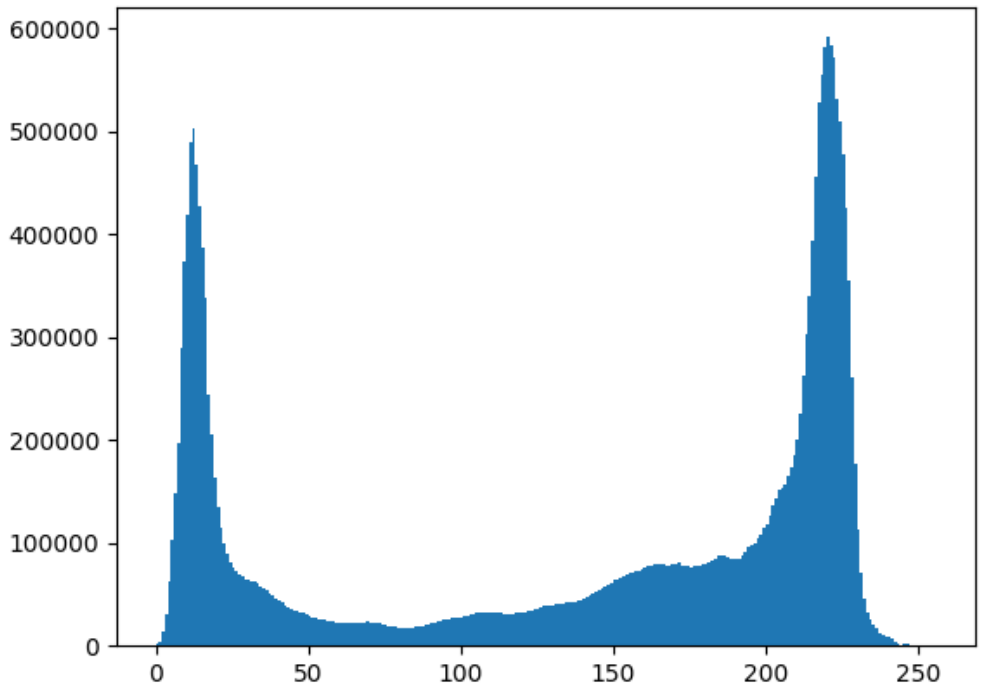


图 8.1: 图片的几种特征值示意

8.2 人脸检测算法基本原理

人脸检测的目标是确定图像中是否有人脸并给出其位置；人脸识别则要进一步判断人脸属于谁，二者不是同一个任务。对于给出的一张图片如何让机器检测出其中的人脸，许多研究者发展了基于颜色、运动和图像特征等方法[7]。2001 年 Viola 与 Jones 提出了基于图像特征的弱分类器级联检测算法，使实时人脸检测取得了重要进展。随着算力不断增强，基于神经网络的人脸检测方法也被提出，进一步提升了检测效果。

虽然 Viola–Jones 人脸检测算法存在对姿态、遮挡和光照较敏感等限制，但它计算量较低，适合在开发板上理解传统检测流程。OpenCV 提供了预训练的 Haar 级联检测器，本章用它介绍 Haar 特征、弱分类器与级联结构。

- Haar 特征计算

  Haar-feature 思想源自匈牙利数学家 Alfréd Haar 于 1904 年提出的 Haar 序列，Haar 序列与 Fourier 序列类似构成了空间的一序列正交基，通过一系列的正交基将对象分解出来。OpenCV 通过计算 Haar-feature 来实现对人脸的检测，论文 [8] 给出了算法的基本原理。图片 $\mathbf{I}(\mathbf{x},\mathbf{y})$ 的特征定义式如式 8.1 所示，其中 $\mathbf{I}$ 为图片，(x,y) 为像素点，$w(x,y) \in (-1,0,1)$ 为权值。

$$f(\mathbf{I}) = \begin{cases} 1, & if \ \sum_{x,y} w(x,y)\mathbf{I}(\mathbf{x},\mathbf{y}) > 0 \\ 0 & else. \end{cases} \tag{8.1}$$

  如图 8.2 所示，某一像素框中的特性定义为在矩形框内灰色部分的像素点的和与白色像素点和之差，通过特性的计算来实现对特性检测。图 8.2c 对应于人脸中眼睛的特征计算，眼睛在鼻子的两侧，眼睛的颜色比鼻子的颜色暗。

需要注意的是，根据上述的定义图片的特征的计算量是非常巨大的，为实现对图像特征的快速计算，OpenCV 通过对图像的积分运算快速实现了对图像特性的计算，像素点 (x, y) 积分的定义为 $ii(x, y) = \sum_{x' \le x, y' \le y} i(x', y')$，图 8.3 给出了面积 D 的图片积分的计算方式为 $D = I(4) - I(2) - I(3) + I(1)$。

- 简单分类器级联

  在解决图片特征值快速计算后，如何从众多的特征值中选择相关的特征值进行来进行图片的分类，AdaBoost 方法被用来构建简单的分类器实现了分类速度的快速提升。更进一步的，如何众多简单分类器的基础上进一步提升检测的性能，核心思想就是将简单的分类器级联，将上一级分类器的判决的结果作为下一个分类器的输入，通过逐级检测以提升检测的概率。其基本原理如图 8.4 所示。

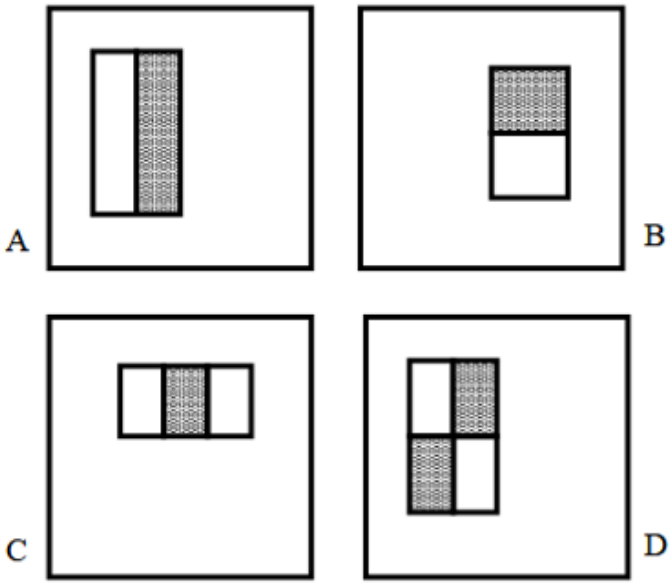


图 8.2: 图片的几种特征值示意

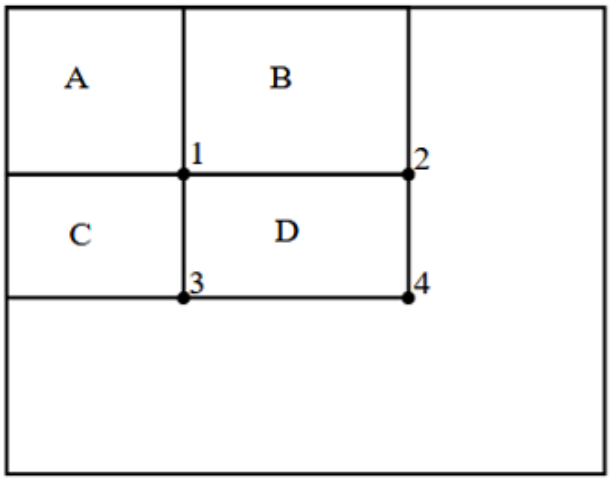


图 8.3: 图积分值快速计算示意图

下面代码给出了 OpenCV 中对人脸检测的代码，这里没有使用 IPython 的 display 模块，而是使用了 matplotlib 的显示功能来显示图片：

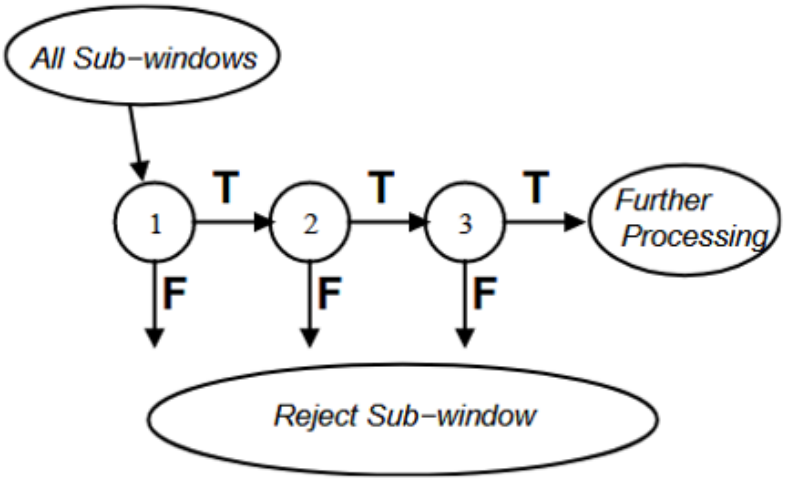


图 8.4: 级联检测器

```python
import cv2 as cv
import matplotlib.pyplot as plt

def detect_and_return_image(filename):
    model = cv.data.haarcascades + 'haarcascade_frontalface_default.xml'
    face_cascade = cv.CascadeClassifier(model)
    if face_cascade.empty():
        raise RuntimeError('Haar 人脸检测模型加载失败')

    img = cv.imread(filename)
    if img is None:
        print(f"无法读取图像: {filename}")
        return None

    gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, 1.3, 5)

    for (x, y, w, h) in faces:
        cv.rectangle(img, (x, y), (x+w, y+h), (255, 0, 0), 2)

    return cv.cvtColor(img, cv.COLOR_BGR2RGB)  # 返回 RGB 图像

result_img = detect_and_return_image('/path/to/my/pic.jpg')
if result_img is not None:
    plt.figure(figsize=(10, 6))
    plt.imshow(result_img)
    plt.title('Face Detected!!')
    plt.axis('off')
    plt.show()
```

在完成对人脸的检测后，可以在人脸区域内进一步检测眼睛。眼睛检测的原理与人脸检测类似，其主要代码如下。

```
eye_model = cv.data.haarcascades + 'haarcascade_eye.xml'
eye_cascade = cv.CascadeClassifier(eye_model)
if eye_cascade.empty():
    raise RuntimeError('Haar␣眼睛检测模型加载失败')
```

8.3 运动物体的追踪

运动物体的追踪主要有两种方式，分别为背景差分法（Background subtractor）和均值平移（Mean-shift and CAMShift）。

- 背景差分法

  背景差分法利用当前帧与背景帧的差别实现对运动物体的定位，如图 8.5 。

  背景差分法将每个当前帧均与背景帧进行差分，从而获取运动到图像中的物体或图像中运动的物体。因此，理论上背景帧不存在运动物体是最理想的状态。

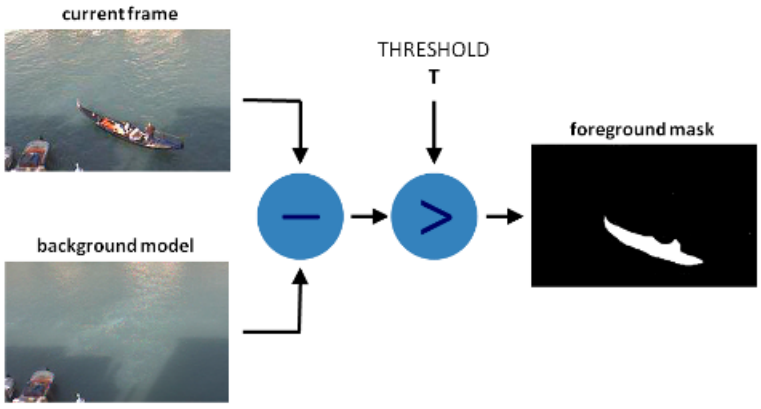


图 8.5: 背景差分法

- 均值平移法

  均值平移主要是根据概率密度的梯度爬升来寻找局部最优。通过直方图反投影将图像的像素值根据所要追踪物体的归一化直方图反投影为概率值，并通过迭代逐步寻找到概率密度最大的区域，作为该物体的最有可能出现的区域，从而追踪到物体，如图 8.6 。

  CamShift 算法的全称是”Continuously Adaptive Mean-SHIFT”，即：连续自适应的 MeanShift 算法。CamShift 是对视频序列的所有图像帧都作 MeanShift 运算，并将上一帧的结果（即搜索窗口的中心位置和窗口大小）作为下一帧 MeanShift 算法的搜索窗口的初始值，如此迭代下去。简单点说，meanShift 是针对单张图片寻找最优迭代结果，而 camShift 则是针对视频序列来处理，并对该序列中的每一帧图片都调用 meanShift 来寻找最优迭代结果。需要注意的是，在使用均值便平移时，非常依赖于参数设置（即初始位置、各个维度的带宽等），因此效果还需具体而定。CAMShift 的例子如下所示：

```
fourcc = cv.VideoWriter_fourcc(*'mp4v')
video_writer = cv.VideoWriter(output_path, fourcc, fps, (width, height))

try:
    while True:
```

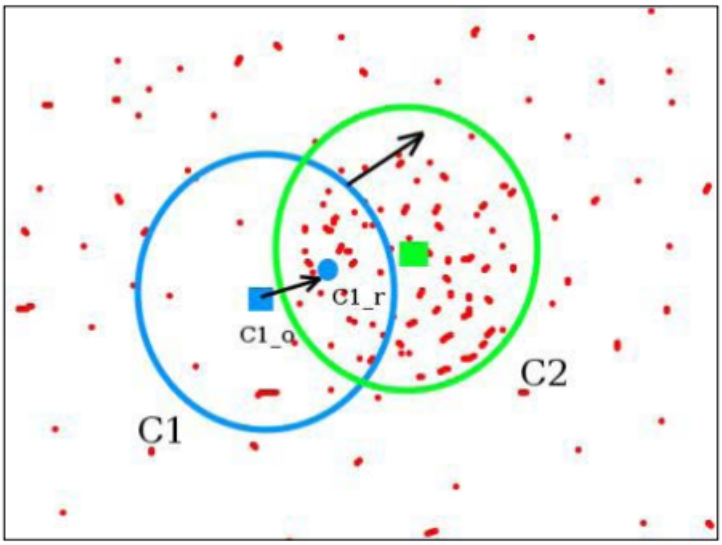


图 8.6: 均值平移法

```
6           ret, frame = cap.read()
7           if not ret:
8               print("视频处理完成")
9               break
10
11          hsv = cv.cvtColor(frame, cv.COLOR_BGR2HSV)
12          dst = cv.calcBackProject([hsv], [0], roi_hist, [0, 180], 1)
13          ret_tuple, track_window = cv.CamShift(dst, track_window, term_crit)
14
15          pts = cv.boxPoints(ret_tuple) # 获取旋转矩形的顶点
16          pts = np.int32(pts)
17
18          # 绘制跟踪框
19          frame_with_box = cv.polylines(frame.copy(), [pts], True, (0, 255, 255),
                2)
20          video_writer.write(frame_with_box) # 写入视频
21
22      print(f"视频已保存至:␣{output_path}")
23
24  except Exception as e:
25      print(f"处理过程中出错:␣{e}")
26
27  finally:
28      cap.release()
29      video_writer.release()
30      print("资源已释放")
```

8.4 动手实验：视频处理及人脸隐私保护

8.4.1 实验目的

1. 掌握编程实现人脸检测和隐私保护的基本方法
2. 掌握移动物体跟踪的方法

8.4.2 预习要求

1. 查看 OpenCV 关于本实验设计函数的手册，了解主要参数的含义
2. 编写本实验要求的相关代码

8.4.3 人脸检测与隐私保护

1. 读取 USB 摄像头，完成自己人脸、眼睛的检测

   采用摄像头检测人脸与在照片中检测人脸类似，主要从摄像头的实时流中获取数据，对每一帧图片完成人脸检测并进行标记。远程 Notebook 中不要使用`cv.imshow()`，可保存某一帧并采用实验三介绍的内联方式显示。读取摄像头时必须检查返回值，并确保释放资源。部分参考代码如下。

```
camera = cv.VideoCapture(0)
if not camera.isOpened():
    raise RuntimeError('无法打开␣USB␣摄像头')

try:
    ret, frame = camera.read()
    if not ret or frame is None:
        raise RuntimeError('摄像头没有返回图像帧')
    gray = cv.cvtColor(frame, cv.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, 1.1, 5)
finally:
    camera.release()
```

2. 完成人脸隐私保护（选做）

   在检测到人脸后，可以对人脸区域进行处理，避免直接显示可辨认的人脸。可以从以下功能中选择一个或多个实现：

   (a) 检测到人脸后自动对人脸区域打码；

   (b) 给人脸区域添加模糊、灰度或其他图像滤镜；

   (c) 比较人脸中心与画面中心的位置，判断人脸是否位于画面中央；

   (d) 根据人脸宽度占画面宽度的比例，粗略估计人脸距离摄像头的远近。

   人脸远近只能通过画面中的相对大小进行粗略判断，实际阈值需要根据摄像头视场角和使用场景调整。人脸打码可以先缩小人脸区域，再使用最近邻插值放大到原尺寸。部分参考代码如下：

```
def pixelate_region(region, blocks=12):
    h, w = region.shape[:2]
    small_w = max(1, w // blocks)
    small_h = max(1, h // blocks)
    small = cv.resize(region, (small_w, small_h),
```

```
                        interpolation=cv.INTER_LINEAR)
    return cv.resize(small, (w, h),
                     interpolation=cv.INTER_NEAREST)

# faces 为人脸检测器返回的矩形区域
for (x, y, w, h) in faces:
    face_roi = frame[y:y + h, x:x + w]
    protected = pixelate_region(face_roi, blocks=14)
    frame[y:y + h, x:x + w] = protected
```

3. 注意事项

   (a) Haar 人脸检测的限制 Haar 检测器对姿态、遮挡、光照和人脸大小较敏感。本实验只根据检测框进行隐私处理，不判断人的身份，也不根据眼睛检测结果推断是否佩戴眼镜。

8.4.4 物体跟踪

1. 跟踪移动的乒乓球

   使用 BackgroundSubtractor 识别移动的乒乓球，并用方框将其圈出来。需要注意的是，在处理的时候可能出现噪声，并且处理不是完美的，因此在画出移动物体的边框时，可以使用 contourArea() 判断边框的大小，从而确定是否为物体，并将物体标出。部分参考代码如下：

```

bs = cv.createBackgroundSubtractorKNN(detectShadows=True)
...
fgmask = bs.apply(frame)
th = cv.threshold(fgmask.copy(), 244, 255,
    cv.THRESH_BINARY)[1]
dilated = cv.dilate(th,
    cv.getStructuringElement(cv.MORPH_ELLIPSE, (3, 3)),
    iterations=2)
contours, _ = cv.findContours(dilated,
    cv.RETR_EXTERNAL, cv.CHAIN_APPROX_SIMPLE) # 提取移动物体轮廓
```

2. 利用 CAMShift 方法来跟踪乒乓球

8.5 思考题

1. 直方图反射投影的基本原理是什么？

2. CAMShift 算法的局限性有哪些？

第九章　人工神经网络

本章将介绍神经网络的概念，从最简单的神经元模型——线性回归，到如何堆叠多层网络构建深度神经网络，并介绍如何在 PyTorch 开源神经网络框架下建立和训练神经网络，最后介绍在视觉领域应用最多的卷积神经网络。在这个单元，我们将动手实验，实现通过手势识别实时控制发光二极管亮度的功能。实验中用到了 AD/DA 的原理，以及前边章节中所介绍过的 OpenCV 等知识。

9.1　神经网络

人工神经网络（Artificial Neural Network）是机器学习的研究热点，是深度学习的基础。它从信息处理角度对人脑神经元网络进行抽象，其中最常用的是用感知器模型来模拟单个神经元，如图9.1 所示。

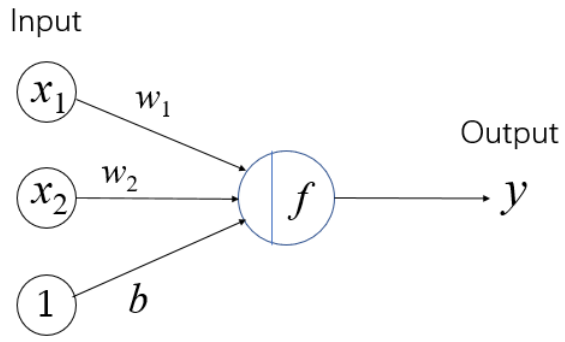


图 9.1: 神经元感知器模型

图9.1 中各个参数的关系如下：

$$y = f(\sum_{i=0}^{n} w_i x_i + b)$$

在这个模型中，神经元接收来自 n 个其它神经元传递过来的输入信号，通过不同权重叠加后并与阈值 b 比较，最后通过 “激活函数”f 处理以产生神经元的输出。理想的激活函数是阶跃函数，输出 “0” 对应神经元抑制，输出 “1” 对应神经元兴奋。但由于阶跃函数不连续、不光滑，实际常用 sigmoid 等函数作为激活函数，它把输出限制在 (0,1) 范围内。复杂的神经网络由多个不同参数的神经元组成网络，通过训练的方式确定参数的值，实现解决问题的目的。

9.2　从线性回归到 MLP

本节把神经元的激活函数取为恒等函数，从而将神经元模型简化为线性模型，再通过学习的方式获得线性模型的参数，进而了解机器学习的基本方法。这样的模型一般被称为线性回归模型。

对于给定数据集 $(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_m, y_m)$，其中 $\mathbf{x}_i = (x_{i1}, x_{i2}, \ldots, x_{in})$ 为输入，y_i 为输出。线性回归模型试图获得一个线性模型

$$f(\mathbf{x}) = w_1 x_1 + w_2 x_2 + \ldots + w_n x_n + b$$

来尽可能的根据输入值预测实际输出的值，即 $f(\mathbf{x}_i) \simeq y_i$。

用预测值与真实值之间欧氏距离的平方定义一个误差函数：

$$E = (f(\mathbf{x}_i) - y_i)^2$$

通过数据学习的目的就是最小化这个误差。从数学上的角度来看，梯度的方向是函数增长速度最快的方向，那么梯度的反方向就是函数减少最快的方向。

$$\frac{\partial E}{\partial w_i} = 2(f(\mathbf{x}_i) - y_i)x_i = 2\Delta_i x_i$$

$$\frac{\partial E}{\partial b} = 2(f(\mathbf{x}_i) - y_i) = 2\Delta_i$$

定义学习率为 η，更新参数 w_i 的方式就是用 $w_i - \eta\Delta_i x_i$ 替代 w_i，偏置 b 的更新类似，用 $b - \eta\Delta_i$ 替代 b；这里将常数项 2 省去。这样根据数据对 w_i 不断更新以获得最优的线性模型参数的方法就是梯度下降法。

前面我们以线性回归为例，介绍了从数据中学习模型参数的基本思路，下面进一步了解由神经元组成的网络。图9.1 中的感知器模型可以看成是两个输入、一个输出的单层人工神经网络（输入层不涉及计算，不记入网络的层数）。

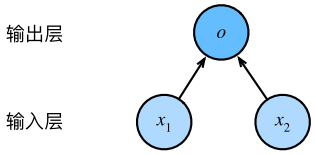


图 9.2: 线性回归模型

更常见的人工神经网络会有更多的层，中间的层称为隐藏层。如图9.3所示，是一个含一个隐藏层的网络。由于线性变换的组合仍然是线性变换，为了体现多层模型的价值，在每个感知器模型中都引入了非线性的激活函数，在前文我们提及了 sigmoid 函数，目前更常用的是一个更简单的 ReLU（Rectified Linear Unit）函数（如图 9.4）：

$$\mathrm{ReLU}(x) = \max(x, 0).$$

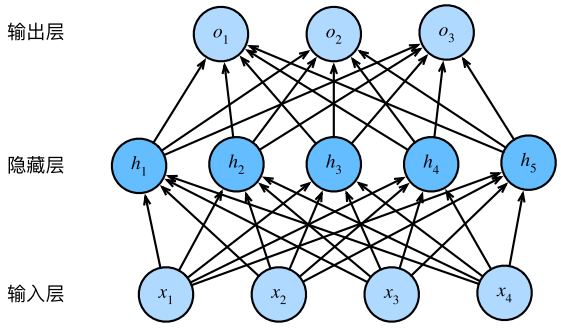


图 9.3: 含一个隐藏层的多层感知器（MLP）结构

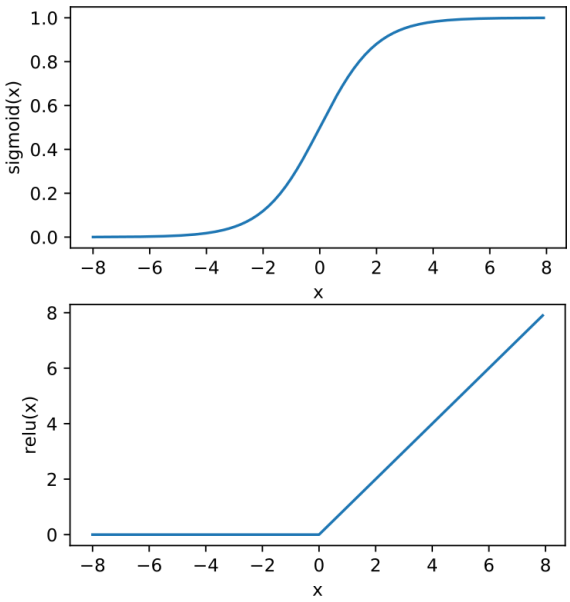


图 9.4: sigmoid 和 ReLU 函数

9.3 PyTorch 简介

PyTorch 是由 Facebook 的人工智能研究小组（FAIR）在 2016 年开发的基于 Torch 的 Python 机器学习库，完全开源，是目前最流行的深度学习框架之一。

PyTorch 的程序可以立即执行计算，这正好符合 Python 的编程方法，不需要完成全部代码才能运行，可以轻松的运行部分代码并实时检查。PyTorch 支持互动式的调试，使得调试和可视化变得非常容易。与 TensorFlow 的静态计算图不同，PyTorch 的计算图是动态的，以便可以在运行时构建计算图，甚至在运行时更改它们，在不知道创建神经网络需要多少内存的情况下这非常有价值。PyTorch 可以顺利地与 Python 数据科学栈集成。它非常类似于 numpy，甚至注意不到它们的差别。

对 PyTorch 的最基本理解包括如下三方面：

- Numpy 风格的 Tensor（张量）操作。Tensor 是神经网络框架中重要的基础数据类型，可以简单理解为 N 维数组的容器对象。tensor 之间通过运算进行连接，从而形成计算图。PyTorch 中 tensor 提供的 API 参考了 Numpy 的设计，因此熟悉 Numpy 的用户基本上可以无缝理解并创建和操作 tensor，同时 torch 中的数组和 Numpy 数组对象可以无缝的对接。torch 为 CPU 和 GPU 分别定义了多种 tensor 数据类型，torch 模块内提供了操作 tensor 的接口，而 Tensor 类型的对象上也设计了对应的接口，例如 torch.add() 与 tensor.add() 等价。
- 变量自动求导。tensor 对象通过一系列的运算可以组成动态图，每个 tensor 对象可以方便的计算自己对目标函数的梯度。这样就可以方便的实现神经网络的后向传播过程。
- 神经网络层与损失函数优化等高层封装。网络层的封装存在于 torch.nn 模块，损失函数由 torch.nn.functional 模块提供，优化函数由 torch.optim 模块提供。

torch.nn 模块提供了创建神经网络的基础构件，这些层都继承自 Module 类。在 nn.functional 模块中，提供多种激活函数的实现。通常对于可训练参数的层使用 module，而对于不需要训练参数的层如 softmax 这些，可以使用 functional 中的函数。用 PyTorch 定义如图9.3所示 MLP 神经网络的代码如下：

```
import torch.nn as nn
import torch.nn.functional as F

class Net(nn.Module):
    def __init__(self):
```

```
        super(Net, self).__init__()
        self.hidden = nn.Linear(4, 5)
        self.out   = nn.Linear(5, 3)

    def forward(self, x):
        x = F.relu(self.hidden(x))
        x = self.out(x)
        return x
net=Net()
print(net)
```

神经网络实现的基本步骤：

- 准备数据：带标签的训练集和测试集
- 定义网络结构 model
- 定义损失函数
- 定义优化算法 optimizer
- 训练
  - 前向传播计算网络输出 output 和计算损失函数 loss
  - 反向传播更新参数：
    - 将上次迭代计算的梯度值清 0：optimizer.zero_grad()
    - 反向传播，计算梯度值：loss.backward()
    - 更新权值参数：optimizer.step()
  - 分别记录训练集与验证集上的 loss 和准确率，并打印训练信息（可选）
- 图示训练过程中 loss 和 accuracy 的变化情况（可选）
- 在测试集上测试

深度学习框架中涉及很多参数，下面介绍一下 batch、iterations 和 epochs 的概念：
深度学习的优化算法，即梯度下降，每次的参数更新有两种方式：

1. 遍历全部数据集算一次损失函数，然后算函数对各个参数的梯度，更新参数。这种方法每更新一次参数都要把数据集里的所有样本都看一遍，计算开销大，计算速度慢，不支持在线学习，这称为 Batch gradient descent，批梯度下降。
2. 每看一个数据就算一下损失函数，然后求梯度更新参数，这个称为随机梯度下降，stochastic gradient descent。这个方法速度比较快，但是收敛性能不太好，可能在最优点附近晃来晃去，得不到最优点。两次参数的更新也有可能互相抵消掉，造成目标函数震荡的比较剧烈。

为了克服两种方法的缺点，现在一般采用的是一种折中手段，mini-batch gradient descent，小批的梯度下降，这种方法把数据分为若干个批，按批来更新参数，这样，一个批中的一组数据共同决定了本次梯度的方向，下降起来就不容易跑偏，减少了随机性。另一方面因为批的样本数与整个数据集相比小了很多，计算量也不是很大。基本上现在的梯度下降都是基于 mini-batch 的，所以深度学习框架的函数中经常会出现 batch_size，就是指这个。

每一次迭代（iteration）就是一次权重更新：用 batch_size 个样本做一次前向运算得到损失，再用反向传播（BP）算法更新参数。

一个 epoch（轮）指整个训练集完整地参与一次训练，即所有批次各做一次前向和反向传播。简单说，epochs 就是训练过程中数据将被完整使用多少次。

例如，训练集有 1000 个样本，batchsize=10，那么训练完整个样本集需要：100 次 iteration，1 次 epoch。

9.4 卷积神经网络(CNN)模型

CNN 相比多层感知机等简单神经网络的特点是：一，局部连接——每个神经元只与输入图像中一小块相邻区域的像素相连，处理来自彼此接近的像素的信息，比如为了识别字母或数字，需要分析像素的相关性，它们决定了元素的形状；二，共享参数——一层中所有神经元之间共享有限数量的权重，由此减少了权重数量，有助于防止过拟合。

9.4.1 卷积层

卷积层是 CNN 的重要组成部分，由一组过滤器（Filters，也称为卷积核，或特征检测器）组成，每个过滤器都应用于输入数据的所有区域。过滤器由一组可学习的权重定义。如图9.5所示，每个输入神经元可以代表一个像素的灰度值，首先，在图像的右上角应用 3x3 过滤器（通常大小是任意的，如 2x2、4x4、5x5 等），由于有九个输入神经元相连，就有 9 个权重，过滤器的输出是其输入的加权和。过滤器的输出表示下一层神经元的激活值，如果对应于该过滤器的特征出现在该处，则下一层相应的神经元将激活，神经元之后使用激活函数，最常见的激活函数是 ReLU。对于下一层的每个新的神经元，过滤器将在输入图像上移动，然后用每组新的输入神经元计算其输出。对于移动，过滤器的权重在整个图像中不会改变（权重共享），即使用所有相同的 9 个过滤器权重计算所有输出神经元的激活，每次使用一组不同的输入神经元。每个过滤器对应于特定的一个特征，通过共享权重，可保证过滤器能够在整个图像中找到特征。

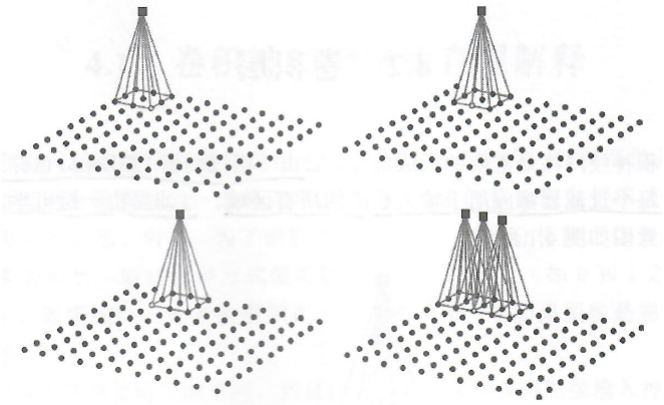


图 9.5: 过滤器及其在图像上的移动

在空间上排列的神经元称为深度切片（Depth Slices）或特征图（Feature Map），切片可以用作网络中其他层的输入。在卷积层，偏置权重也在所有神经元之间共享，对整个切片使用单个偏置权重。对于彩色图片，可以将图像分成多个颜色通道，若为 RGB 图像，则为 3，将每个颜色通道视为一个深度切片，切片的组合称为深度为 3 的输入体积（Input Volume）。如图9.6所示，同一个 3x3 过滤器分别应用于每个颜色通道的切片，将这三个过滤器组合起来就是一个 3x3x3+1（含一个偏置）的大过滤器，共有 28 个权重。

一个过滤器对应于一个特定特征，如边或线，而许多特征都很重要，故一组输入切片上可以应用多个过滤器，每个过滤器将生成一个唯一的输出切片。

输入和输出特征映射具有不同的维度，假设有一个大小为（width, height）的输入层和一个维度为（filter_w, filter_h）的过滤器，应用卷积后，输出层的维度为（width-filter_w+1, height-filter_h+1），如图9.7所示。

设计一个 CNN，需要做的是定义网络架构，如卷积层的数量、输出体积的深度以及过滤器的大小，经训练后网络将找出特征。

过滤器不仅可以每次移动一个像素，也可以移动多个像素，卷积层的这个参数称为步幅（步长），而通常输入的所有维度的步幅都是相同的。把步幅包括进去，输出层的维度为（$\lfloor$(width-filter_w)/stride_w$\rfloor$+1, $\lfloor$(height-filter_h)/stride_h$\rfloor$+1），其中 $\lfloor\cdot\rfloor$ 表示向下取整。例如，一个 28x28 输入图像，使用步幅为 1 的

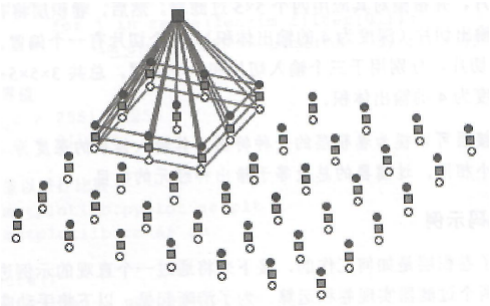


图 9.6: 深度为 3 的输入切片示例

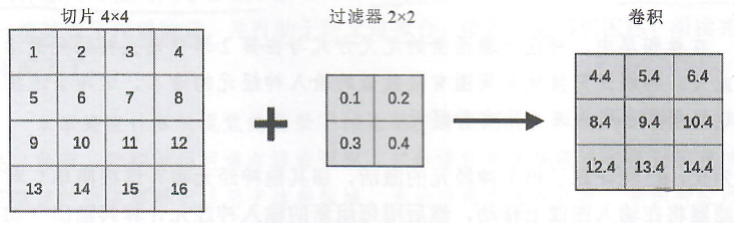


图 9.7: 在 4x4 切片上使用 2x2 过滤器进行卷积的示例

3x3 过滤器卷积生成的正方形切片输出大小为 28-3+1=26；当步长为 2 时，得到 $\lfloor(28\text{-}3)/2\rfloor+1=13$。较大步幅的主要作用是增加输出神经元的感受野，使网络能够检测到输入的更大、更复杂的特征。

如果需要使输出的幅度与输入相同，可以在卷积运算之前用 0 的行和列填充输入片的边缘，如图9.8所示。加入填充这个因素，假设输入切片为 $I=(I_w,I_h)$，过滤器为 $F=(F_w,F_h)$，步幅为 $S=(S_w,S_h)$，填充为 $P=(P_w,P_h)$，则输出切片 $O=(O_w,O_h)$ 由以下公式给出：

$$O_w=\frac{I_w+2P_w-F_w}{S_w}+1$$

$$O_h=\frac{I_h+2P_h-F_h}{S_h}+1$$

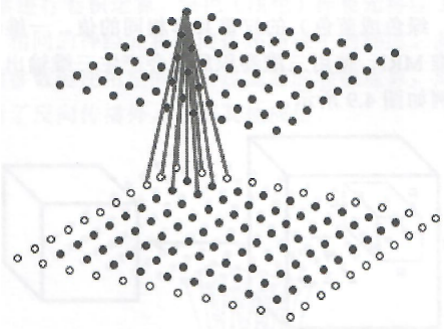


图 9.8: 对卷积层进行填充以使得输出与输入幅度相同

9.4.2 池化层

借助池化层（Pooling Layers）也可以增加神经元的感受野。池化层将输入切片分割成一个网格，每个网格单元代表一个由多个神经元组成的感受野。然后在网格的每个单元上应用池化操作。池化层不会改变体积深度，因为池化操作是在每个切片上独立执行的。

最常用的最大池化会在每个局部感受野（网格单元）中获取具有最高激活值的神经元。感受野为 2x2 的最大池化如图9.9所示。平均池化则输出每个感受野内所有激活的平均值。池化层涉及两个参数，步幅

（与卷积层相同）及感受野大小，实际上只使用两种组合，第一种是 2x2 的感受野，步幅为 2；第二种是 3x3 的感受野，步幅为 2（重叠）。

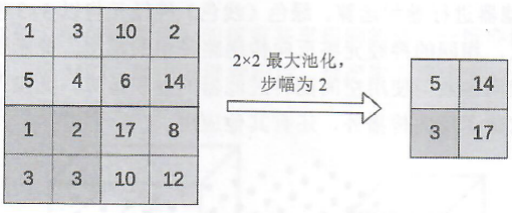


图 9.9: 最大池化示例

用 I 表示输入切片的大小，用 F 表示感受野的大小，用 S 表示步幅的大小，用 O 表示输出的大小。池化层通常没有填充，其输出大小计算公式如下：

$$O_w = \left\lfloor \frac{I_w - F_w}{S_w} \right\rfloor + 1, \qquad O_h = \left\lfloor \frac{I_h - F_h}{S_h} \right\rfloor + 1$$

常用的 2x2 感受野、步幅为 2 的最大池化，就是把切片的高和宽各缩小一半。

9.4.3 卷积神经网络的结构

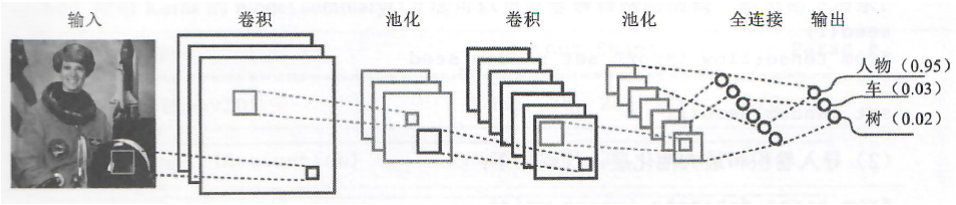


图 9.10: 卷积神经网络的结构

一个基本 CNN 的结构如图9.10所示。大多数 CNN 有如下基本属性：

1. 通常将一个或多个卷积层与一个池化层交替使用，由此卷积层可以在感受野大小的每一级检测特征。较深层的感受野大小大于网络起始处的感受野大小，由此可以从较大的输入区域捕获更复杂的特征。
2. 最深层检测到的特征是高度抽象的，无法直接阅读，通常在最后一个卷积/池化层之后添加一个或多个全连接层。可以将全连接层视为网络语言（人类无法理解）和人类语言之间的翻译器。
3. 较深的卷积层通常比初始层存在更多的过滤器（因此体积深度更高）。网络开始时的特征检测器工作在一个小的感受野上，只能检测在所有类之间共享的有限数量的特征，如边或线。而一个较深的层将检测到更复杂和众多的特征，比如汽车、树或人，每个类都有自己的一组特征，如轮胎、车门、树叶和脸等，因此需要更多的特征检测器。

9.5 动手实验：通过手势识别实时控制发光二极管亮度

9.5.1 实验目的

1. 熟悉 AD/DA 原理，实现控制发光二极管开、关、调亮、调暗的功能。
2. 学习通过 PyTorch 搭建人工神经网络模型的方法。
3. 学习通过卷积神经网络（CNN）实现简单手势识别的方法，掌握调整 CNN 网络参数的方法。
4. 实现通过手势识别实时控制发光二极管亮度的功能。

9.5.1.1 预习要求

1. 阅读第 2.3.2.3 节,掌握 AD/DA 基本原理和使用方法。
2. 参考本实验代码提前准备训练集和测试集。
3. 理解本实验提供的全部程序代码。

9.5.2 实验内容

9.5.2.1 AD/DA 模块的检验及发光二极管的控制

1. 使用面包板搭建电路,通过 A/D 采样读取电位器的电压信号,并打印出每次采样的电压值。参考电路如图 9.11 所示。
2. 调整 DA 输出,点亮发光二极管,设计二极管的发光模式,如开、关、调亮、调暗等功能。参考电路如图 9.11。
3. 用电位器的输出控制发光二极管的明暗。

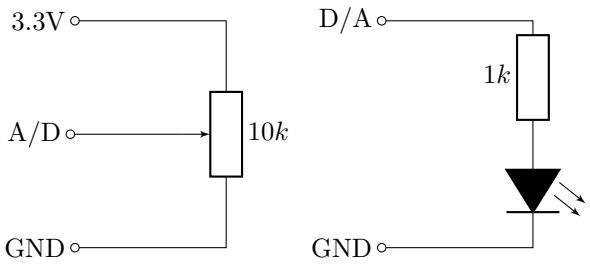


图 9.11: 滑动变阻器电路和发光二极管电路

9.5.2.2 训练集和测试集

本实验所使用的数据集来自 HandGesture-5G,其中包含各种肤色的 5 种不同的手势照片。原数据集图像分辨率比较高,本实验提供的数据已将图片分辨率压缩到 120×120。在进行训练之前,需要将数据集中图片进行预处理,然后随机分成训练集和测试集。

数据集包含的五种手势是:Fist(握拳)、LSign(L 手势)、VSign(V 手势)、Palm(手掌展开)和 Pointing(食指伸出)。可以通过肤色识别找出皮肤区域,进行灰度处理变为单通道,然后改变分辨率为 80×60 供模型进行训练和推理。

预处理的效果直接影响后续的训练和推理,可以把预处理结果保存下来进行人工检查,如果效果不理想可以删除不好处理的数据。剔除不合理数据的过程也可以自动进行:例如把皮肤面积低于一定阈值的样本判定为不合格数据并剔除。

9.5.2.3 搭建神经网络模型

1. 搭建神经网络

   采用“卷积-池化-卷积-池化-一维化-全连接层-输出”的结构,训练采用小批梯度下降及 Adam 优化器,损失函数使用交叉熵。输入是 80×60(宽 $\times$ 高)的单通道灰度图,输出是五种手势的得分。下面给出参考代码,分成“数据准备、网络结构、训练与推理、曲线与混淆矩阵”四个部分。其中网络结构是一个基线网络“卷积 1-池化 1-卷积 2-池化 2-一维化-全连接 1-全连接 2-输出”,它只是一个起点,效果未必理想,后面的任务会让你动手改进它。

```python
# 参考代码框架：数据准备 + 网络结构 + 训练与推理
import torch, torch.nn as nn, torch.optim as optim
import torchvision.datasets as datasets, torchvision.transforms as T
from torch.utils.data import DataLoader
from tqdm.auto import tqdm
import numpy as np, matplotlib.pyplot as plt

# ------------------------- 参数 -------------------------
DATA_DIR   = 'dataset'     # 数据集根目录，内含 train/ 与 test/ 子文件夹
EPOCHS     = 40            # 训练轮数
BATCH_SIZE = 64            # 批大小
LR         = 1e-3          # Adam 学习率
DEVICE     = 'cuda' if torch.cuda.is_available() else 'cpu'

# ------------------------- 数据准备 -------------------------
# dataset/train 与 dataset/test 下各存放五个以手势命名的子文件夹
transform = T.Compose([T.Grayscale(), T.Resize((60, 80)), T.ToTensor()])
train_ds = datasets.ImageFolder(f'{DATA_DIR}/train', transform)
test_ds  = datasets.ImageFolder(f'{DATA_DIR}/test',  transform)
train_dl = DataLoader(train_ds, BATCH_SIZE, shuffle=True)
test_dl  = DataLoader(test_ds,  BATCH_SIZE, shuffle=False)

# ------------------------- 网络结构 -------------------------
# 输入 80x60(宽x高) 单通道灰度图，输出 5 个手势的得分
#   卷积1(1->6通道) : 76x56, 池化4 -> 19x14
#   卷积2(6->12通道): 15x10, 池化4 -> 3x2
#   一维化 12*3*2=72 -> 全连接1(60) -> 全连接2(30) -> 输出(5)
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(1,  6, 5), nn.ReLU(), nn.MaxPool2d(4),
            nn.Conv2d(6, 12, 5), nn.ReLU(), nn.MaxPool2d(4),
            nn.Flatten(),
            nn.Linear(12*3*2, 60), nn.ReLU(),
            nn.Linear(60, 30),     nn.ReLU(),
            nn.Linear(30, 5)
        )
    def forward(self, x):
        return self.net(x)

model = Net().to(DEVICE)

# ------------------------- 训练与推理 -------------------------
opt     = optim.Adam(model.parameters(), lr=LR)
loss_fn = nn.CrossEntropyLoss()

train_losses, val_losses = [], []
```

```
train_accs,   val_accs   = [], []

for epoch in range(1, EPOCHS + 1):
    # ---- 训练: 前向计算、反向传播、参数更新 ----
    model.train()
    running_loss, correct, total = 0.0, 0, 0
    for x, y in tqdm(train_dl, desc=f'Epoch␣{epoch}/{EPOCHS}'):
        x, y = x.to(DEVICE), y.to(DEVICE)
        opt.zero_grad()
        logits = model(x)                    # 前向: 网络对一个批的评分
        loss   = loss_fn(logits, y)          # 交叉熵损失
        loss.backward()                      # 反向: 计算梯度
        opt.step()                           # 更新参数
        running_loss += loss.item() * x.size(0)
        correct += (logits.argmax(1) == y).sum().item()
        total   += x.size(0)
    train_losses.append(running_loss / total)
    train_accs.append(correct / total)

    # ---- 推理: 每轮结束后在测试集上验证 ----
    model.eval()                             # 切换到推理(评估)模式
    running_loss, correct, total = 0.0, 0, 0
    with torch.no_grad():                    # 推理不需要计算梯度
        for x, y in test_dl:
            x, y = x.to(DEVICE), y.to(DEVICE)
            logits = model(x)
            running_loss += loss_fn(logits, y).item() * x.size(0)
            correct += (logits.argmax(1) == y).sum().item()
            total   += x.size(0)
    val_losses.append(running_loss / total)
    val_accs.append(correct / total)
    print(f'epoch␣{epoch:3d}␣␣train_acc␣{train_accs[-1]:.4f}␣␣test_acc␣{
        val_accs[-1]:.4f}')
```

提示 1(最重要):全连接层的输入维度 $12\times3\times2$ 必须与网络结构一一对应:它等于最后一张特征图的“通道数 × 高 × 宽”。以后每修改一次网络结构,都要按下式重新推导各层输出尺寸,并同步修改全连接层的输入维度,否则运行时会报维度不匹配的错误:

$$\text{卷积输出尺寸}=\left\lfloor\frac{\text{输入尺寸}-\text{卷积核}+2\times\text{填充}}{\text{步长}}\right\rfloor+1,\qquad \text{池化输出尺寸}=\left\lfloor\frac{\text{输入尺寸}}{\text{池化窗口}}\right\rfloor.$$

提示 2:训练阶段每轮开始前调用 `model.train()`,验证阶段调用 `model.eval()` 并用 `torch.no_grad()` 关闭自动求导,两者都不可省略:某些层(如 Dropout、BatchNorm)在两种模式下行为不同,而关闭自动求导可以省下大量内存和计算时间。

绘制损失/准确率曲线与混淆矩阵的参考代码如下:

```
# ------------------------- 绘制损失与准确率曲线 -------------------------
plt.figure(figsize=(6, 3))
plt.subplot(1, 2, 1)
plt.plot(train_losses, label='train')
```

```
plt.plot(val_losses,   label='val')
plt.title('Loss'); plt.ylabel('CE'); plt.legend()

plt.subplot(1, 2, 2)
plt.plot(train_accs, label='train')
plt.plot(val_accs,   label='val')
plt.title('Accuracy'); plt.ylabel('acc'); plt.legend()
plt.tight_layout(); plt.show()

# ------------------------- 混淆矩阵 -------------------------
model.eval()
all_preds, all_labels = [], []
with torch.no_grad():
    for x, y in test_dl:
        all_preds.append(model(x.to(DEVICE)).argmax(1).cpu())
        all_labels.append(y)
all_preds  = torch.cat(all_preds).numpy()
all_labels = torch.cat(all_labels).numpy()

num_classes = len(test_ds.classes)
cm = np.zeros((num_classes, num_classes), dtype=int)
for t, p in zip(all_labels, all_preds):
    cm[t, p] += 1

plt.figure(figsize=(4, 4))
plt.imshow(cm, cmap='Blues')
plt.colorbar()
ticks = np.arange(num_classes)
plt.xticks(ticks, test_ds.classes, rotation=45)
plt.yticks(ticks, test_ds.classes)
plt.ylabel('True'); plt.xlabel('Predicted')
for i, j in np.ndindex(cm.shape):
    plt.text(j, i, cm[i, j], ha='center', va='center',
             color='white' if cm[i, j] > cm.max() / 2 else 'black')
plt.tight_layout(); plt.show()
```

任务 1：运行参考代码训练基线网络，记录训练结束后测试集上的准确率，然后回答下面的问题：

- 损失曲线是否出现过拟合？你是如何判断的？
- 混淆矩阵中哪两类手势最容易互相误判？结合手势的外观猜测可能的原因。

可以从损失函数随训练轮次的变化曲线看出是否存在过拟合。

提示 3：过拟合的典型特征是：训练集 loss 持续下降、准确率接近 100%，而测试集 loss 不再下降甚至回升，测试准确率明显低于训练准确率。样本量不大时，参数量越大的网络越容易过拟合。

2. 改进网络，提高手势识别准确率

参考代码中的网络只是一个起点，请动手改进它，目标是让测试集准确率达到 95% 以上。下面给出一些改进方向（不限于此），请先理解每一条的原理再动手，并记录修改前后的对比：

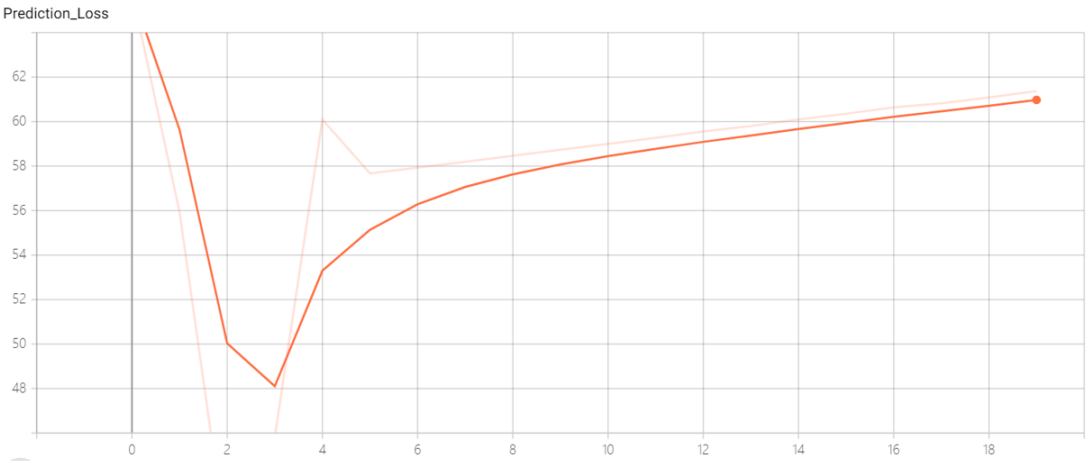


图 9.12: 过拟合情况示例，从第三轮开始

(a) 加深网络：增加一组“卷积-池化”并把卷积输出通道数逐层加倍（例如 8、16、32），让网络学到更抽象的特征；同时把池化窗口从 4×4 调小为 2×2，保留更多空间细节。**别忘了按提示 1 重新计算全连接层的输入维度。**

(b) 抑制过拟合：在全连接层之间插入 `nn.Dropout(p)`（例如 $p = 0.3$），训练时随机丢弃一部分神经元，相当于同时训练多个“瘦身”的子网络，能有效提高泛化能力；也可以减小全连接层的宽度，或减少全连接层的层数。

(c) 数据增强：训练时对样本做小幅随机仿射变换（平移、旋转、缩放）以及亮度、对比度扰动，相当于免费扩充了训练集。可以查看 `torchvision.transforms` 中的 `RandomAffine` 和 `ColorJitter` 的用法，注意增强只应在训练时使用。

(d) 训练策略：把学习率按余弦退火（`CosineAnnealingLR`）从初始值平滑降到 0，损失通常收敛得更平稳；还可以适当增加训练轮数，或微调批大小与学习率。

(e) 固定随机种子：训练开始前设置 `random.seed`、`torch.manual_seed` 等，使每次实验可复现，修改前后的对比才公平。

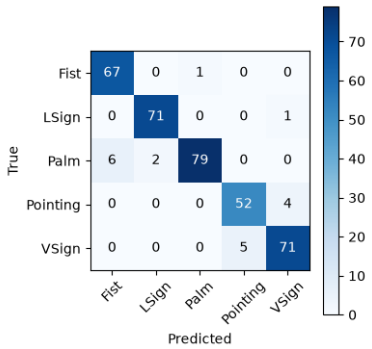


图 9.13: 在测试集上实测的混淆矩阵示例

任务 2：每次只修改一个变量，用同样的曲线与混淆矩阵工具检验改动是否有效，并把每次修改前后的测试准确率记录下来做对比。达到目标后，把效果最好的模型保存下来，下一节实时手势识别会用到它：

```
1 # 保存训练好的模型权重与类别对应关系
```

```
import os, json
os.makedirs('runs', exist_ok=True)
with open('runs/classes.json', 'w', encoding='utf-8') as f:
    json.dump(train_ds.class_to_idx, f, ensure_ascii=False, indent=2)
torch.save(model.state_dict(), 'runs/model.pth')
```

提示 4:混淆矩阵主对角线以外的数值就是误判样本。集中分析误判最多的一对类别，往往能给出最直接的改进方向——例如是预处理不够好、某类样本太少，还是两类手势本身外观过于接近、需要更有区分力的特征。

9.5.2.4 通过手势识别实时控制发光二极管亮度

导入训练到较好效果的模型。与采集训练样本时类似，用摄像头捕捉图片，经过与训练时完全相同的预处理（灰度化、调整大小等），将实时样本送进神经网络，可得到神经网络对各类手势的评分及最后判定结果，根据判定结果分别控制发光二极管的开、关、调亮、调暗。

加载模型的参考代码如下（注意类别顺序必须与训练时一致）：

```
# 1. 重新建一个同结构的网络
model = Net().to(DEVICE)
# 2. 把权重读进来
model.load_state_dict(torch.load('runs/model.pth', map_location=DEVICE))
# 如果后面只做推理，再加一句：
model.eval()

# 读入训练时保存的类别名顺序（与 ImageFolder 的字母序一致）
import json
with open('runs/classes.json', encoding='utf-8') as f:
    class_to_idx = json.load(f)
class_names = [name for name, _ in sorted(class_to_idx.items(), key=lambda kv: kv
    [1])]
```

通过神经网络对实时样本进行分类和判定的部分参考代码如下：

```
# 推理（x 为预处理后的单样本张量，形状 (1, 1, 60, 80)，并已 .to(DEVICE)）
with torch.no_grad():
    logits = model(x)
    probs  = torch.softmax(logits, dim=1)
    conf, pred = torch.max(probs, dim=1)

# 输出结果
print(f'预测类别：{class_names[pred.item()]}')
print(f'置信度：{conf.item():.4f}')
```

9.5.3 注意事项

1. 训练与推理的预处理必须一致

   实时识别时摄像头图像的灰度化、调整大小等处理，必须与训练时的预处理完全相同。一旦两边不一致，输入分布就会发生偏移，准确率会明显下降甚至完全失效，这是本实验最容易踩的坑。修改任何一步预处理后，都应重新检查实时识别的效果。

2. 光照与肤色提取的影响

   如果预处理中包含按颜色空间提取手部（皮肤）区域这一步，识别效果对拍摄时的色调和光照就比较敏感：面向强光、逆光或色温变化较大时，较亮或偏色的区域可能无法被正确提取，进而影响识别；即使不显式提取肤色，强光或色温的大幅变化也会改变图像整体外观，同样会影响识别。实验时应尽量保持实验室的光照与采集数据集时的光照一致，避免强逆光直射镜头。

3. 判定结果需要平滑处理

   实时识别的判定结果会逐帧波动，若直接用每一帧的结果去控制发光二极管，会出现亮度快速跳变、闪烁的现象。常见做法是对最近若干帧（例如 5~10 帧）的预测做多数表决，或只在置信度高于某个阈值时才更新判定；对调亮、调暗这类增量操作，还应避免在边界附近来回振荡。

4. 模型与类别文件要配套使用

   `model.pth` 必须与同一次训练保存的 `classes.json` 配套加载：ImageFolder 按文件夹名字母序确定类别编号，两者不一致时预测出的类别就会张冠李戴；更换网络结构后，旧权重也无法直接加载到新结构中。

5. 通过预处理提升对光照和肤色的鲁棒性

   当识别效果在不同光照或肤色下波动较大时，应优先从预处理上想办法削弱这些影响：灰度化之后可以做直方图均衡化或归一化，减弱整体亮度的影响（注意训练与推理必须做同样的处理）；按颜色空间提取皮肤区域时，选用对光照相对不敏感的颜色空间（如 HSV），并把阈值留有余量；训练时加入随机亮度、对比度扰动等数据增强（见改进方向 3），让模型本身对光照变化更稳健。若这些手段仍不够，再考虑在实验室实际光照条件下补采少量样本作为补充。

9.6 思考题

1. 如果训练时图像预处理和推理时对输入图像进行的处理方式不同会有什么后果？

2. 如何改进容易出现误判或得分较低的手势的正确率？

第十章　语音识别和处理入门

除了占 70% 的视觉信息外，听觉信息是智能体的第二大信息来源。为此我们在本单元介绍开发板音频的输入输出方法，以及语音识别方面的一个经典模型——HMM-GMM 模型。最后动手实验实现用按键控制录音，并在此基础上，基于语音识别方法完成一个简单的语音控制系统。

10.1　开发板音频输入输出方法

开发板没有提供音频输入和输出接口，音频的输入可以使用 USB 2.0 接口的 USB 麦克风（实验室的 USB 摄像头也包含麦克风输入）。

实验采用免驱 USB 麦克风，直接插入开发板上的 USB 2.0 接口即可，不需要再安装驱动，可以录制一段音频，验证麦克风是否正常工作。首先，使用 arecord -l 可以列出所有录音设备，一般输出如下：

```
**** List of CAPTURE Hardware Devices ****
card 0: U0x46d0x825 [USB Device 0x46d:0x825], device 0: USB Audio [USB Audio]
  Subdevices: 1/1
  Subdevice #0: subdevice #0
```

注意这里的 card 编号和 device 编号，后续的命令需要使用。同样地，aplay -l 可以列出所有播放设备，在未接入其它音频设备的情况下，开发板没有播放设备。

可执行 Linux 自带的录音命令，测试硬件是否正常：

```
sudo arecord -D plughw:0,0 -d 5 test.wav
```

arecord 是录音命令，其中 plughw:0,0 表示 arecord -l 命令中列出的 card 0、device 0，如果你的 USB 声卡录音设备与这里显示不同，还要进行相应修改。

在 Python 中执行录音操作需要使用 `pyaudio` 模块。首先，通过 `pyaudio.PyAudio()` 实例化一个 PyAudio 对象。要开始录制或播放音频，需调用 `pyaudio.PyAudio.open()` 方法，在指定设备上以所需的音频参数打开一个音频流，该方法返回一个 `pyaudio.Stream` 对象。

以下是一个典型的录音流配置示例：

```
1  CHUNK = 1024                # 每次读取的音频帧数
2  FORMAT = pyaudio.paInt16    # 音频格式: 16 位整数
3  CHANNELS = 1                # 单声道
4  RATE = 48000                # 采样率: 48 kHz
5
6  p = pyaudio.PyAudio()
7  stream = p.open(format=FORMAT,
8                  channels=CHANNELS,
9                  rate=RATE,
10                 input=True,
11                 frames_per_buffer=CHUNK)
```

通过 pyaudio.Stream.read() 方法读取音频数据，实现录音功能。该操作默认运行在“阻塞模式”，即每次读取会阻塞程序直到指定数量的音频帧被读取完成。若需实时处理音频数据或实现非阻塞式音频处理，应使用“回调模式”，在打开流时传入回调函数。录音完成后，应先调用 stream.stop_stream() 停止流，再调用 stream.close() 关闭流，最后调用 p.terminate() 终止 PyAudio 会话。

保存录音数据推荐使用 soundfile 库来保存为常见音频格式（如 WAV）。使用方法如下：

```
import soundfile as sf
import numpy as np

# 假设录音数据已通过 stream.read() 获取，并存储在变量 data 中
# 注意：data 是 bytes 类型，需转换为 numpy 数组
audio_data = np.frombuffer(data, dtype=np.int16)

# 使用 soundfile 保存为 WAV 文件
sf.write(filename, audio_data, RATE, subtype='PCM_16')
```

其中，subtype='PCM_16' 指定了 16 位线性 PCM 编码，与 pyaudio.paInt16 格式对应。使用 soundfile 可更方便地支持多种音频格式和采样率。

10.2 HMM-GMM 模型介绍

语音识别任务通常需要将一段音频转化成相应的文字，从数学角度上看，也就是求一段音频属于哪段文字的概率最大。传统语音识别框架中，所谓声学模型，就是把语音的声学特征映射（解码）到音素或字词这样的基本单元；语言模型则刻画字词序列出现的概率。定义声学模型输入为 $O = (o_1, o_2, \ldots, o_T)$ ，对应的句子 $W = (w_1, w_2, \ldots, w_N)$，语音识别的任务就是求概率 $P(W|O)$ 最大时对应的字序列 W^*。要计算 $P(W|O)$，可以利用贝叶斯公式：

$$P(W|O) = \frac{P(O|W)P(W)}{P(O)}$$

其中 $P(O)$ 在计算时可以忽略，这样就得到了两部分 $P(O|W)P(W)$，分别对应于传统语音识别框架中的声学模型和语言模型，前者的任务是计算给定文字之后发出这段语音的概率；后者表示某一字词序列发生的概率。

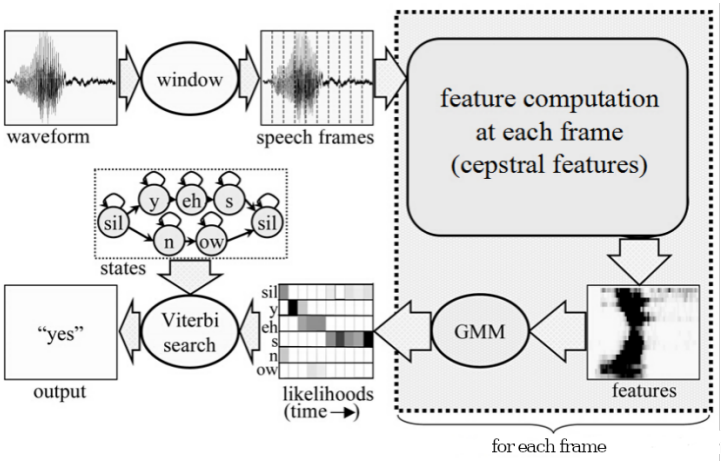


图 10.1: HMM-GMM 模型语音识别示意图

在命令词识别任务中，只需要使用声学模型，本实验采用传统的 HMM-GMM (hidden Markov model and Gaussian mixture model 隐马尔可夫与高斯混合) 模型进行识别。一个 HMM-GMM 模型对应一个孤

立词，首先对输入的语音进行分帧，对每帧计算 MFCC (Mel Frequency Cepstral Coefficients 梅尔频率倒谱系数) 特征，得到一组描述语音信号能量在不同频率范围的分布的特征向量，之后对 HMM-GMM 模型进行训练。解码过程一般采用 Viterbi 算法（在由状态与时间构成的栅格（trellis）图上搜索最优路径的动态规划算法）。识别时输入一组特征向量，对每帧用 GMM 计算每个状态产生该帧观测的概率，结合 HMM 的转移概率，用 Viterbi 算法进行路径搜索，得到最大概率值，这样就获得了最后的识别结果。

本实验只涉及最简单的案例——10 个独立词的识别。程序上，以测试集为例，音频的类别为文件名下划线后面的数字，例如`1_1.wav`。可以使用一个录音程序录制训练音频数据：

```
# %% 1. 录制 0-9 数字语音
import os, json, soundfile as sf
import pyaudio, numpy as np

RATE   = 48000       # 48 kHz 采样，特征提取时再重采样到 8 kHz
CHUNK  = 2000
LENGTH = 1.0         # 每个数字 1 秒
N_EACH = 10          # 每个数字录 10 次，共 100 条

p = pyaudio.PyAudio()
stream = p.open(format=pyaudio.paInt16, channels=1,
                rate=RATE, input=True, frames_per_buffer=CHUNK)

def record_one(sec):
    frames = []
    for _ in range(int(RATE / CHUNK * sec)):
        data = stream.read(CHUNK, exception_on_overflow=False)
        frames.append(np.frombuffer(data, np.int16))
    return np.concatenate(frames).astype(np.float32) / 32768

# 创建目录
os.makedirs("digits_wav", exist_ok=True)
meta = []
for digit in range(10):
    input(f"\n请准备读数字 {digit}，按回车开始录音（共 {N_EACH} 次）")
    for idx in range(N_EACH):
        print(f"录音 {idx+1}/{N_EACH} ...", end="")
        wav = record_one(LENGTH)
        fname = f"digits_wav/{digit}_{idx}.wav"
        sf.write(fname, wav, RATE)
        meta.append({"file": fname, "label": digit})
        print(" 完成")

with open("digits_wav/meta.json", "w") as f:
    json.dump(meta, f, indent=2)

stream.stop_stream(); stream.close(); p.terminate()
```

这个程序将音频数据保存在 `digits_wav` 目录下，并且创建了一个 json 文件，用来标记数据文件对应的标签。

之后的特征提取直接调用 librosa 中的函数实现 MFCC 算法，这里把音频重新采样为 8KHz，可以获

得更小的模型，减少计算量。

```
def extract_mfcc(path, n_mfcc=13):
    y, sr = librosa.load(path, sr=8000)
    mfcc  = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=n_mfcc, n_fft=256, hop_length
        =80)
    return mfcc.T   # (time, n_mfcc)
```

我们利用 hmmlearn 工具包搭建 HMM-GMM，因为需要识别 10 个独立词，需要初始化 10 个独立的 HMM-GMM 模型，即一个 HMM-GMM 模型的列表 models：

```
from hmmlearn import hmm
# 训练 10 个 GMM-HMM
models = []
for digit in range(10):
    model = hmm.GMMHMM(
                n_components=5,       # 隐状态数
                n_mix=3,              # 每个状态 GMM 混合数
                covariance_type="diag",
                n_iter=20,
                verbose=False,
                random_state=42)
    models.append(model)
```

各个参数的意义：

- n_components: 每个孤立词中的状态数
- n_mix: 每个状态包含的混合高斯数量
- covariance_type: 协方差矩阵的类型
- n_iter: 训练迭代次数
- random_state: 随机数种子，用来保证结果稳定

在语音识别中，HMM-GMM 模型通常以音素（phoneme）为基本建模单元。每个音素由一个隐马尔可夫模型（HMM）表示，该 HMM 通常包含 3 到 5 个状态，用于刻画音素内部的时序动态特性（如起始、稳态和结尾）。

每个 HMM 状态的观测概率由高斯混合模型（GMM）建模。对于一个 D 维的声学特征向量 $\mathbf{o} \in \mathbb{R}^D$，状态 j 的观测概率为：

$$b_j(\mathbf{o}) = \sum_{m=1}^{M} w_{jm} \cdot \mathcal{N}(\mathbf{o} \mid \boldsymbol{\mu}_{jm}, \boldsymbol{\Sigma}_{jm})$$

其中 w_{jm} 是第 m 个高斯分量的权重，$\boldsymbol{\mu}_{jm} \in \mathbb{R}^D$ 是均值向量，$\boldsymbol{\Sigma}_{jm} \in \mathbb{R}^{D \times D}$ 是协方差矩阵，M 是混合高斯分量数。

当观测特征为一维时，使用一维高斯分布；实际中特征多为多维（如 MFCC），则使用多维高斯分布。高斯分布的参数包括均值向量 $\boldsymbol{\mu}$ 和协方差矩阵 $\boldsymbol{\Sigma}$。

在 `hmmlearn` 库的 `GaussianHMM` 类中：

- `means_`：形状为 (N, D) 的矩阵，N 为状态数，D 为特征维数，表示每个状态的高斯均值。
- `covars_`：结构依赖于 `covariance_type`：
  - `"full"`：每个状态有独立的协方差矩阵，形状为 (N, D, D)；
  - `"tied"`：所有状态共享一个协方差矩阵，形状为 (D, D)；

– `"diag"`：对角协方差，非对角元素为 0，对角元素可不同，形状为 (N, D)；

– `"spherical"`：球形协方差，协方差矩阵为 $\sigma^2 I$，只需估计方差 σ^2，形状为 $(N,)$。

一个词的发音由其音素序列的 HMM 串联构成，通过状态转移网络进行解码，最终识别出最可能的词序列。

为了评估模型效果，先把数据划分为训练集和测试集（每类前 80% 训练、后 20% 测试），并且只在训练集上拟合归一化参数。利用 sklearn 的 StandardScaler 可以完成标准化，相关代码如下：

```
from sklearn.preprocessing import StandardScaler

# 划分训练集与测试集：每类前 80% 训练，后 20% 测试
train_meta, test_meta = [], []
for d in range(10):
    files_d = [m for m in meta if int(m["label"]) == d]
    k = int(len(files_d) * 0.8)
    train_meta += files_d[:k]
    test_meta  += files_d[k:]

def extract_feats(meta_list, scaler=None):
    """提取特征；scaler 不为 None 时同时做标准化"""
    out = [[] for _ in range(10)]
    for m in meta_list:
        f = extract_mfcc(m["file"])
        if scaler is not None:
            f = scaler.transform(f)
        out[int(m["label"])].append(f)
    return out

train_data = extract_feats(train_meta)
all_feats = np.vstack([f for d in train_data for f in d])  # 遍历数据并堆叠在一起
scaler = StandardScaler()
scaler.fit(all_feats)                          # 只在训练集上计算均值标准差

train_data = extract_feats(train_meta, scaler)  # 标准化训练数据
test_data  = extract_feats(test_meta, scaler)   # 标准化测试数据
```

然后，用同一种类的数据训练特定的模型：

```
for digit in range(10):
    X = np.concatenate(train_data[digit])         # 获取标准化后的特征数据
    lengths = [len(x) for x in train_data[digit]]  # 获取数据长度
    models[digit].fit(X, lengths)                  # 模型训练
```

模型训练完毕后，对待测试的数据分别用十个模型打分，选出得分最高的为识别结果。注意打分时要用平均每帧对数似然（`m.score(feat)/len(feat)`），避免语音时长不同带来的偏差；待识别数据也要先经过与训练相同的标准化。

```
def predict(wav):
    feat = extract_mfcc(wav)         # 提取声学特征
    feat = scaler.transform(feat)    # 标准化（与训练时保持一致）
    # 平均每帧对数似然，避免时长差异的影响
```

```
    scores = [m.score(feat) / len(feat) for m in models]
    return np.argmax(scores), scores

# 在测试集上统计识别率
correct = total = 0
for m in test_meta:
    pred, scores = predict(m["file"])
    total += 1
    correct += (pred == int(m["label"]))
print(f"测试集识别率:␣{correct}/{total}␣=␣{correct/total:.1%}")
```

为了让训练好的模型可以在后续的代码中使用,可以使用 joblib 模块对模型参数进行保存,例如保存和加载的代码如下所示:

```
# 利用 joblib 保存生成的 hmm 模型和归一化参数
joblib.dump({"models": models, "scaler": scaler}, "digit_hmm.pkl")

# 导入 hmm 模型和归一化参数
bundle = joblib.load("digit_hmm.pkl")
models, scaler = bundle["models"], bundle["scaler"]
```

10.3 动手实验:语音控制系统

10.3.1 实验目的

1. 了解开发板音频接口的使用方法。
2. 掌握语音识别基本算法。

10.3.2 预习要求

1. 使用录制训练音频的程序,提前录制训练数据(实验室环境嘈杂,训练效果不好)。
2. 阅读 HMM-GMM 相关资料,了解算法基本原理

10.3.3 实验内容

10.3.3.1 测试麦克风

根据提供的代码,测试麦克风录音效果,在 JupyterLab 中,可以使用 display 函数将音频数据在浏览器中播放。

```
from IPython.display import Audio, display
display(Audio(wav, rate=RATE))
```

10.3.3.2 语音命令的训练与识别

实现 0 到 9 十个数字的语音识别系统,实时录入一段音频,输出对这个音频的识别结果。可以先用固定时长录音测试,例如:

```
# 实时录入一段音频并输出识别结果
wav = record_one(1.0)                    # 录制 1 秒语音
```

```
sf.write("tmp.wav", wav, RATE)
pred, scores = predict("tmp.wav")
print(f"识别结果:␣{pred}, 各模型得分:␣{np.round(scores,␣2)}")
```

10.3.3.3 完成简单的语音控制系统

通过开发板实现对控制命令的识别，完成简单的语音控制系统。例如用来控制 LED 灯的亮度，用来输入数字等。

【选做】在识别命令时，取消使用按键，自动识别是否有命令，并对命令做出反应。

【选做】GMM-HMM 模型的性能并不是很优秀，可以换成 LSTM 模型，并重新进行训练，以提高识别率。

10.3.4 提高识别率的建议

GMM-HMM 数字识别的效果受特征、模型结构和数据质量影响很大。如果测试集识别率不理想，可以从易到难依次尝试下面的方法，每改一处就在测试集上重新评估，确认是否真的有效：

1. **扩充特征为 39 维**。13 维 MFCC 只刻画了每一帧的静态频谱，加入一阶、二阶差分（`librosa.feature.delta`）后可以描述特征的时序变化，通常能显著提升识别率：

```
mfcc   = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13, n_fft=256, hop_length=80)
delta  = librosa.feature.delta(mfcc, order=1)
delta2 = librosa.feature.delta(mfcc, order=2)
feat   = np.vstack([mfcc, delta, delta2]).T   # (time, 39)
```

2. **裁剪首尾静音**。用 `librosa.effects.trim` 去掉录音前后的静音段，再在两端各补约 60ms 静音，减少端点噪声对特征的影响；也可以调整帧长、帧移（如 25ms 帧长、10ms 帧移）。
3. **使用“从左到右”（Bakis）拓扑**。语音状态随时间单向推进，`GMMHMM` 默认允许任意状态之间跳转，不符合发音过程。训练前手工设置初始概率与转移矩阵，并令 `init_params` 和 `params` 都为 `"mc"`，使训练只更新均值和协方差：

```
n = model.n_components
model.startprob_ = np.zeros(n); model.startprob_[0] = 1.0
model.transmat_  = np.eye(n) * 0.6
for i in range(n - 1):
    model.transmat_[i, i + 1] = 0.4
model.transmat_[-1, -1] = 1.0
```

4. **保证训练数值稳定**。`hmmlearn` 默认的协方差先验可能使某些高斯成分的方差塌缩为 0，训练出现 NaN；可将 `covars_prior` 和 `covars_weight` 设为 1.0 缓解。
5. **让模型充分收敛**。把 `n_iter` 增大到 100 以上并设置 `tol=1e-4`；还可以在 3–5 个状态、4–8 个混合分量的网格中搜索，用测试集识别率选择最优参数。
6. **扩充训练数据**。录制更多、更多人的数据；训练集与测试集最好来自不同录音批次，否则识别率会虚高。
7. **数据增强**。对波形叠加随机噪声、微调语速或音调来扩充数据，例如 `librosa.effects.time_stretch` 和 `librosa.effects.pitch_shift`。
8. **实时识别加语音活动检测（VAD）**。用短时能量自动判断语音起止（对应选做内容），避免把静音和环境噪声录进语音段。
9. **环境嘈杂时先降噪**。可以在识别前用谱减法（估计噪声谱并从信号谱中减去）处理录音。

10. **更换更强的模型**。GMM-HMM 属于经典浅层模型，可以改用 LSTM/CTC 或端到端预训练模型（如 Whisper），识别率通常会有大幅提升（但对于本实验平台，训练时间也会大大提高，建议使用 GPU 训练）。

10.4 思考题

1. 识别打分时，为什么要用平均每帧对数似然 `m.score(feat)/len(feat)`，而不是直接比较 `m.score(feat)`？如果两段录音时长相差很大，直接比较会带来什么问题？
2. 为什么 `StandardScaler` 只能在训练集上拟合，而不能用全部数据（含测试集）？若用全部数据拟合，测得的测试集识别率会怎样变化？为什么？
3. 本实验的命令词识别为什么只需要声学模型 $P(O|W)$，不需要语言模型 $P(W)$？如果要把识别对象从 10 个孤立词扩展到连续数字串，还需要增加哪些模块？

第十一章　自然语言处理入门

11.1　词向量的编码与处理方法

11.1.1　WordNet 简介

WordNet 是由一个由普林斯顿大学认知科学实验室在心理学教授乔治•A•米勒的指导下建立和维护的英语字典。WordNet 根据词条的意义将它们分组，每一个具有相同意义的词条组称为一个 synset（同义词集合）。WordNet 为每个 synset 提供了简短，概要的定义，并记录不同 synset 之间的语义关系。在 WordNet 中，名词、动词、形容词和副词各自被组织成一个同义词网络，其中名词网络的主干是蕴含关系的层次（上位/下位关系），占据了关系中的将近 80%，层次中的最顶层是 11 个抽象概念，称为基本类别始点（unique beginners），例如实体（entity）等，层次越深，对应的 synset 定义越具体。如图11.1上位 (Hypernym): 一个词比给定单词更具有一般意义；下位 (Hyponym): 一个词比给定单词有更具体的意义；部分 (meronym): 一个词是给定单词的一部分；整体 (holonym): 一个词是给定单词的全部或整体。

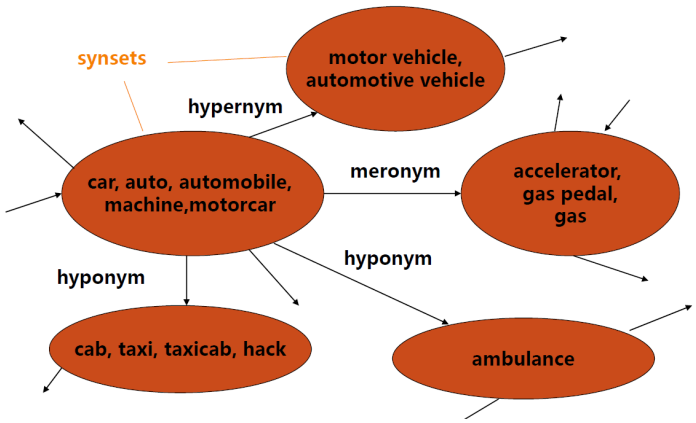


图 11.1: A WordNet Snapshot

11.1.2　基于语义词典的词汇相似度计算

词汇相似度（word similarity）指的是词汇间基于词义的关系，两个词汇之间具有越多的共性越相似。注意区分词汇相似度（word similarity）与词汇相关性（word relatedness），例如 car 与 bicycle 相似，而 car 与 gasoline 相关但不相似。使用语义词典计算词汇相似度时主要有两种方法，分别是 path based similarity 和 information content similarity，下面将会进行简单的介绍。

顾名思义，path based similarity 主要考虑两个词在词典层次结构中的相对位置，两个词在词典层次结构中越相邻，这两个词越相似。

图11.2 为 path based similarity 的一个示意，词汇间虚线连线上的数字表示它们之间路径的长度，路径越短代表相似度越高。在此基础上，有多种改进后的词汇相似度计算方法，例如 Wu-Palmer 相似度考虑

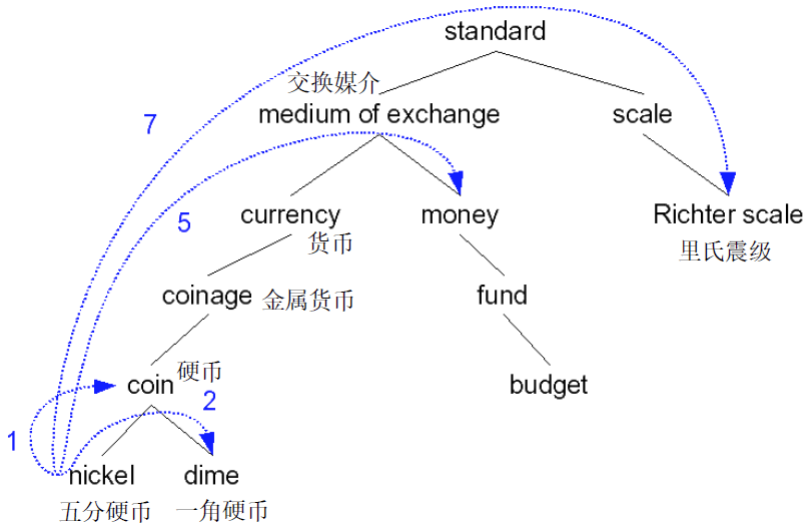


图 11.2: Path Based Similarity 示意

两个词汇和最低公共祖先(LCS,又称最近公共父节点)的路径深度,计算方法如下所示。

$$Sim_{wup} = \frac{2 * depth(LCS)}{depth(concept_1) + depth(concept_2)}$$

information content similarity 与上述方法稍有不同,首先定义 P(concept) 表示从一个语料库中随机选择一个词,这个词属于概念 concept 的概率。

words(c): 概念c所包容的词集(包含子孙后代节点)

N: 词语总数

$$P(c) = \frac{\sum_{w \in words(c)} count(w)}{N}$$

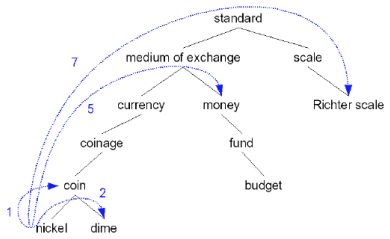


图 11.3: P(concept) 含义

C(concept) = -log P(concept) 表示概念 concept 包含的信息内容。Lin 相似度考虑词汇和它们的最低公共祖先所包含的信息内容来计算词汇相似度。

$$Sim_{lin} = \frac{2 * IC(LCS)}{IC(concept_1) + IC(concept_2)}$$

NLTK 包含众多一系列的语料库,这些语料库可以通过 nltk.package 导入使用。每一个语料库可以通过一个叫做“语料库读取器”的工具读取语料库,例如:nltk.corpus。前述 WordNet 词典也是 NLTK 语料库的一部分。

11.1.3 基于语料统计的词汇相似度计算

Word2Vec 是 google 在 2013 年推出的一个 NLP 工具,它的特点是能够将单词转化为向量来表示,这样词与词之间就可以定量的去度量他们之间的关系,挖掘词之间的联系。Word2Vec 的思路是通过训练,将原来 One-Hot 编码的每个词都映射到一个较短的词向量上来,而这个较短的词向量的维度可以在训练时根据任务需要来指定,用 Distributed Representation 表示的较短的词向量,就可以较容易的分析词之

间的关系。(one-hot representation：用一个很长的向量来表示一个词，向量的长度为词典的大小，向量的分量只有一个 1，其他全为 0，1 的位置对应该词在词典中的位置；distributed Representation：通过训练将某种语言中的每一个词映射成一个固定长度的短向量，将所有这些向量放在一起形成一个词向量空间，而每一向量则为该空间中的一个点，在这个空间上引入“距离”，则可以根据词之间的距离来判断它们之间的（词法、语义上的）相似性。)

Word2Vec 的训练模型本质上是只具有一个隐含层的神经元网络。

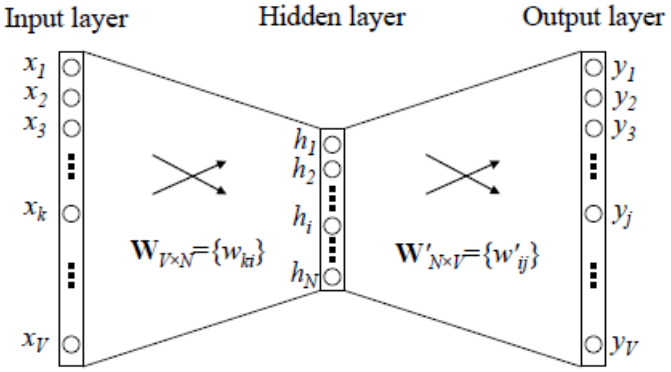


图 11.4: Word2Vec 模型结构

它的输入是采用 One-Hot 编码的词汇表向量，它的输出也是 One-Hot 编码的词汇表向量。使用所有的样本，训练这个神经元网络，等到收敛之后，从输入层到隐含层的那些权重，便是每一个词的采用 Distributed Representation 的词向量。这样就把原本维数为 V 的词向量变成了维数为 N 的词向量（N 远小于 V），并且词向量间保留了一定的相关关系。

Mikolov 在关于 Word2Vec 的论文中提出了 CBOW 和 Skip-gram 两种模型，CBOW 适合于数据集较小的情况，而 Skip-Gram 在大型语料中表现更好。其中 CBOW 如图11.5左部分所示，使用围绕目标单词的其他单词（语境）作为输入，在映射层做加权处理后输出目标单词。与 CBOW 根据语境预测目标单词不同，Skip-gram 根据当前单词预测语境，如图11.5右部分所示。假如我们有一个句子“There is an apple on the table”作为训练数据，CBOW 的输入为（is,an,on,the），输出为 apple。而 Skip-gram 的输入为 apple，输出为（is,an,on,the）。

利用 Word2Vec 模型，得到词语的词向量表示后，采用余弦相似度计算词语相似度。

$$similarity = cos(\theta) = \frac{A \cdot B}{\|A\|\|B\|} = \frac{\sum_{i=1}^{n} A_i \times B_i}{\sqrt{\sum_{i=1}^{n}(A_i)^2} \times \sqrt{\sum_{i=1}^{n}(B_i)^2}}$$

ensim 是一款开源的第三方 Python NLP 工具包，用于从原始的非结构化的文本中，无监督地学习到文本隐层的主题向量表达。它支持包括 TF-IDF，LSA，LDA，和 word2vec 在内的多种主题模型算法，支持流式训练，并提供了诸如相似度计算，信息检索等一些常用任务的 API 接口。在 gensim 中，word2vec 相关的 API 都在包 gensim.models.word2vec 中，和算法有关的参数都在类 gensim.models.word2vec.Word2Vec 中，需要注意的参数有：

1) sentences: 我们要分析的语料，可以是一个列表，或者从文件中遍历读出；

2) size: 词向量的维度，默认值是 100。这个维度的取值一般与我们的语料的大小相关，如果是不大的语料，比如小于 100M 的文本语料，则使用默认值一般就可以了。如果是超大的语料，建议增大维度。

3) window：即词向量上下文最大距离，这个参数在我们的算法原理篇中标记为 c，window 越大，则和某一词较远的词也会产生上下文关系。默认值为 5。在实际使用中，可以根据实际的需求来动态调整这个 window 的大小。如果是小语料则这个值可以设的更小。对于一般的语料这个值推荐在 [5,10] 之间。

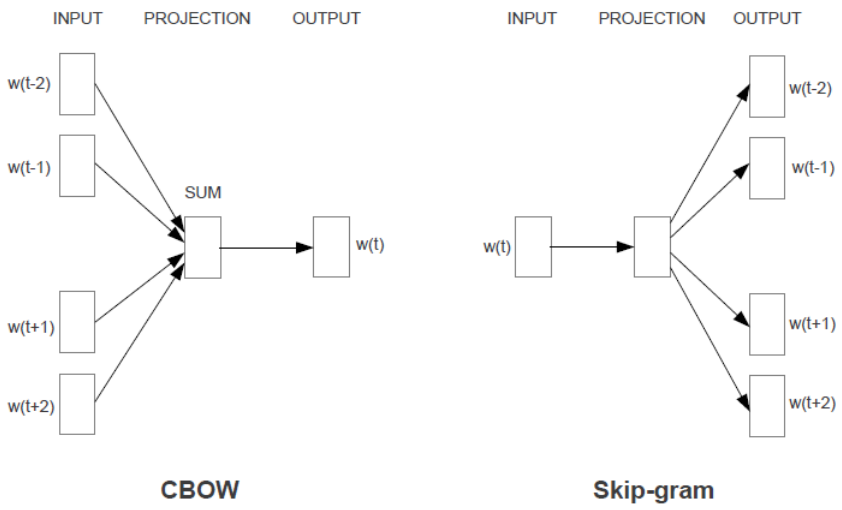


图 11.5: CBOW 模型与 Skip-gram 模型

4) sg: 即 word2vec 两个模型的选择。如果是 0，则是 CBOW 模型，是 1 则是 Skip-Gram 模型，默认是 0 即 CBOW 模型。

5) hs: 即 word2vec 两个解法的选择，如果是 0，则是 Negative Sampling，是 1 的话并且负采样个数 negative 大于 0，则是 Hierarchical Softmax。默认是 0 即 Negative Sampling。

6) negative: 即使用 Negative Sampling 时负采样的个数，默认是 5。推荐在 [3,10] 之间。

7) min_count: 需要计算词向量的最小词频。这个值可以去掉一些很生僻的低频词，默认是 5。如果是小语料，可以调低这个值。

11.1.4 结果评估方法——Spearman's rank correlation coefficient

Mturk-771 数据集提供了每对词语的相似度向量，需要与我们得到的词语相似度向量进行比较，评估各种方法效果优劣。pandas 是基于 NumPy 的一种工具，该工具是为了解决数据分析任务而创建的，提供了大量能使我们快速便捷地处理数据的函数和方法。pandas 中 DataFrame 对象的 corr() 方法用来计算 DataFrame 对象中所有列之间的相关系数（df.corr()，包括'pearson'相关系数、'kendall'相关系数和'spearman'秩相关）。相关性是两个变量之间关联的度量，用以检查两个变量之间变化趋势的方向以及程度，值范围-1 到 +1，0 表示两个变量不相关，正值表示正相关，负值表示负相关，值越大相关性越强。当两个变量都有正态分布时，很容易计算和解释。而当我们不知道变量的分布时，我们必须使用非参数的秩相关（Rank Correlation，或称为等级相关）方法。秩相关是指使用变量之间序数的关联（而不是特定值）来量化变量之间的关联的方法。有序数据是具有标签值并具有顺序或秩相关的数据，例如：'低'，'中'和'高'。Spearman 秩相关以 Charles Spearman 命名，这个统计方法量化了等级变量与单调函数相关联的程度，即递增或递减的关系。在没有重复数据的情况下，如果一个变量是另外一个变量的严格单调函数，那么二者之间的 spearman 秩相关系数就是 1 或 +1，称为完全 spearman 相关。如果 Y 随 X 的增加而增加，则 spearman 秩相关系数是正的，否则是负的。spearman 秩相关系数为 0 表示随着 X 的增加 Y 没有增加或减小的趋势，随着 X 和 Y 的越来越接近严格单调函数管系，spearman 秩相关系数在数值上越来越大。

由于基于词典和基于语料的方法中，都会有异常值的出现（信息内容文件没有条目，词典收入词条有限，语料中词汇有限），会对 Pearson correlation coefficient 的结果造成较大干扰，而 Spearman's rank correlation coefficient 对异常值不敏感。

11.2 Twitter 文本情感分析

11.2.1 情感分析介绍

情感分析是文本分类的一个分支，对带有情感色彩（正向/中立/负向）的主观性文本进行分析，以确定该文本的观点、喜好和情感倾向。主流方法有基于情感词典的情感分析和基于机器学习的情感分析两种方式。

基于情感词典的情感分析是指根据已构建的情感词典，对待分析文本进行文本处理，抽取情感词计算文本的情感倾向，最终的分类效果主要取决于情感词典的完善性。

基于机器学习的情感分析需要对文本进行一系列预处理（去除停用词、Stem、Tokenization 等），将文本转化为词向量表示后，可以利用逻辑回归、朴素贝叶斯及支持向量机等方法进行分类。

SemEval 数据集完成基本任务是推特的情感分析（Sentiment Analysis in Twitter）。对于推特的文本情感分析基于 SemEval 数据集始于 2013 年，之后任务和数据都在不断发展为更复杂。在 13 年到 15 年，任务是简单给一个推特文本，然后进行文本情感分类，分为 3 类（积极、消极、中立），称为任务 A；于 2015 年，在任务和任务中引入了 Topic 的概念，任务升级为给一个推特，并给一个 topic；推断推特内容关于这个 topic 的情感倾向，积极或消极（任务 B）；于 2016 年，引入了两个分支，一是加入了 tweet quantification，也就是推特的量化分析；二是 five-point ordinal classification，也就是之前是推特的三分类，16 年拓展为五分类。

SemEval-2017 任务 4 由五个子任务组成，每个都提供阿拉伯语和英语：

1.Subtask A: 分析一个推特的情感，可以分为积极、消极、中立

2.Subtask B: 给一个推特，并给一个 topic；推断推特内容关于这个 topic 的情感倾向，积极或消极。

3.Subtask C: 在 B 任务的基础上，更加精细地分类，分为非常积极、弱倾向积极、中立、弱倾向于消极、非常消极（五个程度）

4.Subtask D: 关于一个 topic，给出一组的推特，估计这些推特在积极和消极的分布

5.Subtask E: 关于一个 topic，给出一组的推特，估计这些推特在五个情感程度的分布。

本单元基于 SemEval2017 Task 4 Subtask A: Message Polarity Classification，对给定的英文 twitter 进行情感分析，将文本划分为 positive，neutral 和 negative 三种情感。

11.2.2 双向 LSTM 介绍

LSTM 的全称是 Long Short-Term Memory，它是 RNN（Recurrent Neural Network）的一种。由于其设计的特点，LSTM 非常适合用于对时序数据（如文本数据）的建模。BiLSTM 是 Bi-directional Long Short-Term Memory 的缩写，是由前向 LSTM 与后向 LSTM 组合而成。两者在自然语言处理任务中都常被用来建模上下文信息。

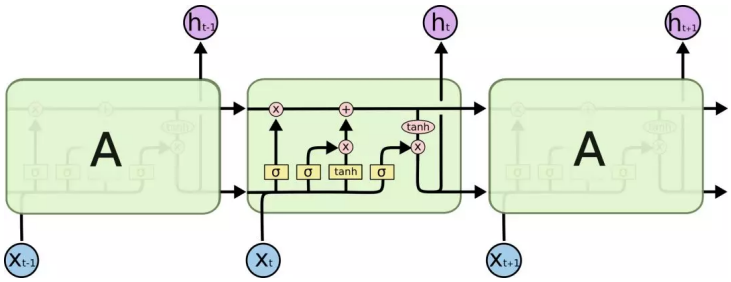
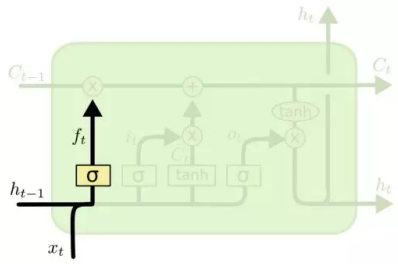


图 11.6: LSTM 总体框架

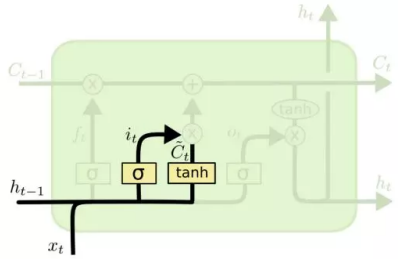
LSTM 的总体框架如图11.6 所示，由 t 时刻的输入词 X_t ，细胞状态 C_t，临时细胞状态 $\tilde{C}_t$ 隐层状态 hh_t，遗忘门 f_t，记忆门 i_t 和输出门 o_t 组成。其计算过程可以概括为，通过对细胞状态中信息遗忘和记忆新的信息使得对后续时刻计算有用的信息得以传递，而无用信息被抛弃。



$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \ + \ b_f\right)$$

图 11.7: 遗忘门计算

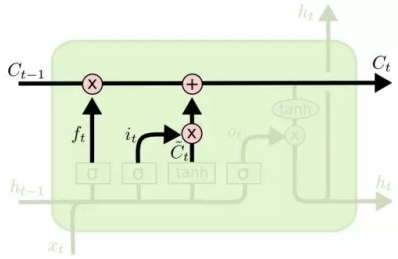
首先计算遗忘门，选择要遗忘的信息，输入为前一层的隐层状态 h_{t-1}，当前时刻的输入词 X_t，输出为遗忘门的值 f_t，如图11.7所示。其中 W_f 为计算遗忘门对应的权重，b_f 为偏置，σ 为激活函数。



$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \ + \ b_i\right)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \ + \ b_C)$$

图 11.8: 记忆门和临时细胞状态计算

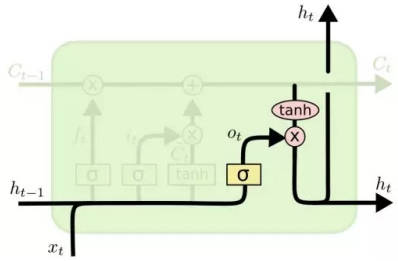
计算记忆门，选择要记忆的信息，输入为前一层的隐层状态 h_{t-1}，当前时刻的输入词 X_t，输出为记忆门的值 i_t 和临时细胞状态 $\tilde{C}_t$，如图11.8 所示。



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

图 11.9: 当前时刻细胞状态计算

计算当前时刻细胞状态，输入为记忆门的值 i_t，遗忘门的值 f_t，临时细胞状态 $\tilde{C}_t$ 和上一时刻细胞状态 C_{t-1}，输出为当前时刻细胞状态 C_t，如图11.9 所示。



$$o_t = \sigma\left(W_o \ [h_{t-1}, x_t] \ + \ b_o\right)$$
$$h_t = o_t * \tanh\left(C_t\right)$$

图 11.10: 计算输出门和当前时刻隐层状态

计算输出门和当前时刻隐层状态，输入为前一时刻的隐层状态 h_{t-1}，当前时刻的输入词 X_t 和当前时刻细胞状态 C_t，输出为输出门的值 o_t 和隐层状态 h_t，如图 11.10 所示。

最终可以获得与句子长度相同的隐层状态序列 $h_0, h_1, ..., h_(n-1)$。

但是 LSTM 存在一个问题，即只能编码从前到后的信息，而无法编码从后到前的信息。双向 LSTM 由一个前向 LSTM 和一个后向 LSTM 组成，因此具备了编码从后到前信息的能力。以编码“我爱中国”为例，如图11.11 所示，分词操作将句子划分为 [“我”, ”爱”, ”中国”] 三个词，前向 LSTML 得到 h_L0, h_L1, h_L2，后向 LSTMR 得到 h_R0, h_R1, h_R2，将前向和后向的隐向量拼接得到 $[h_{L0}, h_{R0}], [h_{L1}, h_{R1}], [h_{L2}, h_{R2}]$，即 h_0, h_1, h_2。

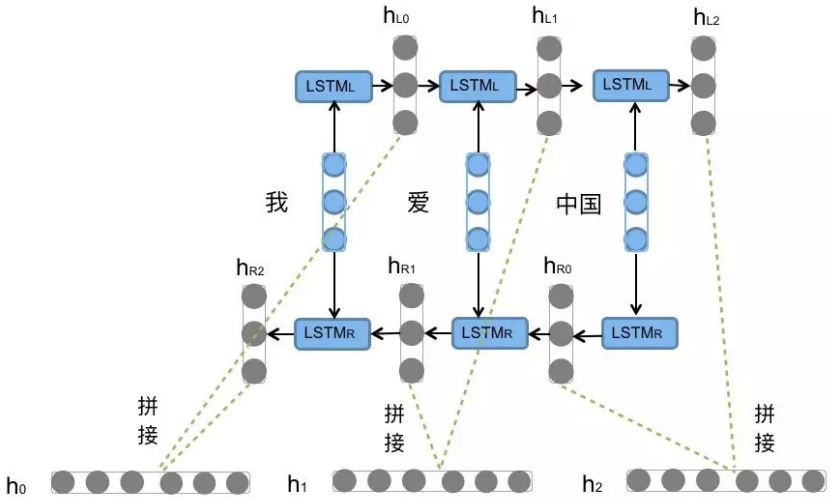


图 11.11: 双向 LSTM 编码句子示意

```
    # PyTorch 双向 LSTM 模型定义
    # bidirectional 控制是否是双向 LSTM
    self.lstm = nn.LSTM(embedding_dim, hidden_dim, layer_num, dropout=drop_prob,
        batch_first=True, bidirectional=self.bidirectional)
```

利用双向 LSTM 编码文本信息后，使用全连接层进行文本的情感分类。

```
    # 全连接层做情感分类
    self.fc = nn.Linear(self.layer_num * hidden_dim, output_size)
```

PyTorch 提供的 LSTM 函数包括 7 个参数，含义如下：

- input_size: 输入特征维数，与示例中 embedding_dim 对应，代表每个单词用多少维向量表示，通常选择数百量级的向量维度
- hidden_size: 隐藏层的特征维度
- num_layers: LSTM 隐层的层数，默认为 1
- bias: 偏移值，表示隐层状态是否带 bias，默认为 True
- batch_first：为 True 时，表示输入输出的数据格式为 (batch, seq, feature)
- dropout：除最后一层，每一层以特定的概率进行 dropout，默认为 0
- bidirectional: 表示是否是双向 LSTM

其他参数：

- vocab_size：表示根据当前语料构建的词典大小
- ouput_size：分类判别的取值数量，此处为 positive/neutral/negative 三种
- epochs：表示模型训练迭代次数，需要根据数据集大小和模型拟合能力确定。设置太小会欠拟合，太大会过拟合

- clip：为梯度裁剪阈值，防止梯度爆炸/梯度消失
- learning_rate: 为模型学习率，设置过小容易使得训练速度过慢或陷入局部最优点，过大可能导致震荡而无法收敛

11.3 未名教学二号集群使用说明

请至少提前一周申请未名教学二号集群账号：http://hpc.pku.edu.cn/guide.html，申请成功后可以使用 ssh 或 Putty 远程登陆集群。

集群使用教程参见：http://hpc.pku.edu.cn/_book/guide/soft_env/python.html，重点参考“5. 使用软件“部分的”5.5.python”中的内容。

在集群上运行 Python 程序时，每个人需要在 conda 里创建一个自己的虚拟环境，再在自己的虚拟环境里运行，所用到的库都需要自己安装，但在集群上安装库比较方便。

```
1    module avial anaconda         # 查看可用的 conda 环境
2    module load anaconda/3.7.1    # 载入可用的 conda 环境
3    conda create -n myEnvPython python=<version>       # 创建虚拟环境
4    source activate myEnvPython  # 进入已创建的虚拟环境
```

进入虚拟环境后通过 pip install 或 conda install 安装需要的库

任务提交方式：参考上述链接中的“6. 作业调度系统”，注意：不能直接在集群上运行任务，需要按教程中的方式编写脚本后，通过提交任务的方式执行。

```
1    sinfo 查看可用的分区和节点
2    sbatch 提交任务
3    squeue 查看任务运行状态
```

脚本 job.sh 的示例如下：

```
1  #!/bin/bash
2  #SBATCH -o nohup.out
3  #SBATCH -p compute
4  #SBATCH -q normal
5  #SBATCH -J word2vec
6  #SBATCH --time=24:00:00
7
8  python3 main.py
```

通过 sbatch job.sh 提交任务，通过 squeue 查看任务状态。

11.4 动手实验：文本的情感分析

11.4.1 实验目的

1. 学习词向量的编码与处理方法
2. 初步了解自然语言处理的基本方法
3. 学习计算机集群的远程登陆与作业管理

11.4.2 实验内容

11.4.2.1 词汇相似度计算实验

基于 Mturk-771 数据集进行实验，计算英文词汇的相似度，对比 3 种不同的词汇相似度计算方法的效果。Mturk-771 数据集存放在 data 文件夹中，数据格式为（word1, word2, sim），表示 word1 与 word2 的相似度为 sim。

Word2Vec 模型基于 text8（wikipedia）语料进行训练，该语料存放在 data 文件夹中。

utils.py 提供了读写.xlsx 文件、计算 Spearman's rank correlation coefficient 和绘图函数。

ContextBased.py 提供了基于 Word2Vec 模型的词汇相似度计算方法。

DictionaryBased.py 提供了基于语义词典的词汇相似度计算方法。

实验时，运行 main.py 即可。

```
# 返回基于语义词典的相似度
wupAns, linAns = DicSimilarity(rawData)
#返回基于 Word2Vec 的相似度
ConAns = ConSimilarity(rawData, vecLength=150, isTrain=True)
```

评估方法采用 Spearman's rank correlation coefficient，理由是 Mturk-771 数据集提供了每对词语的相似度向量，需要与我们得到的词语相似度向量进行比较，评估各种方法效果优劣。

1）在开发板上运行已训练好的模型，运行 main.py 即可，比较不同方法的优劣，同学们也可以尝试其它的基于语义词典的方法；

2）在未名二号教学集群上对 Word2Vec 模型进行重新训练，须调整 Word2Vec 模型的超参数，重新训练后，将模型下载到开发板观察效果。

11.4.2.2 Twitter 文本情感分析实验

数据集存放在 data 文件夹中，data_processor.py 文件中提供了文本预处理相关的函数。model.py 中定义了模型结构。

```
# 模型超参数设置
batch_size = 25
vocab_size = len(vocab_to_int) + 1
output_size = 3
embedding_dim = 400
hidden_dim = 256
n_layers = 2
epochs = 5
clip = 5
print_every = 10
learning_rate = 0.0005
```

1）在开发板上运行已训练好的模型，运行 main.py 即可；2）感兴趣的同学可以定义自己的网络结构，或修改模型的超参数，在未名二号教学集群上进行重新训练，之后将模型下载到开发板观察效果。

第十二章　大语言模型简介

12.1　Transformer 模型

Google 在 2017 的论文：Attention is All You Need 中，提出了一个新的神经网络模型：Transformer。这个模型完全采用注意力机制（Attention）处理序列数据，在性能上相对之前的 RNN 模型有了很大的提高。目前几乎所有的自然语言模型都采用 Transformer 结构作为基本的组成单元，在翻译、问答、知识处理等多个方面都取得了长足的进步。进而在 2013 年由 OpenAI 推出 ChatGPT 开始，推动了一波大语言模型广泛应用的热潮。

12.1.1　注意力机制

注意力机制（Attention Mechanism）是一种用于深度学习模型中的重要技术，旨在使模型能够在处理序列数据时更加关注输入序列中的不同部分，从而提高模型的性能。注意力机制最初被广泛用于序列到序列的翻译任务，但后来被扩展到各种其他任务中，如语音识别、图像处理和自然语言处理等。

注意力机制的核心思想是在生成输出的过程中，根据输入序列的不同部分来分配不同的权重。这使得模型可以根据输入序列的重要性动态地调整关注的位置，从而更好地捕捉输入序列之间的关系。在序列到序列的任务中，例如翻译，模型需要根据输入句子的每个单词来生成输出句子的每个单词，这就需要一种方法来确定输入句子中每个单词对于生成当前输出单词的重要性。

在注意力机制中，通常会计算一个注意力权重向量，该向量表示输入序列中每个位置的权重。这个权重向量可以基于不同的方法计算，但最常见的方法是使用点积、加性（一层神经网络）或缩放点积等。意力机制允许模型根据输入的不同部分分配不同的关注权重，以便更好地处理序列数据。这对于捕捉长距离依赖关系、处理变长输入序列以及改善模型性能都非常有帮助。

12.1.2　Transformer 网络结构

传统的序列模型，如循环神经网络（RNN）和长短时记忆网络（LSTM），在处理长序列数据时存在一些问题，例如难以捕捉长距离依赖关系和计算效率较低。Transformer 模型的设计就是为了解决这些问题。Transformer 模型的核心思想包括自注意力机制（Self-Attention）和多头注意力（Multi-Head Attention）。它将输入序列分别映射为查询（Q）、键（K）、值（V）向量，然后通过计算注意力权重，将值向量与权重相乘并求和，从而获得上下文信息。通过多个注意力头的组合，模型可以在不同的表示子空间中学习不同的关系。

Transformer 模型结构如图12.1 所示，其主要的组成部分包括：

编码器（Encoder）：编码器由多个相同结构的层组成，每一层都包含了多头自注意力机制和前馈神经网络。编码器将输入序列映射为一系列高维表示，其中包含了输入序列的上下文信息。

解码器（Decoder）：解码器也由多个相同结构的层组成，除了编码器的结构外，解码器还包含一个多头注意力层，用于对编码器输出进行自注意力计算，以便更好地聚焦在输入序列的不同部分。

自注意力机制（Self-Attention）：这是 Transformer 模型的关键机制，用于计算输入序列中每个位置的权重，从而捕捉序列内不同位置的依赖关系。

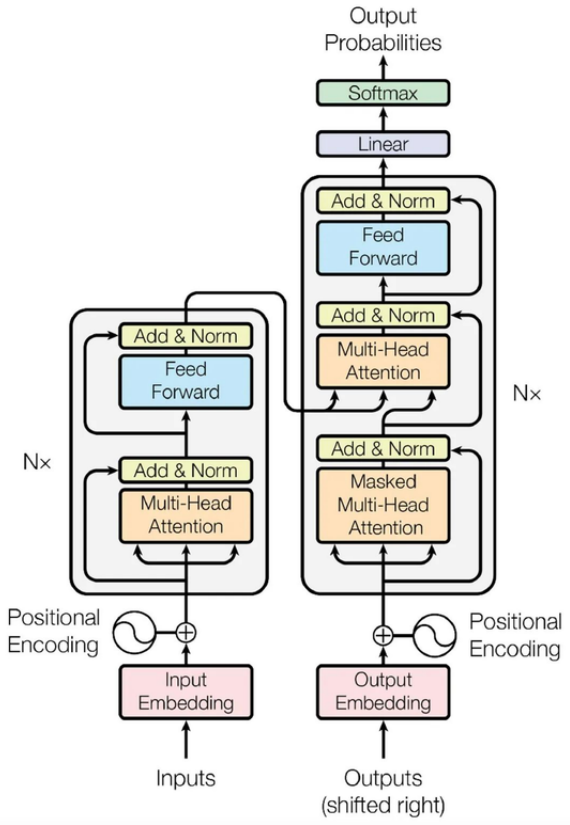


图 12.1: Transformer 网络结构

多头注意力（Multi-Head Attention）：通过多个注意力头的并行计算，模型可以学习不同类型的关系，从而更好地表示输入序列。

前馈神经网络（Feedforward Neural Network）：每个编码器和解码器层都包含一个前馈神经网络，用于在注意力计算后对表示进行进一步的非线性变换。

残差连接和层归一化：每个子层内都有残差连接和层归一化，有助于防止训练过程中的梯度消失问题。

Transformer 模型的优势在于能够并行计算，因此在训练过程中可以更高效地利用硬件资源。此外，Transformer 模型在预训练和微调的框架下，在许多自然语言处理任务中取得了显著的性能提升。例如，BERT、GPT 系列和 T5 等模型都是基于 Transformer 架构的重要变体，它们在各自的领域内都取得了领先的地位。

12.2 大语言模型

2023 年可以看作是现代大模型发展的重要分水岭，行业的主要逻辑自此从追求参数规模的” 军备竞赛”，开始向追求能力与成本平衡的实用化阶段过渡。与早期相比，当前[1]的开源生态已极为繁荣，出现了众多在特定能力上可与闭源模型抗衡的选择，且开源模型在数量和质量上均有显著提升。

12.2.1 OpenAI 系列模型

OpenAI 是当前大语言模型领域最具影响力的机构之一。其旗舰模型已演进至 GPT-5 系列。GPT-5 采用统一系统设计，包含快速响应模型（GPT-5-main）和深度推理模型（GPT-5-thinking），系统可根据对话复杂度自动路由，用户无需手动切换模型。在性能方面，GPT-5 在 AIME 数学竞赛中达到 94.6%，在

[1]本小节的信息内容截至 2026 年 7 月

SWE-bench 编程测试中达到 74.9%，幻觉错误较 GPT-4o 减少了 45%。GPT-5 首次向免费用户开放，打破了” 最新模型仅限付费” 的传统逻辑。OpenAI 模型具备强大的零样本（zero-shot）和小样本学习能力，可执行包括复杂推理、长文本生成、代码开发在内的广泛自然语言处理任务。

OpenAI 的 API 接口遵循简洁的聊天补全（Chat Completion）范式，调用方式如下：

```python
from openai import OpenAI

client = OpenAI(api_key="your-api-key")

response = client.chat.completions.create(
    model="gpt-5.1",
    messages=[
        {"role": "system", "content": "你是一位␣helpful␣助手。"},
        {"role": "user", "content": "请介绍一下大语言模型"}
    ],
    max_tokens=2048,
    temperature=0.7,
    top_p=0.9
)

print(response.choices[0].message.content)
```

其中 messages 列表用于构建对话历史，max_tokens 控制回复最大长度，temperature 调节随机性（值越高越多样），top_p 控制核采样截断阈值。

12.2.2 Anthropic 系列模型

Anthropic 是另一家领先的人工智能公司，其开发的 Claude 系列模型以出色的推理能力、长上下文处理和高度可控的输出风格著称。目前 Claude 已迭代至 Claude 4 系列，包括 Claude Opus 4 和 Claude Sonnet 4 等型号。Opus 4 被定位为” 世界上最好的编程模型”，专为长周期任务和复杂问题求解而设计；Sonnet 4 则在性能、成本和延迟之间提供了良好的平衡。Claude 4 系列支持扩展思考模式（Extended Thinking），可在推理与工具使用之间灵活切换。Anthropic 特别强调模型的安全性（Constitutional AI），在减少有害输出和提升透明度方面投入了大量研究。

Anthropic 的 API 同样采用消息（messages）接口，与 OpenAI 的风格高度相似，便于开发者迁移：

```python
from anthropic import Anthropic

client = Anthropic(api_key="your-api-key")

response = client.messages.create(
    model="claude-4.0",
    max_tokens=2048,
    temperature=0.7,
    top_p=0.9,
    system="你是一位␣helpful␣助手。",
    messages=[
        {"role": "user", "content": "请介绍一下大语言模型"}
    ]
)

```

```
print(response.content[0].text)
```

其中 system 参数用于设定全局的系统提示词，messages 记录对话轮次，max_tokens、temperature 和 top_p 的含义与 OpenAI 的对应参数一致。

12.2.3 国内 API 资源：通义千问与阿里百炼平台

对于国内开发者而言，调用海外大模型 API 常面临网络不稳定、支付门槛和合规性等问题。阿里云推出的通义千问（Qwen）系列模型及配套的百炼平台（DashScope），为国内用户提供了稳定、合规且高性价比的选择。

通义千问系列已演进至 Qwen3.8-Max，总参数达 2.4 万亿，在中文理解和多语言任务上表现优异。其 API 接口兼容 OpenAI 的调用范式，开发者只需修改 base_url 和 model 参数即可快速接入，几乎无需改动现有代码。以 Qwen-Max 模型为例，输入价格约为 $0.58/百万 Token，输出约为 $1.74/百万 Token，相比 OpenAI 同级别模型可节省约 92% 的成本。

通过阿里百炼平台（dashscope.aliyun.com）可快速获取 API Key，新用户通常享有一定的免费额度。调用方式如下：

```
from openai import OpenAI
import os

# 使用阿里百炼的 API 地址和密钥
client = OpenAI(
    api_key=os.getenv("DASHSCOPE_API_KEY"),  # 从环境变量读取 API Key
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1"  # 百炼兼容接口地
        址
)

response = client.chat.completions.create(
    model="qwen-max",  # 可替换为 qwen-plus、qwen-turbo 等
    messages=[
        {"role": "system", "content": "你是一位␣helpful␣助手。"},
        {"role": "user", "content": "请介绍一下大语言模型"}
    ],
    max_tokens=2048,
    temperature=0.7,
    top_p=0.9
)

print(response.choices[0].message.content)
```

国内主流模型厂商的 API 已基本兼容 OpenAI 格式，这意味着本实验后续的所有代码均可无缝切换至通义千问、DeepSeek、智谱 GLM 等国产模型，只需更改 base_url 和 model 名称即可。

12.2.4 开源模型生态

当前开源社区蓬勃发展，已形成从” 单点突破” 走向” 群体涌现” 的格局。

智谱 AI 开源的 GLM 系列模型已演进至 GLM-5 系列。GLM-5 在代码能力上对标 Claude Opus 4.5，擅长智能体（Agent）长程规划与执行；GLM-5.1 的编程能力已对齐 Claude Opus 4.6，支持长程任务自主工作长达 8 小时；GLM-5.2 作为最新旗舰模型，拥有 1M 无损上下文，在编程能力上达到开源 SOTA。

深度求索(DeepSeek)的 DeepSeek-V3 系列同样表现突出。该系列采用混合专家(MoE)架构,总参数量达 6710 亿,单次推理仅激活约 370 亿参数。DeepSeek-V3 在多项评测中性能与同期 GPT-4o、Claude-3.5-Sonnet 相当,而训练成本仅为约 557 万美元。其后续版本已将上下文处理能力提升至 1M Token 级别。

月之暗面于 2026 年 7 月正式开源了 Kimi K3,总参数达 2.8 万亿,成为当时全球参数规模最大的开源大模型。Kimi K3 原生支持视觉理解,拥有 100 万 Token 上下文窗口,在代码和智能体任务上已逼近最强的闭源模型。

Meta 的 LLaMA 系列也持续迭代,LLaMA 4 Maverick 采用 MoE 架构(170 亿激活参数 / 4000 亿总参数),支持 100 万 Token 上下文和原生视觉理解;LLaMA 4 Scout 则将上下文窗口拓展至 1000 万 Token,创下行业纪录。

此外,阿里巴巴于 2026 年 8 月发布了新一代基座大模型 Qwen3.8-Max,总参数达 2.4 万亿,并宣布将开源模型权重。据全球开源平台报告,中国研发的开源模型下载量已占全球总量的 41%,超越美国位居世界第一。开源开放正成为推动全球 AI 生态繁荣的重要力量。

12.3 动手实验:调用大语言模型

12.3.1 实验目的

1. 了解大语言模型的基本原理和 API 调用方法。
2. 掌握提示工程(Prompt Engineering)技巧,通过优化提示词提升模型输出质量。
3. 利用大语言模型 API 实现一个完整的主观题自动评分系统。

12.3.2 实验内容

本次实验将利用大语言模型实现对在线考试中主观题的评分。具体题目和参考答案可以自主选择,但要求评分为 0~5 分。参考题目与答案如下:

题目:“第一次工业革命有哪些意义?” 答案应该包含如下几点,每回答对一点加 1 分,最多 5 分:

- 机器的大量使用,提高了生产效率。
- 城市化进程加快,人口大幅增加。
- 增加了商品的供应,提高人民生活品质。
- 社会结构变化,催生了社会改革运动。
- 工业废料的产生,对环境造成不利影响。

12.3.3 提示词探索与优化

目标:通过与大语言模型的交互,找到能够稳定输出结构化评分结果的有效提示词。

具体任务:

1. 使用网页版或 API 方式访问任意一个大语言模型(推荐使用通义千问、DeepSeek 或智谱 GLM 的在线 demo)。
2. 设计至少 3 种不同的提示词策略,对比其效果:
   - 策略 A(零样本):直接给出题目、参考答案和评分标准,要求模型评分。
   - 策略 B(角色扮演):让模型扮演” 具有 10 年经验的阅卷老师”,再给出评分任务。
   - 策略 C(思维链):要求模型” 先逐点分析学生答案是否覆盖每个得分点,再给出总分”。
3. 使用同一份学生答案分别测试三种策略,记录每次的输出格式和评分结果。
4. 选择效果最佳的提示词方案,作为后续步骤的基础。

参考提示词模板：

```
你是一位经验丰富的阅卷老师。请根据以下评分标准，对学生答案进行评分。

【题目】
第一次工业革命有哪些意义？

【评分标准】（共5分，每点1分）
1. 机器的大量使用，提高了生产效率。
2. 城市化进程加快，人口大幅增加。
3. 增加了商品的供应，提高人民生活品质。
4. 社会结构变化，催生了社会改革运动。
5. 工业废料的产生，对环境造成不利影响。

【学生答案】
（此处填入待评分的答案）

请按以下 JSON 格式输出评分结果：
{
    "得分点覆盖": ["覆盖的得分点编号"],
    "未覆盖": ["未覆盖的得分点编号"],
    "得分": 整数(0-5),
    "评语": "简要评语"
}
```

12.3.4 API 编程实现基础评分功能

目标：编写 Python 程序，通过 API 调用大模型实现批量自动评分。

具体任务：

1. 注册并获取 API Key（推荐使用阿里百炼平台或 DeepSeek 平台，新用户有免费额度）。
2. 编写 Python 程序，从文本文件中读取学生答案（每行一个答案）。
3. 将步骤一优化好的提示词与每个学生答案组合，通过 API 发送请求。
4. 解析 API 返回的 JSON 结果，提取得分并输出到屏幕或保存到 CSV 文件。

参考代码框架：

```python
from openai import OpenAI
import os
import json

# 初始化客户端（使用国内 API）
client = OpenAI(
    api_key=os.getenv("DASHSCOPE_API_KEY"),
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1"
)

def build_prompt(student_answer):
    """构建包含评分标准的完整提示词"""
    system_prompt = "你是一位经验丰富的阅卷老师。"
```

```
    user_prompt = f"""
【评分标准】...
【学生答案】{student_answer}
请按JSON格式输出...
"""
    return system_prompt, user_prompt

def grade_answer(student_answer):
    """调用API对单个答案进行评分"""
    system_prompt, user_prompt = build_prompt(student_answer)
    response = client.chat.completions.create(
        model="qwen-max",  # 可替换为 deepseek-chat 等
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt}
        ],
        temperature=0.1,  # 低温度提高稳定性
        response_format={"type": "json_object"}  # 部分模型支持
    )
    return json.loads(response.choices[0].message.content)

# 主程序: 批量处理
with open("student_answers.txt", "r", encoding="utf-8") as f:
    answers = f.readlines()

results = []
for idx, answer in enumerate(answers, 1):
    result = grade_answer(answer.strip())
    result["序号"] = idx
    results.append(result)
    print(f"学生{idx}:{result['得分']}分")

# 保存结果
with open("scores.csv", "w", encoding="utf-8") as f:
    # 写入 CSV 头部和数据
    pass
```

12.3.5 系统优化与评估

目标:完善评分系统,进行效果评估和多模型对比。

具体任务:

1. 准备 5-10 份模拟学生答案(包括完整答案、部分正确答案、完全错误答案)。
2. 对比两个不同模型(如 qwen-max 和 deepseek-chat)在同一批答案上的评分结果,分析差异。
3. 尝试改进提示词,解决模型输出不规范(如格式错误、评分不一致)的问题:

   - 添加 Few-shot 示例(给出 1-2 个评分范例)。
   - 设置 temperature=0 增强输出确定性。
   - 要求模型额外输出" 评分理由" 便于调试。

4. 编写简单的评估脚本，统计评分分布、平均分等指标。

测试数据样例：

```
# 学生答案示例
示例1（满分）：机器的大量使用大大提高了生产效率，城市化进程加快使人口大幅增加，商品供应增加提高了人民生活品质，社会结构变化催生了改革运动，同时工业废料对环境造成了不利影响。

示例2（3分）：工业革命让机器代替了手工劳动，生产效率提高了。很多人从农村搬到城市，人口变多了。但是对环境的破坏也很严重。

示例3（1分）：工业革命让英国变得强大了。
```

进阶挑战（可选）：

- 实现多轮对话评分：让模型先输出分析，再输出最终分数。
- 接入不同的国产模型 API，对比评分一致性和成本。
- 开发简单的 Web 界面，方便非技术用户上传答案文件并查看评分结果。

第十三章　综合实验

13.1　实验目的

1. 使用开发板完成一款实用产品的设计
2. 深入了解一种智能算法，将其应用到系统中

13.2　实验内容

13.2.1　实验题目要求

综合实验要求每位同学利用实验平台的现有资源，完成一款智能设备的原型设计。这款设备可以是任何应用场景的一个功能组件，也可以是完整的解决方案。但必须充分考虑现有平台的计算能力，既不能仅处理简单数据而浪费计算资源，也不要因为项目过于复杂而无法胜任。

这里所谓的“智能”，指的是完成的项目原型具有如下一种或者多种功能：

- 具有一定的“机器视觉”，可以利用摄像头进行数据输入
- 具有一定的“机器听觉”，能根据环境中的声音或者语音命令完成任务
- 具有一定的“机器智能”，可以在尽可能少的人工干预情况下完成工作
- 具有一定的学习能力，可以根据历史的经验改善处理问题的方式
- 具有一定的推理能力，可以在复杂输入条件中做出最正确的判断

13.2.2　实验报告要求

实验报告要求完整、系统的对自己的作品进行介绍，主要内容包括：

- 作品简介，简要描述作品的功能和指标
- 系统方案，介绍作品的基本设计思路及软硬件结构
- 系统实现，包括完整的系统搭建流程、具体的实现细节，以便读者可以重复同一工作
- 系统测试，作品的展示内容和指标等数据
- 附录，包含的程序及参考的开源项目或网站

13.2.3　开发板离线推理介绍

CANN（Compute Architecture for Neural Networks）是华为公司针对人工智能应用场景推出的软件架构，为用户提供了多种编程接口。通过使用底层软件包，用户可便捷迅速地进行基于昇腾设备的产品业务开发。除此之外，用户还可以基于该软件包实现自定义算子、模型转换等功能。随着昇腾设备的更新迭代，CANN 软件包也在不断进行版本升级，但包内的主要工具作用大同小异。在实际使用时要根据开发板的型号选择合适的版本。

ACL（Ascend Computing Language）是指 CANN 中提供的一系列设备管理、内存管理、模型加载与执行、媒体数据处理等 C 语言的 API 库。pyACL（Python Ascend Computing Language）就是在 ACL 的

基础上使用 CPython 封装得到的 python API 库，用户可以通过使用 python API 库对昇腾 AI 处理器进行资源和设备管理工作，在这些封装好的接口库上开发深度神经网络应用，用于实现目标识别、图像分类等功能。

pyACL（Python Ascend Computing Language）是一套在 AscendCL 的基础上使用 CPython 封装得到的 Python API 库，使用户可以通过 Python 进行昇腾 AI 处理器的运行管理、资源管理等，实现在昇腾 CANN 平台上进行深度学习推理计算、图形图像预处理、单算子加速计算等能力。

一般采用 pyTorch 保存的模型参数需要先转换为 onnx 格式，再通过 atc 工具转换为 NPU 可用的 om 格式，最后通过 pyACL 对 npu 资源进行调度，并加载模型进行推理。

使用 pyACL 的推理过程主要代码包括：

```
import acl

# 1. 初始化
acl.init()
device_id = 0
acl.rt.set_device(device_id)
context, stream = acl.rt.create_context(), acl.rt.create_stream()

# 2. 加载模型
model_id = acl.mdl.load_from_file("resnet50.om")
model_desc = acl.mdl.create_desc()
acl.mdl.get_desc(model_desc, model_id)

# 3. 申请输入/输出 device 内存
input_sizes  = [acl.mdl.get_input_size_by_index(model_desc, i) for i in range(
    num_inputs)]
output_sizes = [...]
input_buffers  = [acl.rt.malloc(sz) for sz in input_sizes]
output_buffers = [...]

# 4. 拷贝输入到 device，执行推理
acl.rt.memcpy(input_buffers[0], host_data, size, ACL_MEMCPY_HOST_TO_DEVICE)
acl.mdl.execute(model_id, input_buffers, output_buffers, stream)
acl.rt.synchronize_stream(stream)

# 5. 取结果并后处理
acl.rt.memcpy(host_output, output_buffers[0], size, ACL_MEMCPY_DEVICE_TO_HOST)

# 6. 清理
acl.rt.free(input_buffers[0])
acl.mdl.unload(model_id)
acl.rt.destroy_context(context)
acl.finalize()
```

这个过程比较复杂，对于简单推理，可以采用 ais_bench.infer.interface.InferSession 的高级接口来简化代码：

```
import numpy as np
from ais_bench.infer.interface import InferSession

```

```
# 1. 创建推理会话并加载模型
device_id = 0 # 指定设备号
model_path = "resnet50.om" # 模型路径
session = InferSession(device_id, model_path)

# 2. 准备输入数据 (假设我们已经有一张处理好的图片)
# 注意: 输入数据的shape和dtype必须与模型要求一致
input_data = np.ones([1, 3, 224, 224]).astype(np.float32)

# 3. 执行推理 (一步到位)
# outputs 直接就是一个包含所有输出Tensor的List
outputs = session.infer([input_data])

# 4. 处理输出
# 例如, 获取第一个输出并做argmax得到分类结果
output_array = outputs[0]
predicted_class_id = np.argmax(output_array, axis=1)[0]
print(f"Predicted class id: {predicted_class_id}")

# 5. 显式销毁资源
session.free_resource()
```

对于第 9.5.2.3小节中代码保存的模型参数,可以通过下面的代码转换为 onnx 格式:

```
# export_onnx.py
import torch
import torch.nn as nn

# ------------------ 1. 网络结构保持与训练时一致 ------------------
class Net(nn.Module):
    def __init__(self, num_classes=4):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(1,  6, 5), nn.ReLU(), nn.MaxPool2d(4),
            nn.Conv2d(6, 12, 5), nn.ReLU(), nn.MaxPool2d(4),
            nn.Flatten(),
            nn.Linear(12*3*2, 60), nn.ReLU(),
            nn.Linear(60, 30),     nn.ReLU(),
            nn.Linear(30, num_classes)
        )
    def forward(self, x): return self.net(x)

# ------------------ 2. 载入权重 ------------------
PTH_PATH  = 'runs/model.pth'        # <— 你的 pth
ONNX_PATH = 'runs/model.onnx'       # <— 输出 onnx
INPUT_SHAPE = (1, 1, 60, 80)        # (B,C,H,W) 必须与训练输入一致

device = 'cpu'
model = Net(num_classes=4).to(device)
model.load_state_dict(torch.load(PTH_PATH, map_location=device))
```

```
model.eval()

# ------------------ 3. 导出 ------------------
dummy = torch.randn(INPUT_SHAPE)
torch.onnx.export(
    model,
    dummy,
    ONNX_PATH,
    opset_version=11,                    # ATC 7.0+ 支持 11
    do_constant_folding=True,
    input_names=['input'],
    output_names=['output'],
    dynamic_axes={'input': {0: 'batch'}, 'output': {0: 'batch'}}
)
print('ONNX 已生成: ', ONNX_PATH)
```

生成 OM 格式文件的时候可以参考下面的 atc 命令：

```
$ atc   --model=runs/model.onnx \    # ONNX 模型文件
        --framework=5 \              # 格式编号
        --output=runs/model \        # 生成文件 model.om
        --input_format=NCHW \        # 模型文件格式
        --input_shape="input:1,1,60,80" \
        --soc_version=Ascend310B4    # NPU 型号
```

模式转换的时间会比较长，最好在高性能主机上运行 atc 命令。

最后推理的代码如下：

```
import numpy as np
import cv2
from ais_bench.infer.interface import InferSession
from scipy.special import softmax
import json

# ------------------ 参数 ------------------
OM_PATH    = 'runs/model.om'          # 上一步生成的 om
IMG_PATH   = 'test.png' # 任意待测图片
INPUT_HW   = (60, 80)                 # 模型输入尺寸
# 读 JSON
with open('runs/classes.json', 'r', encoding='utf-8') as f:
    class_to_idx = json.load(f)

class_names = [None] * len(class_to_idx)
for name, idx in class_to_idx.items():
    class_names[idx] = name
# ------------------ 1. 建立 session ------------------
sess = InferSession(device_id=0, model_path=OM_PATH)

# ------------------ 2. 读取并预处理图片 ------------------
img = cv2.imread(IMG_PATH, cv2.IMREAD_GRAYSCALE)      # H,W uint8
img = cv2.resize(img, (INPUT_HW[1], INPUT_HW[0]))     # resize
img = img.astype(np.float32) / 255.0                  # 归一化
```

```
25 img = img[np.newaxis, np.newaxis, ...]                    # -> 1,1,60,80 (NCHW)
26
27 # ------------------ 3. 推理 ------------------
28 prob = softmax(outs[0].squeeze())
29 pred = int(prob.argmax())
30 conf = float(prob[pred])
31
32 # ------------------ 4. 结果 ------------------
33 print(f'预测类别: {class_names[pred]}')
34 print(f'置信度␣␣: {conf:.4f}')
```

13.2.4 参考实验题目

13.2.4.1 对弈机器人

使用摄像头识别棋盘状态，控制机械臂移动棋子，完成对弈机器人的设计。对弈所使用的棋类可以是围棋、五子棋、象棋、国际象棋等。棋盘面积不大于 15cm x 15cm；棋子采用塑料或者纸质，可以通过吸盘吸取并移动。

13.2.4.2 智能移动机器人

采用开发板控制移动底盘，控制方式可以是基于视觉的自主控制，也可以通过声音命令等给与辅助控制，还可以通过激光雷达进行环境及障碍物的识别。底盘可以使用实验室的 Turtlebot，也可以使用采用 Arduino 控制的智能小车。

Turtlebot 的串口通信协议可以参考官方的文档，Arduino 智能小车的通信协议如下：串口波特率 38400，控制命令如下：

A 前进
B 右转前进
C 仅右转
D 右转后退
E 后退
F 左转后退
G 仅左转
H 左转前进
Z 停止
I 关闭/打开电机

还支持发送命令更新部分内部执行参数，格式为{}中的命令：

- {P}，通过串口打印 PID 参数
- {0:xx}，更新基础运行速度
- {1:xx}，更新 PID 的 P 参数
- {2:xx}，更新 PID 的 I 参数

串口会即时反馈小车运行数据，格式为：{A:左轮数据:右轮数据:电池电压:舵机角度1} 和 {B:舵机角度2:电池电压:左轮数据:右轮数据}

13.2.4.3 猜拳机器人

通过摄像头识别手势，并控制机械手掌与用户玩猜拳游戏。机械手掌通过 5 个舵机进行控制，可以实现不同的手势动作。

13.2.4.4 绘画机器人

通过控制机械臂在纸上书写或绘画，实现创意应用产品。例如给用户画像，解答数学题等。

13.3 注意事项

13.3.1 使用 USB 网卡

在一些项目中，开发板无法通过 USB 接口和主机进行连接时可以使用 USB 网卡，通过网络完成连接控制。在插入网卡后（保持原 USB 连接）执行 `nmcli dev wifi list`，如果没有任何输出需要执行下面命令：

```
$ sudo nmcli radio wifi on
```

之后可以运行 `sudo nmtui` 选择 Active a connection 按钮，连接到 WiFi 热点。连接成功后，运行 `ip a` 命令查看 USB 网卡对应的 IPv4 地址并记录下来。

此时使用连接到同样热点的笔记本就可以通过 ssh 命令连接到开发板，假设开发板的 IP 地址是 192.168.31.70，ssh 命令如下：

```
$ ssh HwHiAiUser@192.168.31.70
```

登陆后启动 JupyterLab 的时候注意也要指定新的 IP 地址：

```
$ ./start_jupyterlab.sh 192.168.31.70
```

13.3.2 供电

开发板可以通过 12V 锂电池组来供电，使用之前注意查看锂电池的电压，当电压低于 11V 时必须要先充满电才可以使用。请尽量只在测试时使用电池供电，减少电池电量消耗。

参考文献

[1] *OpenCV: Trackbar as the Color Palette*. URL: https://docs.opencv.org/4.x/d9/dc8/tutorial_py_trackbar.html (visited on 06/23/2022).

[2] *OpenCV: Arithmetic Operations on Images*. URL: https://docs.opencv.org/4.x/d0/d86/tutorial_py_image_arithmetics.html.

[3] *Hough Transform*. In: *Wikipedia*. Feb. 20, 2022. URL: https://en.wikipedia.org/w/index.php?title=Hough_transform&oldid=1073024927.

[4] *OpenCV: Geometric Transformations of Images*. URL: https://docs.opencv.org/4.x/da/d6e/tutorial_py_geometric_transformations.html.

[5] *NumPy Quickstart —NumPy v1.24.Dev0 Manual*. URL: https://numpy.org/devdocs/user/quickstart.html.

[6] M.J. Swain and D.H. Ballard. "Indexing via Color Histograms". In: *[1990] Proceedings Third International Conference on Computer Vision*. [1990] Proceedings Third International Conference on Computer Vision. 1990, pp. 390–393. DOI: 10.1109/ICCV.1990.139558.

[7] *Face Detection: Facial Recognition and Finding Homepage*. Mar. 13, 2015. URL: https://facedetection.com/ (visited on 07/27/2022).

[8] P. Viola and M. Jones. "Rapid Object Detection Using a Boosted Cascade of Simple Features". In: *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition. CVPR 2001*. 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition. CVPR 2001. Vol. 1. Kauai, HI, USA: IEEE Comput. Soc, 2001, pp. I-511-I–518. ISBN: 978-0-7695-1272-3. DOI: 10.1109/CVPR.2001.990517.